SAP DEVELOPMENT STUDY GUIDE

ABAP Objects & OData

From beginner to advanced — concepts, code, practical labs and interview preparation

Part I ABAP Objects (OO ABAP) Part II OData & SAP Gateway / RAP
Part III Interview Questions Part IV Capstone Projects & Cheat Sheets

Contents

How to Use This Guide . . . 7
Suggested study plan (6 weeks) . . . 7
Systems & tools you need . . . 7
Naming conventions used in this guide . . . 8
1. Why Object-Oriented ABAP? . . . 10
Procedural vs. Object-Oriented . . . 10
The four pillars of OOP . . . 10
Key vocabulary . . . 10
2. Classes and Objects . . . 11
Local vs. global classes . . . 11
Anatomy of a class . . . 11
Visibility sections . . . 12
Attributes . . . 12
Methods and their parameters . . . 13
Instance vs. static components . . . 14
Reference variables, me, and object lifetime . . . 15
Global class in SE24 / ADT – step by step . . . 15
3. Constructors . . . 17
Constructor order with inheritance . . . 18
4. Encapsulation . . . 19
Controlling instantiation: CREATE PUBLIC / PROTECTED / PRIVATE . . . 19
FINAL and friendship . . . 19
5. Inheritance . . . 21
Rules of redefinition . . . 22
Abstract classes and methods . . . 22
Abstract class vs. interface (classic interview question) . . . 23
6. Polymorphism and Casting . . . 25
Up-cast (narrowing cast) – always safe . . . 25
Down-cast (widening cast) – checked at runtime . . . 25
Why polymorphism matters – the "no IF/CASE on type" rule . . . 26
7. Interfaces . . . 28
Interface rules you must know . . . 29
Interfaces + polymorphism: dependency injection . . . 29
8. Events . . . 32

9. Class-Based Exceptions . . . 35

The exception class hierarchy . . . 35

TRY / CATCH / CLEANUP / RETRY / RESUME . . . 35

Creating your own exception class . . . 36

Exception chaining with previous . . . 37

Classic vs. class-based – what to remember . . . 37

10. Design Patterns in ABAP . . . 39

Singleton – exactly one instance . . . 39

Factory – centralise object creation . . . 40

Strategy – interchangeable algorithms . . . 40

Observer . . . 41

Decorator . . . 41

Facade . . . 41

MVC (Model–View–Controller) . . . 41

Persistence layer / Repository (DAO) . . . 42

11. ABAP Unit Testing . . . 44

Fixture methods . . . 45

Test attributes . . . 45

Test doubles: isolating dependencies . . . 45

12. Modern ABAP, SALV, RTTS and Real-World OO . . . 48

Modern syntax you will use inside every method (7.40 – 7.58) . . . 48

SALV – the OO way to display an ALV . . . 48

RTTS – Run Time Type Services . . . 49

Dynamic programming with objects . . . 50

BAdIs – OO-based enhancements . . . 50

Clean ABAP & SOLID principles . . . 50

Other advanced OO topics (know the definition) . . . 51

13. REST and OData Fundamentals . . . 54

Why OData? . . . 54

REST in five minutes . . . 54

HTTP status codes you must know . . . 54

OData versions . . . 55

The building blocks of an OData service (EDM) . . . 55

Service document and $metadata . . . 55

A V2 JSON response . . . 56

14. SAP Gateway Architecture . . . 57

Deployment options . . . 57

Request flow . . . 57
Key transactions & tools . . . 57
15. Building Your First Service in SEGW . . . 59
Step-by-step . . . 59
16. Implementing CRUD in the DPC_EXT . . . 60
Method signature essentials . . . 60
GET_ENTITYSET – read a list . . . 60
GET_ENTITY – read one . . . 61
CREATE_ENTITY – POST . . . 61
UPDATE_ENTITY – PUT / PATCH . . . 62
DELETE_ENTITY . . . 62
Error handling in Gateway . . . 62
17. Query Options in Depth . . . 65
$filter operators . . . 65
Reading the filter manually (select-options) . . . 66
18. Associations, Navigation and $expand . . . 67
Modelling a header–item relationship in SEGW . . . 67
Implementing navigation – ORDERITEMSET_GET_ENTITYSET . . . 67
$expand: default behaviour vs. GET_EXPANDED_ENTITYSET . . . 68
19. Deep Insert, $batch, Function Imports, Media and ETags . . . 69
Deep insert – create header and items in one POST . . . 69
$batch – many operations in one HTTP request . . . 70
Function imports (custom operations) . . . 71
Media / streams (files, images, PDFs) . . . 71
ETags – optimistic concurrency . . . 72
Soft state, caching and annotations (brief) . . . 72
20. CDS-Based OData Services . . . 74
Example: CDS view with @OData.publish . . . 74
SEGW with Reference Data Source . . . 75
CDS concepts you need for OData . . . 75
21. RAP – ABAP RESTful Application Programming Model (OData V4) . . . 77
The RAP layers . . . 77
Managed vs. unmanaged . . . 77
Example: Travel booking (managed, simplified) . . . 77
EML cheat sheet . . . 81
SEGW (classic) vs. RAP – interview comparison . . . 81

22. Consuming OData Services . . . 83

From SAPUI5 (Fiori) . . . 83

From Postman / any HTTP client – the CSRF dance . . . 83

From ABAP (consume a remote OData service) . . . 83

OData in SAP integration . . . 84

23. Performance, Security and Troubleshooting . . . 85

Performance checklist . . . 85

Security checklist . . . 85

Troubleshooting guide . . . 85

24. ABAP Objects – Interview Questions . . . 88

Beginner . . . 88

Intermediate . . . 89

Advanced . . . 90

25. OData & SAP Gateway – Interview Questions . . . 93

Beginner . . . 93

Intermediate . . . 94

Advanced . . . 95

Scenario questions (senior level) . . . 97

26. Capstone Project: Leave Request Management . . . 99

Requirements . . . 99

Architecture (layers) . . . 99

Step 1 – Data model (SE11) . . . 99

Step 2 – Interfaces and exception . . . 100

Step 3 – Rules (Strategy pattern) . . . 100

Step 4 – Service class with events . . . 101

Step 5 – Unit tests with test doubles . . . 102

Step 6 – OData service (SEGW) . . . 103

Step 7 – Test script (Gateway Client / Postman) . . . 105

Stretch goals . . . 105

27. Exercise Bank (extra practice) . . . 106

ABAP Objects . . . 106

OData . . . 106

28. Cheat Sheets . . . 107

ABAP Objects syntax . . . 107

OData / Gateway quick reference . . . 107

RAP quick reference . . . 108

Final interview checklist 109

How to Use This Guide

This guide takes you from **zero to advanced** in two of the most important skills of a modern SAP developer: **ABAP Objects (OO ABAP)** and **OData services** (SAP Gateway / SEGW, CDS-based services and RAP). Each chapter follows the same rhythm:

1. **Concept** – what it is and why it exists, in plain words.
2. **Syntax & example** – real ABAP code you can paste into SE38/SE24/ADT (Eclipse).
3. **Watch-outs** – the traps that cost developers hours (and interviewers love to ask).
4. **Practical Lab** – a small task for you to build yourself, followed by a model **Solution**.

At the end you will find **Part III – Interview Questions** (grouped Beginner → Intermediate → Advanced, each with a model answer) and **Part IV – Capstone projects** that combine everything into one realistic scenario, plus cheat sheets for quick revision before an interview.

Suggested study plan (6 weeks)

Week	Topic	Output you should have
1	Ch. 1–4: classes, objects, methods, constructors, encapsulation	3 local classes in a report, all labs done
2	Ch. 5–8: inheritance, polymorphism, interfaces, events	A small class hierarchy + interface-based design
3	Ch. 9–12: exceptions, design patterns, ABAP Unit, modern syntax, SALV	OO report with SALV + unit tests
4	Ch. 13–16: REST/OData basics, Gateway architecture, SEGW, CRUD	Working OData service with GET/POST/PUT/DELETE
5	Ch. 17–20: query options, navigation/$expand, deep insert, $batch, function imports, media	Header–Item service with deep insert
6	Ch. 21–23: CDS-based OData, RAP, consumption, performance; Part III & IV	RAP business object + revise interview questions

Systems & tools you need

- An SAP system with developer access: company sandbox, **SAP BTP ABAP Environment (trial)**, or an **ABAP Platform Developer Edition** (Docker image).
- **SAP GUI** (SE24, SE80, SE38, SEGW, /IWFND/MAINT_SERVICE) and **Eclipse + ABAP Development Tools (ADT)** – mandatory for CDS and RAP.
- An HTTP tool for testing: **/IWFND/GW_CLIENT** (in SAP GUI) and/or **Postman**.

> **Tip: How to get the most from the labs**
>
> Type the code yourself instead of copy-pasting. Then break it on purpose (remove a `RAISING`, change a visibility section, call a private method) and read the syntax error. Understanding *why* the compiler complains is exactly what interviewers check.

Naming conventions used in this guide

Object	Prefix / pattern	Example
Global class / interface	`ZCL_` / `ZIF_`	`ZCL_SALES_ORDER`, `ZIF_PRINTABLE`
Local class / interface	`LCL_` / `LIF_`	`LCL_CALCULATOR`
Exception class	`ZCX_` / `LCX_`	`ZCX_ORDER_NOT_FOUND`
Instance attribute	`mv_` (value), `ms_` (structure), `mt_` (table), `mo_` (object)	`mv_order_id`, `mt_items`
Static attribute	`gv_`, `gt_`, `go_` (or `sv_`)	`go_instance`
Method parameter	`iv_ ev_ cv_ rv_` (+ `s/t/o`)	`iv_matnr`, `rt_items`, `ro_order`
Local variable	`lv_ ls_ lt_ lo_`	`lt_orders`

> **Note: Clean ABAP**
> SAP's official *Clean ABAP* style guide recommends dropping most Hungarian prefixes in new code (e.g. `order_id` instead of `mv_order_id`). Many companies still use prefixes, so this guide keeps them for readability – but know both styles for interviews.



Part I — ABAP Objects

Classes, objects, inheritance, interfaces, events, exceptions, design patterns, unit testing & modern ABAP

1. Why Object-Oriented ABAP?

ABAP started as a **procedural** language: reports, subroutines (`FORM`) and function modules. Since release 4.6 SAP added **ABAP Objects**, and today almost everything new in SAP is built on it: BAdIs, SALV, Gateway/OData (DPC/MPC classes), RAP behavior pools, AMDP, Workflow, ABAP Unit. You **cannot** be an advanced SAP developer without OO.

Procedural vs. Object-Oriented

Aspect	Procedural ABAP	ABAP Objects
Building block	Program, FORM, function module	Class, object, interface
Data	Global data shared by everything	Data encapsulated inside objects
Reuse	Copy code / call FM	Inheritance, interfaces, composition
Multiple instances	Hard (function group = one global state)	Easy – create as many objects as you need
Extensibility	Modify or copy	Subclass / implement interface / BAdI
Testability	Hard	ABAP Unit + test doubles
Stricter syntax	Obsolete statements allowed	Obsolete statements forbidden inside classes

The four pillars of OOP

- **Encapsulation** – an object hides its internal data and exposes only a public interface (methods). Others cannot corrupt its state.
- **Abstraction** – you work with *what* an object does, not *how* (e.g. `lo_printer->print( )`).
- **Inheritance** – a subclass reuses and specialises a superclass (`ZCL_TRUCK` inherits from `ZCL_VEHICLE`).
- **Polymorphism** – one call, many behaviours: the same method call runs different code depending on the real object type at runtime.

Key vocabulary

Term	Meaning
Class	The blueprint / template (definition of attributes + methods)
Object (instance)	A concrete thing created from a class at runtime, lives in memory
Attribute	Data inside a class (variable, constant, structure, table, reference)
Method	Behaviour – a procedure that belongs to the class
Reference variable	A pointer to an object: `DATA lo_car TYPE REF TO lcl_car.`
Instance component	Belongs to each object separately (`DATA`, `METHODS`)
Static component	Belongs to the class itself, shared by all objects (`CLASS-DATA`, `CLASS-METHODS`)
Interface	A contract – list of methods without implementation
Event	A notification raised by one object and handled by others

2. Classes and Objects

Local vs. global classes

- **Local class** – defined inside a program (report, include, function group, or the "Local Types" of a global class). Visible only in that program. Great for learning and for helper classes.
- **Global class** – stored in the class library (**SE24** or **ADT**: *New → ABAP Class*). Usable from any program in the system. Names start with `ZCL_`/`YCL_` for customer objects.

Anatomy of a class

Every class has two parts: the **DEFINITION** (what it has – declarations) and the **IMPLEMENTATION** (how methods work).

```
REPORT zoo_demo_01.

CLASS lcl_bank_account DEFINITION.
  PUBLIC SECTION.
    " visible to everyone
    METHODS constructor IMPORTING iv_owner TYPE string
                                  iv_balance TYPE p DEFAULT 0.
    METHODS deposit     IMPORTING iv_amount TYPE p.
    METHODS withdraw    IMPORTING iv_amount TYPE p.
    METHODS get_balance RETURNING VALUE(rv_balance) TYPE decfloat34.
    CLASS-DATA gv_count TYPE i READ-ONLY.     " static: shared by all accounts

  PROTECTED SECTION.
    " visible to this class and its subclasses

  PRIVATE SECTION.
    " visible only inside this class
    DATA mv_owner   TYPE string.
    DATA mv_balance TYPE decfloat34.
ENDCLASS.

CLASS lcl_bank_account IMPLEMENTATION.
  METHOD constructor.
    mv_owner   = iv_owner.
    mv_balance = iv_balance.
    gv_count  += 1.
  ENDMETHOD.

  METHOD deposit.
    mv_balance = mv_balance + iv_amount.
  ENDMETHOD.

  METHOD withdraw.
    IF iv_amount > mv_balance.
      MESSAGE 'Insufficient funds' TYPE 'I'.
      RETURN.
    ENDIF.
    mv_balance = mv_balance - iv_amount.
  ENDMETHOD.

  METHOD get_balance.
    rv_balance = mv_balance.
  ENDMETHOD.
ENDCLASS.
```

```
START-OF-SELECTION.
  DATA(lo_acc1) = NEW lcl_bank_account( iv_owner = 'Hassan' iv_balance = 1000 ).
  DATA(lo_acc2) = NEW lcl_bank_account( iv_owner = 'Mona' ).

  lo_acc1->deposit( 500 ).
  lo_acc1->withdraw( 200 ).

  WRITE: / 'Balance 1:', lo_acc1->get_balance( ).
  WRITE: / 'Balance 2:', lo_acc2->get_balance( ).
  WRITE: / 'Accounts :', lcl_bank_account=>gv_count.
```

Things to notice:

- `->` accesses **instance** components (via an object reference); `=>` accesses **static** components (via the class name).
- `NEW lcl_bank_account( ... )` creates an object and returns a reference. The older equivalent is `CREATE OBJECT lo_acc1 EXPORTING iv_owner = 'Hassan'.`
- A method with a single non-optional importing parameter can be called with the value only: `lo_acc1->deposit( 500 )`.
- `READ-ONLY` lets outsiders read a public attribute but not change it.

Visibility sections

Section	Accessible from	Typical content
`PUBLIC SECTION`	Everywhere	The API: methods consumers call, constants, types
`PROTECTED SECTION`	The class itself + its subclasses	Helper methods/attributes subclasses need
`PRIVATE SECTION`	Only the class itself (and its *friends*)	Internal state and helpers

> **Watch out: Order of sections**
> In the definition the sections **must** appear in the order PUBLIC → PROTECTED → PRIVATE. Each one is optional.

Attributes

```
CLASS lcl_demo DEFINITION.
  PUBLIC SECTION.
    TYPES: BEGIN OF ty_item,
             matnr TYPE matnr,
             menge TYPE i,
           END OF ty_item,
           tt_items TYPE STANDARD TABLE OF ty_item WITH EMPTY KEY.

    CONSTANTS gc_max_items TYPE i VALUE 100.     " constant (always static)
    DATA      mv_name      TYPE string.          " instance attribute
    DATA      mt_items     TYPE tt_items.        " instance internal table
    DATA      mo_logger    TYPE REF TO lcl_logger. " reference attribute
    CLASS-DATA gv_counter  TYPE i.                " static attribute
ENDCLASS.
```

Methods and their parameters

Parameter kind	Direction	Notes
`IMPORTING`	Caller → method	Read-only inside the method (unless `VALUE( )` is used)
`EXPORTING`	Method → caller	Not initialised automatically when passed by reference
`CHANGING`	Both ways	Caller's value comes in, modified value goes out
`RETURNING VALUE( )`	Method → caller	Exactly one; always pass by value; makes it a **functional method** usable in expressions
`RAISING`	–	Lists class-based exceptions the method may propagate

```
CLASS lcl_calc DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS add
      IMPORTING iv_a          TYPE i
                iv_b          TYPE i
      RETURNING VALUE(rv_sum) TYPE i.

    CLASS-METHODS divide
      IMPORTING iv_a          TYPE decfloat34
                iv_b          TYPE decfloat34
      RETURNING VALUE(rv_res) TYPE decfloat34
      RAISING   cx_sy_zerodivide.

    CLASS-METHODS split_name
      IMPORTING iv_full  TYPE string
      EXPORTING ev_first TYPE string
                ev_last  TYPE string.

    CLASS-METHODS increment
      CHANGING cv_value TYPE i.
ENDCLASS.

CLASS lcl_calc IMPLEMENTATION.
  METHOD add.
    rv_sum = iv_a + iv_b.
  ENDMETHOD.
  METHOD divide.
    rv_res = iv_a / iv_b.            " raises CX_SY_ZERODIVIDE if iv_b = 0
  ENDMETHOD.
  METHOD split_name.
    SPLIT iv_full AT space INTO ev_first ev_last.
  ENDMETHOD.
  METHOD increment.
    cv_value += 1.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  " functional call inside an expression
  DATA(lv_total) = lcl_calc=>add( iv_a = 2 iv_b = 3 ) * 10.

  lcl_calc=>split_name( EXPORTING iv_full  = 'Hassan Diab'
                        IMPORTING ev_first = DATA(lv_first)
                                  ev_last  = DATA(lv_last) ).
  DATA(lv_n) = 5.
  lcl_calc=>increment( CHANGING cv_value = lv_n ).   " lv_n = 6
```

> **Note: Pass by value vs. pass by reference**
> By default IMPORTING/EXPORTING/CHANGING are passed **by reference** (fast, no copy). `VALUE( iv_x )` passes **by value** (a copy). RETURNING is always by value. With EXPORTING by reference the parameter keeps the caller's old value until you assign it – always `CLEAR` exporting tables before filling them.

Optional and default parameters, preferred parameter

```
METHODS log
  IMPORTING iv_text TYPE string
            iv_type TYPE symsgty DEFAULT 'I'
            iv_ctx  TYPE string OPTIONAL.
" inside the method:  IF iv_ctx IS SUPPLIED. ... ENDIF.
```

`PREFERRED PARAMETER` lets you call a method with several optional parameters using a single unnamed value.

Instance vs. static components

```
CLASS lcl_counter DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS get_total RETURNING VALUE(rv) TYPE i.   " static method
    METHODS       hit.                                    " instance method
  PRIVATE SECTION.
    CLASS-DATA gv_total TYPE i.     " one value for the whole class
    DATA       mv_mine  TYPE i.     " one value per object
ENDCLASS.

CLASS lcl_counter IMPLEMENTATION.
  METHOD hit.
    mv_mine  += 1.
    gv_total += 1.                " instance methods may use static attributes
  ENDMETHOD.
  METHOD get_total.
    rv = gv_total.                " static methods can NOT use mv_mine or me
  ENDMETHOD.
ENDCLASS.
```

> **Watch out: Static methods have no `me`**
> A static method is not executed on an object, so it cannot access instance attributes, call instance methods directly, or use the self-reference `me`.

Reference variables, `me`, and object lifetime

- `DATA lo_obj TYPE REF TO lcl_x.` declares a reference – it is **initial** (points to nothing) until you create an object.
- Calling a method on an initial reference raises the runtime error `CX_SY_REF_IS_INITIAL` (dump `OBJECTS_OBJREF_NOT_ASSIGNED`). Check with `IF lo_obj IS BOUND.`
- Assigning one reference to another (`lo_b = lo_a.`) copies the **pointer**, not the object – both now point at the *same* object.
- `me` is the self-reference inside instance methods: `me->mv_name = iv_name.` (needed when a parameter has the same name as an attribute).
- When no reference points to an object any more, the **garbage collector** frees it. `CLEAR lo_obj.` or `FREE lo_obj.` releases your reference.

```
DATA(lo_a) = NEW lcl_counter( ).
DATA(lo_b) = lo_a.            " same object!
lo_b->hit( ).
" lo_a and lo_b both see mv_mine = 1
IF lo_a = lo_b.               " compares references (identity)
  WRITE / 'Same object'.
ENDIF.
```

Global class in SE24 / ADT – step by step

1. ADT: right-click package → *New* → *ABAP Class* → name `ZCL_BANK_ACCOUNT`, description. (SE24: enter name → *Create* → *Usual ABAP class*, *Final* checkbox as needed.)
2. In ADT you write the full DEFINITION/IMPLEMENTATION in source code. In SE24 you use the form-based tabs **Attributes**, **Methods**, **Events**, **Types**, **Friends** – or switch to *Source Code-Based* mode.
3. Activate (**Ctrl+F3**). Test with **F8** (SE24 test environment) or by implementing `IF_OO_ADT_CLASSRUN` in ADT and pressing **F9**:

```
CLASS zcl_hello DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
ENDCLASS.

CLASS zcl_hello IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    out->write( |Hello from ABAP Objects - { sy-uname }| ).
  ENDMETHOD.
ENDCLASS.
```

Practical Lab: Lab 2 – Your first class

Create a local class `LCL_STUDENT` with private attributes name, list of grades (internal table of integers). Public methods: `constructor` (name), `add_grade` (one grade 0–100, ignore invalid values), `get_average` (functional, returns decfloat34), `get_max` (functional). Add a static attribute that counts how many students were created. Create three students in START-OF-SELECTION and print their averages.

Solution: Lab 2

```
CLASS lcl_student DEFINITION.
  PUBLIC SECTION.
    TYPES tt_grades TYPE STANDARD TABLE OF i WITH EMPTY KEY.
    CLASS-DATA gv_created TYPE i READ-ONLY.
    METHODS constructor IMPORTING iv_name TYPE string.
```

```
    METHODS add_grade   IMPORTING iv_grade TYPE i.
    METHODS get_average RETURNING VALUE(rv_avg) TYPE decfloat34.
    METHODS get_max     RETURNING VALUE(rv_max) TYPE i.
    METHODS get_name    RETURNING VALUE(rv_name) TYPE string.
  PRIVATE SECTION.
    DATA mv_name   TYPE string.
    DATA mt_grades TYPE tt_grades.
ENDCLASS.

CLASS lcl_student IMPLEMENTATION.
  METHOD constructor.
    mv_name = iv_name.
    gv_created += 1.
  ENDMETHOD.
  METHOD add_grade.
    CHECK iv_grade BETWEEN 0 AND 100.
    APPEND iv_grade TO mt_grades.
  ENDMETHOD.
  METHOD get_average.
    CHECK mt_grades IS NOT INITIAL.
    rv_avg = REDUCE decfloat34( INIT s = 0 FOR g IN mt_grades NEXT s = s + g )
             / lines( mt_grades ).
  ENDMETHOD.
  METHOD get_max.
    LOOP AT mt_grades INTO DATA(lv_g).
      rv_max = nmax( val1 = rv_max val2 = lv_g ).
    ENDLOOP.
  ENDMETHOD.
  METHOD get_name.
    rv_name = mv_name.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA(lo_s1) = NEW lcl_student( 'Ali' ).
  lo_s1->add_grade( 90 ). lo_s1->add_grade( 70 ). lo_s1->add_grade( 150 ).
  WRITE: / lo_s1->get_name( ), lo_s1->get_average( ), lo_s1->get_max( ).
  WRITE: / 'Students created:', lcl_student=>gv_created.
```

3. Constructors

A **constructor** is a special method called automatically when something is created. ABAP has two kinds.

	Instance constructor	Static constructor
Name	`constructor`	`class_constructor`
Declared with	`METHODS constructor`	`CLASS-METHODS class_constructor`
Called	Once per object, on `NEW` / `CREATE OBJECT`	Once per class per internal session, before the first access to the class
Parameters	IMPORTING only (+ RAISING)	**No** parameters
Section	Public (or protected/private with CREATE PROTECTED/PRIVATE)	Public
Typical use	Initialise object state, validate input	Fill static buffers, read customizing once

```
CLASS lcl_config DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS class_constructor.
    CLASS-DATA gt_plants TYPE STANDARD TABLE OF t001w WITH EMPTY KEY READ-ONLY.
    METHODS constructor IMPORTING iv_werks TYPE werks_d
                        RAISING   cx_abap_invalid_value.
  PRIVATE SECTION.
    DATA mv_werks TYPE werks_d.
ENDCLASS.

CLASS lcl_config IMPLEMENTATION.
  METHOD class_constructor.
    " runs only once - before first use of LCL_CONFIG
    SELECT * FROM t001w INTO TABLE @gt_plants UP TO 100 ROWS.
  ENDMETHOD.

  METHOD constructor.
    IF NOT line_exists( gt_plants[ werks = iv_werks ] ).
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.
    mv_werks = iv_werks.
  ENDMETHOD.
ENDCLASS.
```

Constructor order with inheritance

When you create an object of a subclass:

1. Static constructors run first, **top-down** (superclass, then subclass) – only if they have not run yet in this session.
2. The subclass instance constructor runs; it **must** call `super->constructor( )` before accessing instance components (the compiler enforces this when the superclass has an explicit constructor).

Watch out: Constructor gotchas

- If a constructor raises an exception, the object is **not** created and the reference stays initial (or keeps its old value).
- In the superclass constructor, calls to redefined methods run the **superclass** version (the subclass part is not ready yet).
- Exceptions from `class_constructor` cannot be caught – they cause a runtime error. Keep static constructors simple.

Practical Lab: Lab 3 – Constructor order

Create `LCL_PARENT` and `LCL_CHILD INHERITING FROM lcl_parent`, each with a static and an instance constructor that only `WRITE`s its name. Create two `LCL_CHILD` objects and predict the output before running.

Solution: Lab 3

Output: `Parent static` → `Child static` → `Child instance (before super)` → `Parent instance` → `Child instance (after super)` → then for the second object only the three instance lines. Static constructors run once per internal session.

```
CLASS lcl_parent DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS class_constructor.
    METHODS constructor.
ENDCLASS.
CLASS lcl_parent IMPLEMENTATION.
  METHOD class_constructor. WRITE / 'Parent static'.   ENDMETHOD.
  METHOD constructor.       WRITE / 'Parent instance'. ENDMETHOD.
ENDCLASS.

CLASS lcl_child DEFINITION INHERITING FROM lcl_parent.
  PUBLIC SECTION.
    CLASS-METHODS class_constructor.
    METHODS constructor.
ENDCLASS.
CLASS lcl_child IMPLEMENTATION.
  METHOD class_constructor. WRITE / 'Child static'. ENDMETHOD.
  METHOD constructor.
    WRITE / 'Child instance (before super)'.
    super->constructor( ).
    WRITE / 'Child instance (after super)'.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA(lo1) = NEW lcl_child( ).
  DATA(lo2) = NEW lcl_child( ).
```

4. Encapsulation

Encapsulation = **hide the data, expose behaviour**. Outsiders change an object's state only through methods that can validate the change.

```
CLASS lcl_employee DEFINITION.
  PUBLIC SECTION.
    METHODS set_salary IMPORTING iv_salary TYPE p
                       RAISING   cx_abap_invalid_value.
    METHODS get_salary RETURNING VALUE(rv_salary) TYPE decfloat34.
  PRIVATE SECTION.
    DATA mv_salary TYPE decfloat34.
ENDCLASS.

CLASS lcl_employee IMPLEMENTATION.
  METHOD set_salary.
    IF iv_salary < 0.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.   " validation lives in one place
    ENDIF.
    mv_salary = iv_salary.
  ENDMETHOD.
  METHOD get_salary.
    rv_salary = mv_salary.
  ENDMETHOD.
ENDCLASS.
```

Controlling instantiation: CREATE PUBLIC / PROTECTED / PRIVATE

Addition	Who may create objects
`CREATE PUBLIC` (default)	Anyone
`CREATE PROTECTED`	The class itself and its subclasses
`CREATE PRIVATE`	Only the class itself (and friends) – used for **Singleton** and **Factory** patterns

FINAL and friendship

- `CLASS lcl_x DEFINITION FINAL.` – the class cannot be inherited from. `METHODS m FINAL.` – a method cannot be redefined.
- `CLASS lcl_x DEFINITION FRIENDS lcl_y.` – `LCL_Y` may access protected and private components of `LCL_X` and create its instances even if it is CREATE PRIVATE. Friendship is **one-way** (LCL_X does not get access to LCL_Y). Subclasses of the *friend* also become friends (so grant it carefully), but subclasses of LCL_X do **not** automatically grant friendship to LCL_Y. It is typically used for test classes (`FRIENDS ltc_test`) or tightly coupled helper classes.

> **Tip: Interview phrase**
> "Encapsulation is not about getters and setters for every attribute. It means the object owns its invariants: a bank account never lets its balance go negative, no matter who calls it."

> **Practical Lab: Lab 4 – Protect the invariant**
> Create `LCL_TEMPERATURE` that stores a temperature in Celsius privately. Provide `set_celsius` (reject values below -273.15 with an exception), `get_celsius`, `get_fahrenheit` (F = C × 9/5 + 32). Make the class FINAL and CREATE PRIVATE with a static factory `from_celsius( iv ) RETURNING VALUE(ro)`.

Solution: Lab 4

```
CLASS lcl_temperature DEFINITION FINAL CREATE PRIVATE.
  PUBLIC SECTION.
    CLASS-METHODS from_celsius IMPORTING iv_c       TYPE decfloat34
                               RETURNING VALUE(ro)   TYPE REF TO lcl_temperature
                               RAISING   cx_abap_invalid_value.
    METHODS set_celsius    IMPORTING iv_c TYPE decfloat34
                           RAISING   cx_abap_invalid_value.
    METHODS get_celsius    RETURNING VALUE(rv) TYPE decfloat34.
    METHODS get_fahrenheit RETURNING VALUE(rv) TYPE decfloat34.
  PRIVATE SECTION.
    CONSTANTS gc_abs_zero TYPE decfloat34 VALUE '-273.15'.
    DATA mv_c TYPE decfloat34.
ENDCLASS.

CLASS lcl_temperature IMPLEMENTATION.
  METHOD from_celsius.
    ro = NEW #( ).
    ro->set_celsius( iv_c ).
  ENDMETHOD.
  METHOD set_celsius.
    IF iv_c < gc_abs_zero.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.
    mv_c = iv_c.
  ENDMETHOD.
  METHOD get_celsius.    rv = mv_c.                ENDMETHOD.
  METHOD get_fahrenheit. rv = mv_c * 9 / 5 + 32.   ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  TRY.
      DATA(lo_t) = lcl_temperature=>from_celsius( 25 ).
      WRITE: / lo_t->get_fahrenheit( ).            " 77
      lo_t->set_celsius( -300 ).                   " raises
    CATCH cx_abap_invalid_value.
      WRITE: / 'Invalid temperature rejected'.
  ENDTRY.
```

5. Inheritance

A subclass **inherits all components** of its superclass (public and protected are accessible; private exist but cannot be accessed directly by the subclass). The subclass can **add** new components and **redefine** inherited instance methods. ABAP supports **single inheritance** only (one direct superclass); multiple "inheritance of behaviour contracts" is done with interfaces. The root of all classes is `OBJECT`.

```
CLASS lcl_vehicle DEFINITION.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_plate TYPE string.
    METHODS describe RETURNING VALUE(rv) TYPE string.
    METHODS get_wheels RETURNING VALUE(rv) TYPE i.
  PROTECTED SECTION.
    DATA mv_plate TYPE string.
ENDCLASS.

CLASS lcl_vehicle IMPLEMENTATION.
  METHOD constructor.
    mv_plate = iv_plate.
  ENDMETHOD.
  METHOD describe.
    rv = |Vehicle { mv_plate } with { get_wheels( ) } wheels|.
  ENDMETHOD.
  METHOD get_wheels.
    rv = 4.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_truck DEFINITION INHERITING FROM lcl_vehicle.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_plate TYPE string
                                  iv_load  TYPE i.
    METHODS describe   REDEFINITION.
    METHODS get_wheels REDEFINITION.
  PRIVATE SECTION.
    DATA mv_load TYPE i.
ENDCLASS.

CLASS lcl_truck IMPLEMENTATION.
  METHOD constructor.
    super->constructor( iv_plate ).     " mandatory, first
    mv_load = iv_load.
  ENDMETHOD.
  METHOD describe.
    " reuse the superclass logic and extend it
    rv = super->describe( ) && | and load { mv_load } t|.
  ENDMETHOD.
  METHOD get_wheels.
    rv = 18.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA(lo_truck) = NEW lcl_truck( iv_plate = 'CAI-123' iv_load = 20 ).
  WRITE / lo_truck->describe( ).
  " -> Vehicle CAI-123 with 18 wheels and load 20 t
```

Note: Dynamic binding inside the superclass

> `describe( )` in `LCL_VEHICLE` calls `get_wheels( )`. Because the object is a truck, the **redefined** `get_wheels` of `LCL_TRUCK` runs – even though the call is written in the superclass. This is polymorphism at work (also called *late binding*).

Rules of redefinition

- Only **instance** methods can be redefined (not static methods, not constructors – a subclass declares its *own* constructor).
- The signature cannot change – you only write `METHODS name REDEFINITION.`
- The visibility stays as in the superclass. You cannot redefine private methods.
- `FINAL` methods/classes cannot be redefined/inherited.
- `super->method( )` calls the superclass implementation.

Abstract classes and methods

An **abstract class** cannot be instantiated; it exists to be inherited. An **abstract method** has no implementation in the abstract class; every concrete subclass must implement it.

```
CLASS lcl_shape DEFINITION ABSTRACT.
  PUBLIC SECTION.
    METHODS area     ABSTRACT RETURNING VALUE(rv) TYPE decfloat34.
    METHODS describe RETURNING VALUE(rv) TYPE string.     " concrete method
ENDCLASS.

CLASS lcl_shape IMPLEMENTATION.
  METHOD describe.
    rv = |{ cl_abap_classdescr=>get_class_name( me ) } has area { area( ) }|.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_circle DEFINITION INHERITING FROM lcl_shape.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_r TYPE decfloat34.
    METHODS area REDEFINITION.
  PRIVATE SECTION.
    DATA mv_r TYPE decfloat34.
ENDCLASS.

CLASS lcl_circle IMPLEMENTATION.
  METHOD constructor.
    super->constructor( ).
    mv_r = iv_r.
  ENDMETHOD.
  METHOD area.
    rv = '3.14159' * mv_r * mv_r.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_rect DEFINITION INHERITING FROM lcl_shape.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_w TYPE decfloat34 iv_h TYPE decfloat34.
    METHODS area REDEFINITION.
  PRIVATE SECTION.
    DATA: mv_w TYPE decfloat34, mv_h TYPE decfloat34.
ENDCLASS.

CLASS lcl_rect IMPLEMENTATION.
  METHOD constructor.
```

```
    super->constructor( ).
    mv_w = iv_w. mv_h = iv_h.
  ENDMETHOD.
  METHOD area.
    rv = mv_w * mv_h.
  ENDMETHOD.
ENDCLASS.
```

Abstract class vs. interface (classic interview question)

Abstract class	Interface
Can contain implemented methods **and** abstract ones	Only declarations (no implementation) – except DEFAULT behaviour in newer releases
Can have instance attributes with state, constructors	Attributes are possible but discouraged; no constructor
A class inherits from **one** abstract class	A class can implement **many** interfaces
Models an "is-a" relationship with shared code	Models a "can-do" capability / contract
Has visibility sections	All components are public

Practical Lab: Lab 5 – Employee hierarchy

Build abstract `LCL_EMPLOYEE` (name, abstract method `calculate_salary`, concrete method `print_payslip` that writes name + salary). Subclasses: `LCL_FULLTIME` (monthly salary), `LCL_CONTRACTOR` (hours × rate), `LCL_MANAGER` inheriting from `LCL_FULLTIME` adding a bonus (redefine `calculate_salary` and use `super->`). Put all employees in one internal table of `REF TO lcl_employee` and loop to print payslips.

Solution: Lab 5

```
CLASS lcl_employee DEFINITION ABSTRACT.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_name TYPE string.
    METHODS calculate_salary ABSTRACT RETURNING VALUE(rv) TYPE decfloat34.
    METHODS print_payslip.
  PROTECTED SECTION.
    DATA mv_name TYPE string.
ENDCLASS.
CLASS lcl_employee IMPLEMENTATION.
  METHOD constructor. mv_name = iv_name. ENDMETHOD.
  METHOD print_payslip.
    WRITE: / mv_name, calculate_salary( ).
  ENDMETHOD.
ENDCLASS.

CLASS lcl_fulltime DEFINITION INHERITING FROM lcl_employee.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_name TYPE string iv_monthly TYPE decfloat34.
    METHODS calculate_salary REDEFINITION.
  PROTECTED SECTION.
    DATA mv_monthly TYPE decfloat34.
ENDCLASS.
CLASS lcl_fulltime IMPLEMENTATION.
  METHOD constructor.
    super->constructor( iv_name ).
    mv_monthly = iv_monthly.
  ENDMETHOD.
  METHOD calculate_salary. rv = mv_monthly. ENDMETHOD.
ENDCLASS.

CLASS lcl_contractor DEFINITION INHERITING FROM lcl_employee.
```

```
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_name TYPE string
                                  iv_hours TYPE i iv_rate TYPE decfloat34.
    METHODS calculate_salary REDEFINITION.
  PRIVATE SECTION.
    DATA: mv_hours TYPE i, mv_rate TYPE decfloat34.
ENDCLASS.
CLASS lcl_contractor IMPLEMENTATION.
  METHOD constructor.
    super->constructor( iv_name ).
    mv_hours = iv_hours. mv_rate = iv_rate.
  ENDMETHOD.
  METHOD calculate_salary. rv = mv_hours * mv_rate. ENDMETHOD.
ENDCLASS.

CLASS lcl_manager DEFINITION INHERITING FROM lcl_fulltime.
  PUBLIC SECTION.
    METHODS constructor IMPORTING iv_name TYPE string
                                  iv_monthly TYPE decfloat34 iv_bonus TYPE decfloat34.
    METHODS calculate_salary REDEFINITION.
  PRIVATE SECTION.
    DATA mv_bonus TYPE decfloat34.
ENDCLASS.
CLASS lcl_manager IMPLEMENTATION.
  METHOD constructor.
    super->constructor( iv_name = iv_name iv_monthly = iv_monthly ).
    mv_bonus = iv_bonus.
  ENDMETHOD.
  METHOD calculate_salary.
    rv = super->calculate_salary( ) + mv_bonus.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA lt_staff TYPE STANDARD TABLE OF REF TO lcl_employee WITH EMPTY KEY.
  lt_staff = VALUE #( ( NEW lcl_fulltime( iv_name = 'Ali'  iv_monthly = 10000 ) )
                      ( NEW lcl_contractor( iv_name = 'Sara' iv_hours = 120 iv_rate = 150 ) )
                      ( NEW lcl_manager( iv_name = 'Omar' iv_monthly = 20000 iv_bonus = 5000 )
) ).
  LOOP AT lt_staff INTO DATA(lo_emp).
    lo_emp->print_payslip( ).      " polymorphic call
  ENDLOOP.
```

6. Polymorphism and Casting

Polymorphism means a reference of a general type (superclass or interface) can point to objects of many specific types, and a method call is dispatched to the implementation of the **actual (dynamic) type** at runtime.

- **Static type** – the type used in the declaration: `DATA lo_shape TYPE REF TO lcl_shape.`
- **Dynamic type** – the class of the object the reference points to at runtime (e.g. `LCL_CIRCLE`).

Up-cast (narrowing cast) – always safe

Assign a subclass reference to a superclass (or interface) reference. Allowed with a normal `=`; checked at compile time.

```
DATA lo_shape  TYPE REF TO lcl_shape.
DATA lo_circle TYPE REF TO lcl_circle.

lo_circle = NEW #( iv_r = 2 ).
lo_shape  = lo_circle.              " up-cast: OK
WRITE / lo_shape->area( ).          " runs LCL_CIRCLE->AREA (dynamic binding)
```

Down-cast (widening cast) – checked at runtime

Assign a general reference to a more specific one. You must use `?=` or `CAST #( )`. If the object is not of that type, the exception `CX_SY_MOVE_CAST_ERROR` is raised.

```
TRY.
    lo_circle ?= lo_shape.                    " classic syntax
    lo_circle = CAST lcl_circle( lo_shape ).  " modern syntax (7.40+)
  CATCH cx_sy_move_cast_error.
    WRITE / 'Not a circle'.
ENDTRY.

" Safer: test first (7.50+)
IF lo_shape IS INSTANCE OF lcl_circle.
  lo_circle = CAST #( lo_shape ).
ENDIF.

" Branch by type (7.50+)
CASE TYPE OF lo_shape.
  WHEN TYPE lcl_circle INTO DATA(lo_c).
    WRITE / 'Circle'.
  WHEN TYPE lcl_rect INTO DATA(lo_r).
    WRITE / 'Rectangle'.
  WHEN OTHERS.
    WRITE / 'Unknown shape'.
ENDCASE.
```

> **Watch out: The naming trap**
>
> SAP documentation calls the up-cast a **narrowing cast** and the down-cast a **widening cast** (named after what happens to the set of components you can reach through the reference). In interviews, say "up-cast (narrowing, implicit, always safe)" and "down-cast (widening, explicit with `?=`/`CAST`, can raise `CX_SY_MOVE_CAST_ERROR`)" – then you are right either way.

Why polymorphism matters – the "no IF/CASE on type" rule

Bad (procedural thinking):

```
CASE lv_shape_type.
  WHEN 'C'. lv_area = '3.14' * lv_r * lv_r.
  WHEN 'R'. lv_area = lv_w * lv_h.
  " every new shape = change this code (violates Open/Closed principle)
ENDCASE.
```

Good (OO thinking): every shape knows its own area – adding `LCL_TRIANGLE` needs **no change** to existing code.

```
LOOP AT lt_shapes INTO DATA(lo_shape).
  lv_total = lv_total + lo_shape->area( ).
ENDLOOP.
```

Practical Lab: Lab 6 – Payment methods

Abstract class `LCL_PAYMENT` with abstract method `pay( iv_amount ) RETURNING VALUE(rv_msg)`. Subclasses `LCL_CASH`, `LCL_CARD` (adds 2 % fee), `LCL_WALLET` (has an extra public method `get_cashback( )` returning 1 % of the last amount). Loop over a table of payments, call `pay( 1000 )` for each, and – only for wallets – also print the cashback using a down-cast.

Solution: Lab 6

```
CLASS lcl_payment DEFINITION ABSTRACT.
  PUBLIC SECTION.
    METHODS pay ABSTRACT IMPORTING iv_amount TYPE decfloat34
                         RETURNING VALUE(rv_msg) TYPE string.
ENDCLASS.

CLASS lcl_cash DEFINITION INHERITING FROM lcl_payment.
  PUBLIC SECTION. METHODS pay REDEFINITION.
ENDCLASS.
CLASS lcl_cash IMPLEMENTATION.
  METHOD pay. rv_msg = |Cash paid: { iv_amount }|. ENDMETHOD.
ENDCLASS.

CLASS lcl_card DEFINITION INHERITING FROM lcl_payment.
  PUBLIC SECTION. METHODS pay REDEFINITION.
ENDCLASS.
CLASS lcl_card IMPLEMENTATION.
  METHOD pay. rv_msg = |Card charged: { iv_amount * '1.02' }|. ENDMETHOD.
ENDCLASS.

CLASS lcl_wallet DEFINITION INHERITING FROM lcl_payment.
  PUBLIC SECTION.
    METHODS pay REDEFINITION.
    METHODS get_cashback RETURNING VALUE(rv) TYPE decfloat34.
  PRIVATE SECTION.
    DATA mv_last TYPE decfloat34.
ENDCLASS.
CLASS lcl_wallet IMPLEMENTATION.
  METHOD pay.
    mv_last = iv_amount.
    rv_msg = |Wallet paid: { iv_amount }|.
  ENDMETHOD.
  METHOD get_cashback. rv = mv_last / 100. ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA lt_pay TYPE STANDARD TABLE OF REF TO lcl_payment WITH EMPTY KEY.
```

```
lt_pay = VALUE #( ( NEW lcl_cash( ) ) ( NEW lcl_card( ) ) ( NEW lcl_wallet( ) ) ).
LOOP AT lt_pay INTO DATA(lo_pay).
  WRITE / lo_pay->pay( 1000 ).
  IF lo_pay IS INSTANCE OF lcl_wallet.
    WRITE: / '  cashback:', CAST lcl_wallet( lo_pay )->get_cashback( ).
  ENDIF.
ENDLOOP.
```

7. Interfaces

An **interface** is a pure contract: a named list of method signatures (plus optionally types, constants, attributes and events) **without implementation**. Any class that says `INTERFACES zif_x` promises to implement all its methods. Interfaces give you:

- **Multiple "inheritance" of contracts** – a class may implement many interfaces.
- **Loose coupling** – code depends on `REF TO zif_x`, not on a concrete class, so you can swap implementations (e.g. a real DB access class vs. a test double).
- The basis of **BAdIs**, SAP standard frameworks (`IF_HTTP_EXTENSION`, `IF_SALV_*`, `IF_OO_ADT_CLASSRUN`) and RAP.

```
INTERFACE lif_printable.
  CONSTANTS gc_width TYPE i VALUE 80.
  METHODS print RETURNING VALUE(rv_text) TYPE string.
ENDINTERFACE.

INTERFACE lif_exportable.
  METHODS to_json RETURNING VALUE(rv_json) TYPE string.
ENDINTERFACE.

CLASS lcl_invoice DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_printable, lif_exportable.
    ALIASES print FOR lif_printable~print.      " shorter name
    METHODS constructor IMPORTING iv_no TYPE string iv_amount TYPE decfloat34.
  PRIVATE SECTION.
    DATA mv_no     TYPE string.
    DATA mv_amount TYPE decfloat34.
ENDCLASS.

CLASS lcl_invoice IMPLEMENTATION.
  METHOD constructor.
    mv_no = iv_no. mv_amount = iv_amount.
  ENDMETHOD.
  METHOD lif_printable~print.                   " interface method: intf~method
    rv_text = |Invoice { mv_no }: { mv_amount }|.
  ENDMETHOD.
  METHOD lif_exportable~to_json.
    rv_json = |\{"no":"{ mv_no }","amount":{ mv_amount }\}|.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA lo_p TYPE REF TO lif_printable.
  lo_p = NEW lcl_invoice( iv_no = 'INV-1' iv_amount = 500 ).   " up-cast to interface
  WRITE / lo_p->print( ).

  DATA(lo_inv) = NEW lcl_invoice( iv_no = 'INV-2' iv_amount = 99 ).
  WRITE / lo_inv->print( ).                       " via alias
  WRITE / lo_inv->lif_exportable~to_json( ).      " full name
  WRITE / lif_printable=>gc_width.                " interface constant
```

Interface rules you must know

- All interface components are **public**; the implementing class must declare `INTERFACES` in its **PUBLIC SECTION**.
- Components are addressed with the **interface component selector** `~`: `lif_printable~print`.
- You **cannot instantiate** an interface, but you can declare reference variables of it.
- An interface can include other interfaces (**nested / composite interface**): `INTERFACE lif_doc. INTERFACES lif_printable. ENDINTERFACE.`
- When a class implements an interface, all its **subclasses** implement it too (and can redefine its methods: `METHODS lif_printable~print REDEFINITION.`).
- Partial implementation: `INTERFACES lif_x ABSTRACT METHODS m1` (in an abstract class) or `ALL METHODS ABSTRACT`; in test doubles you can use `PARTIALLY IMPLEMENTED` (only in test classes).
- Newer releases allow `METHODS m DEFAULT IGNORE` / `DEFAULT FAIL` in an interface, so implementing classes may skip optional methods.

Interfaces + polymorphism: dependency injection

```
INTERFACE lif_notifier.
  METHODS send IMPORTING iv_to TYPE string iv_text TYPE string.
ENDINTERFACE.

CLASS lcl_email_notifier DEFINITION.
  PUBLIC SECTION. INTERFACES lif_notifier.
ENDCLASS.
CLASS lcl_email_notifier IMPLEMENTATION.
  METHOD lif_notifier~send.
    WRITE: / 'EMAIL to', iv_to, ':', iv_text.  " real code: CL_BCS
  ENDMETHOD.
ENDCLASS.

CLASS lcl_sms_notifier DEFINITION.
  PUBLIC SECTION. INTERFACES lif_notifier.
ENDCLASS.
CLASS lcl_sms_notifier IMPLEMENTATION.
  METHOD lif_notifier~send.
    WRITE: / 'SMS to', iv_to, ':', iv_text.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_order_service DEFINITION.
  PUBLIC SECTION.
    " the dependency is INJECTED through the constructor
    METHODS constructor IMPORTING io_notifier TYPE REF TO lif_notifier.
    METHODS release_order IMPORTING iv_order TYPE vbeln.
  PRIVATE SECTION.
    DATA mo_notifier TYPE REF TO lif_notifier.
ENDCLASS.
CLASS lcl_order_service IMPLEMENTATION.
  METHOD constructor.
    mo_notifier = io_notifier.
  ENDMETHOD.
  METHOD release_order.
    " ... business logic ...
    mo_notifier->send( iv_to = 'customer' iv_text = |Order { iv_order } released| ).
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
```

```
  NEW lcl_order_service( NEW lcl_sms_notifier( ) )->release_order( '0000004711' ).
```

`LCL_ORDER_SERVICE` does not know whether it sends e-mail or SMS – and in a unit test we can inject a fake notifier. This is the core idea behind testable OO code.

Practical Lab: Lab 7 – Storage contract

Define `LIF_STORAGE` with methods `save( iv_key iv_value )`, `load( iv_key ) RETURNING rv_value` and `count( ) RETURNING rv`. Implement it twice: `LCL_MEMORY_STORAGE` (hashed internal table) and `LCL_LOGGING_STORAGE` that *wraps* another `LIF_STORAGE` and writes a log line before delegating (this is the **Decorator** pattern). Use both through a `REF TO lif_storage`.

Solution: Lab 7

```
INTERFACE lif_storage.
  METHODS save  IMPORTING iv_key TYPE string iv_value TYPE string.
  METHODS load  IMPORTING iv_key TYPE string RETURNING VALUE(rv_value) TYPE string.
  METHODS count RETURNING VALUE(rv) TYPE i.
ENDINTERFACE.

CLASS lcl_memory_storage DEFINITION.
  PUBLIC SECTION. INTERFACES lif_storage.
  PRIVATE SECTION.
    TYPES: BEGIN OF ty_kv, key TYPE string, value TYPE string, END OF ty_kv.
    DATA mt_data TYPE HASHED TABLE OF ty_kv WITH UNIQUE KEY key.
ENDCLASS.
CLASS lcl_memory_storage IMPLEMENTATION.
  METHOD lif_storage~save.
    INSERT VALUE #( key = iv_key value = iv_value ) INTO TABLE mt_data.
    IF sy-subrc <> 0.                       " key exists -> update
      mt_data[ key = iv_key ]-value = iv_value.
    ENDIF.
  ENDMETHOD.
  METHOD lif_storage~load.
    rv_value = VALUE #( mt_data[ key = iv_key ]-value OPTIONAL ).
  ENDMETHOD.
  METHOD lif_storage~count.
    rv = lines( mt_data ).
  ENDMETHOD.
ENDCLASS.

CLASS lcl_logging_storage DEFINITION.
  PUBLIC SECTION.
    INTERFACES lif_storage.
    METHODS constructor IMPORTING io_inner TYPE REF TO lif_storage.
  PRIVATE SECTION.
    DATA mo_inner TYPE REF TO lif_storage.
ENDCLASS.
CLASS lcl_logging_storage IMPLEMENTATION.
  METHOD constructor. mo_inner = io_inner. ENDMETHOD.
  METHOD lif_storage~save.
    WRITE: / 'LOG save', iv_key.
    mo_inner->save( iv_key = iv_key iv_value = iv_value ).
  ENDMETHOD.
  METHOD lif_storage~load.
    WRITE: / 'LOG load', iv_key.
    rv_value = mo_inner->load( iv_key ).
  ENDMETHOD.
  METHOD lif_storage~count.
    rv = mo_inner->count( ).
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA(lo_store) = CAST lif_storage(
```

```
                    NEW lcl_logging_storage( NEW lcl_memory_storage( ) ) ).
lo_store->save( iv_key = 'A' iv_value = 'Apple' ).
lo_store->save( iv_key = 'A' iv_value = 'Avocado' ).
WRITE: / lo_store->load( 'A' ), lo_store->count( ).   " Avocado 1
```

8. Events

Events let an object **announce** that something happened without knowing who is interested (publish/subscribe – the **Observer** pattern). Four steps:

1. **Declare** the event in the publishing class: `EVENTS` (instance) or `CLASS-EVENTS` (static). Events may only have `EXPORTING` parameters, always passed by value. Every instance event automatically has the implicit parameter `sender` (reference to the raising object).
2. **Raise** it in a method of the publisher: `RAISE EVENT evt EXPORTING p = v.`
3. **Declare a handler method** in the subscriber: `METHODS on_evt FOR EVENT evt OF lcl_publisher IMPORTING p sender.`
4. **Register** the handler at runtime: `SET HANDLER lo_sub->on_evt FOR lo_pub.` (or `FOR ALL INSTANCES`; static events need no `FOR`). Deregister with `ACTIVATION abap_false`.

```
CLASS lcl_stock DEFINITION.
  PUBLIC SECTION.
    EVENTS low_stock EXPORTING VALUE(ev_matnr) TYPE matnr
                               VALUE(ev_qty)   TYPE i.
    METHODS constructor IMPORTING iv_matnr TYPE matnr iv_qty TYPE i.
    METHODS withdraw    IMPORTING iv_qty TYPE i.
  PRIVATE SECTION.
    CONSTANTS gc_min TYPE i VALUE 10.
    DATA: mv_matnr TYPE matnr, mv_qty TYPE i.
ENDCLASS.

CLASS lcl_stock IMPLEMENTATION.
  METHOD constructor.
    mv_matnr = iv_matnr. mv_qty = iv_qty.
  ENDMETHOD.
  METHOD withdraw.
    mv_qty -= iv_qty.
    IF mv_qty < gc_min.
      RAISE EVENT low_stock EXPORTING ev_matnr = mv_matnr ev_qty = mv_qty.
    ENDIF.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_purchasing DEFINITION.
  PUBLIC SECTION.
    METHODS on_low_stock FOR EVENT low_stock OF lcl_stock
      IMPORTING ev_matnr ev_qty sender.
ENDCLASS.

CLASS lcl_purchasing IMPLEMENTATION.
  METHOD on_low_stock.
    WRITE: / 'Create purchase requisition for', ev_matnr, 'qty left', ev_qty.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_alert DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS on_low_stock FOR EVENT low_stock OF lcl_stock
      IMPORTING ev_matnr.
ENDCLASS.

CLASS lcl_alert IMPLEMENTATION.
  METHOD on_low_stock.
    WRITE: / 'ALERT: low stock of', ev_matnr.
  ENDMETHOD.
ENDCLASS.
```

```
START-OF-SELECTION.
  DATA(lo_stock) = NEW lcl_stock( iv_matnr = 'MAT-01' iv_qty = 15 ).
  DATA(lo_purch) = NEW lcl_purchasing( ).

  SET HANDLER lo_purch->on_low_stock FOR lo_stock.     " one publisher
  SET HANDLER lcl_alert=>on_low_stock FOR ALL INSTANCES. " every stock object

  lo_stock->withdraw( 3 ).    " 12 - nothing happens
  lo_stock->withdraw( 5 ).    " 7  - both handlers run
```

Note: Where you meet events in real projects

ALV grids (`CL_GUI_ALV_GRID` events `double_click`, `toolbar`, `user_command`, `hotspot_click`), `CL_SALV_TABLE` (`get_event( )` → `added_function`, `link_click`), Workflow (`BOR`/class events with `IF_WORKFLOW`), and custom frameworks.

Watch out: Event pitfalls

- Handlers run **synchronously** in registration order – the raising method continues only after all handlers finish.
- If no handler is registered, `RAISE EVENT` simply does nothing.
- Registered handlers keep the handler object alive (it is referenced by the registration) – deregister when no longer needed.

Practical Lab: Lab 8 – Order status observer

Create `LCL_ORDER` with method `set_status( iv_status )` that raises event `status_changed` with old and new status. Register two observers: `LCL_AUDIT_LOG` (writes "old → new") and `LCL_CUSTOMER_MAIL` that only reacts when new status = 'SHIPPED'. Then deregister the mail handler and change the status again.

Solution: Lab 8

```
CLASS lcl_order DEFINITION.
  PUBLIC SECTION.
    EVENTS status_changed EXPORTING VALUE(ev_old) TYPE string
                                    VALUE(ev_new) TYPE string.
    METHODS set_status IMPORTING iv_status TYPE string.
  PRIVATE SECTION.
    DATA mv_status TYPE string VALUE 'NEW'.
ENDCLASS.
CLASS lcl_order IMPLEMENTATION.
  METHOD set_status.
    DATA(lv_old) = mv_status.
    mv_status = iv_status.
    RAISE EVENT status_changed EXPORTING ev_old = lv_old ev_new = iv_status.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_audit_log DEFINITION.
  PUBLIC SECTION.
    METHODS on_change FOR EVENT status_changed OF lcl_order IMPORTING ev_old ev_new.
ENDCLASS.
CLASS lcl_audit_log IMPLEMENTATION.
  METHOD on_change. WRITE: / |AUDIT: { ev_old } -> { ev_new }|. ENDMETHOD.
ENDCLASS.

CLASS lcl_customer_mail DEFINITION.
  PUBLIC SECTION.
    METHODS on_change FOR EVENT status_changed OF lcl_order IMPORTING ev_new.
```

```
ENDCLASS.
CLASS lcl_customer_mail IMPLEMENTATION.
  METHOD on_change.
    CHECK ev_new = 'SHIPPED'.
    WRITE: / 'MAIL: your order has been shipped'.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  DATA(lo_order) = NEW lcl_order( ).
  DATA(lo_audit) = NEW lcl_audit_log( ).
  DATA(lo_mail)  = NEW lcl_customer_mail( ).
  SET HANDLER: lo_audit->on_change FOR lo_order,
               lo_mail->on_change  FOR lo_order.
  lo_order->set_status( 'PACKED' ).
  lo_order->set_status( 'SHIPPED' ).
  SET HANDLER lo_mail->on_change FOR lo_order ACTIVATION abap_false.
  lo_order->set_status( 'DELIVERED' ).   " only audit reacts
```

9. Class-Based Exceptions

Modern ABAP handles errors with **exception classes** (`TRY ... CATCH ... ENDTRY`) instead of `sy-subrc` and classic `EXCEPTIONS`. An exception is an **object** that carries information (text, attributes, previous exception).

The exception class hierarchy

Superclass	Checked by	Must be declared in RAISING?	Use for
`CX_STATIC_CHECK`	Compiler (syntax check warns)	Yes	Expected business errors the caller must handle (e.g. order not found)
`CX_DYNAMIC_CHECK`	Runtime	Only checked at runtime if not handled	Errors the caller can prevent with a check (e.g. `CX_SY_ZERODIVIDE`, invalid parameter)
`CX_NO_CHECK`	Never	Not allowed/needed (implicitly propagated)	Fatal / technical errors that can occur anywhere (e.g. out of memory)

All three inherit from `CX_ROOT`. Useful methods of `CX_ROOT`: `get_text( )`, `get_longtext( )`, `get_source_position( )`, attribute `previous`.

TRY / CATCH / CLEANUP / RETRY / RESUME

```
TRY.
    DATA(lv_res) = lcl_calc=>divide( iv_a = 10 iv_b = 0 ).
  CATCH cx_sy_zerodivide INTO DATA(lx_zero).
    WRITE / lx_zero->get_text( ).
  CATCH cx_sy_arithmetic_error cx_sy_conversion_error INTO DATA(lx_other).
    " several classes in one CATCH; order matters: specific first!
    WRITE / lx_other->get_text( ).
  CLEANUP.
    " runs when the exception is NOT caught here but propagates upwards
    " (use it to release locks, close files, restore state)
ENDTRY.
```

- **CATCH order matters** – catch subclasses before superclasses, otherwise the specific handler is unreachable (syntax warning).
- `RETRY` (inside CATCH) restarts the TRY block – useful after fixing the cause (be careful with endless loops).
- `RESUME` (inside CATCH) continues after the `RAISE RESUMABLE EXCEPTION` statement – only for resumable exceptions declared `RAISING RESUMABLE(cx_...)`.
- `CATCH BEFORE UNWIND` keeps the context of the raising point alive while the handler runs (needed for RESUME).

Creating your own exception class

In SE24/ADT create `ZCX_ORDER_ERROR` inheriting from `CX_STATIC_CHECK` and implement interface `IF_T100_MESSAGE` (texts from a message class in SE91) – or `IF_T100_DYN_MSG` to use the `MESSAGE` addition of `RAISE EXCEPTION`.

```
CLASS zcx_order_error DEFINITION PUBLIC INHERITING FROM cx_static_check
  CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_t100_message.     " or IF_T100_DYN_MSG for RAISE ... MESSAGE

    CONSTANTS:
      BEGIN OF order_not_found,          " text ID -> message ZSD 001
        msgid TYPE symsgid VALUE 'ZSD',
        msgno TYPE symsgno VALUE '001',
        attr1 TYPE scx_attrname VALUE 'MV_VBELN',
        attr2 TYPE scx_attrname VALUE '',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF order_not_found,
      BEGIN OF order_locked,             " message ZSD 002: Order &1 locked by &2
        msgid TYPE symsgid VALUE 'ZSD',
        msgno TYPE symsgno VALUE '002',
        attr1 TYPE scx_attrname VALUE 'MV_VBELN',
        attr2 TYPE scx_attrname VALUE 'MV_USER',
        attr3 TYPE scx_attrname VALUE '',
        attr4 TYPE scx_attrname VALUE '',
      END OF order_locked.

    DATA mv_vbeln TYPE vbeln READ-ONLY.
    DATA mv_user  TYPE syuname READ-ONLY.

    METHODS constructor
      IMPORTING textid   LIKE if_t100_message=>t100key OPTIONAL
                previous LIKE previous OPTIONAL
                mv_vbeln TYPE vbeln OPTIONAL
                mv_user  TYPE syuname OPTIONAL.
ENDCLASS.

CLASS zcx_order_error IMPLEMENTATION.
  METHOD constructor ##ADT_SUPPRESS_GENERATION.
    super->constructor( previous = previous ).
    me->mv_vbeln = mv_vbeln.
    me->mv_user  = mv_user.
    CLEAR me->textid.
    IF textid IS INITIAL.
      if_t100_message~t100key = if_t100_message=>default_textid.
    ELSE.
      if_t100_message~t100key = textid.
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

Raising and handling it:

```
METHOD get_order.     " ... RAISING zcx_order_error
  SELECT SINGLE * FROM vbak WHERE vbeln = @iv_vbeln INTO @rs_order.
  IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE zcx_order_error
      EXPORTING textid   = zcx_order_error=>order_not_found
                mv_vbeln = iv_vbeln.
    " 7.52+: RAISE EXCEPTION NEW zcx_order_error( textid = ... mv_vbeln = ... ).
```

```
  ENDIF.
ENDMETHOD.

" caller
TRY.
    DATA(ls_order) = lo_repo->get_order( '0000000001' ).
  CATCH zcx_order_error INTO DATA(lx).
    MESSAGE lx TYPE 'I' DISPLAY LIKE 'E'.     " exception object as message
ENDTRY.
```

Exception chaining with `previous`

Wrap low-level technical exceptions into meaningful business ones without losing the root cause:

```
TRY.
    lo_db->insert_order( ls_order ).
  CATCH cx_sy_open_sql_db INTO DATA(lx_sql).
    RAISE EXCEPTION TYPE zcx_order_error
      EXPORTING textid = zcx_order_error=>order_not_found
                previous = lx_sql.          " keep the original
ENDTRY.
```

Classic vs. class-based – what to remember

Classic (`EXCEPTIONS` / `sy-subrc`)	Class-based (`RAISING` / `TRY`)
Only a number in `sy-subrc`	An object with text, attributes, chain
Must be checked right after the call	Propagates automatically up the call stack
Function modules, old methods	All new code; mandatory in RAP/Gateway
`MESSAGE ... RAISING`	`RAISE EXCEPTION TYPE` / `NEW`

Watch out: You cannot mix both in one method signature

A method uses either `EXCEPTIONS` (classic) or `RAISING` (class-based), never both. To call a classic FM from OO code, check `sy-subrc` and convert it into an exception.

Practical Lab: Lab 9 – Safe division service

Create exception class `LCX_CALC_ERROR` (inherit `CX_STATIC_CHECK`, attribute `mv_reason` and redefine `get_text` to return it). Class `LCL_SAFE_CALC=>divide` raises it on division by zero, wrapping the original `CX_SY_ZERODIVIDE` as `previous`. In the caller, print both the business text and the previous exception's text, and use `CLEANUP` in an intermediate method to write "cleanup done".

Solution: Lab 9

```
CLASS lcx_calc_error DEFINITION INHERITING FROM cx_static_check.
  PUBLIC SECTION.
    DATA mv_reason TYPE string READ-ONLY.
    METHODS constructor IMPORTING iv_reason TYPE string
                                  previous  LIKE previous OPTIONAL.
    METHODS get_text REDEFINITION.
ENDCLASS.
CLASS lcx_calc_error IMPLEMENTATION.
  METHOD constructor.
    super->constructor( previous = previous ).
    mv_reason = iv_reason.
```

```
  ENDMETHOD.
  METHOD get_text. result = mv_reason. ENDMETHOD.
ENDCLASS.

CLASS lcl_safe_calc DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS divide IMPORTING iv_a TYPE decfloat34 iv_b TYPE decfloat34
                         RETURNING VALUE(rv) TYPE decfloat34
                         RAISING lcx_calc_error.
    CLASS-METHODS process RAISING lcx_calc_error.
ENDCLASS.
CLASS lcl_safe_calc IMPLEMENTATION.
  METHOD divide.
    TRY.
        rv = iv_a / iv_b.
      CATCH cx_sy_zerodivide INTO DATA(lx).
        RAISE EXCEPTION NEW lcx_calc_error( iv_reason = 'Divisor must not be 0'
                                            previous  = lx ).
    ENDTRY.
  ENDMETHOD.
  METHOD process.
    TRY.
        divide( iv_a = 1 iv_b = 0 ).
      CLEANUP.
        WRITE / 'cleanup done'.
    ENDTRY.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  TRY.
      lcl_safe_calc=>process( ).
    CATCH lcx_calc_error INTO DATA(lx_calc).
      WRITE: / lx_calc->get_text( ),
             / 'Root cause:', lx_calc->previous->get_text( ).
  ENDTRY.
```

10. Design Patterns in ABAP

Design patterns are proven solutions to recurring design problems. Interviewers for mid/senior roles almost always ask "Which patterns have you used in ABAP?" – know at least Singleton, Factory, Strategy, Observer, Decorator, MVC and Facade, with an example from SAP.

Singleton – exactly one instance

Use for: a central logger, an application buffer, a configuration reader.

```
CLASS lcl_logger DEFINITION FINAL CREATE PRIVATE.
  PUBLIC SECTION.
    CLASS-METHODS get_instance RETURNING VALUE(ro) TYPE REF TO lcl_logger.
    METHODS add  IMPORTING iv_msg TYPE string.
    METHODS show.
  PRIVATE SECTION.
    CLASS-DATA go_instance TYPE REF TO lcl_logger.
    DATA mt_log TYPE string_table.
ENDCLASS.

CLASS lcl_logger IMPLEMENTATION.
  METHOD get_instance.
    IF go_instance IS NOT BOUND.          " lazy creation
      go_instance = NEW #( ).
    ENDIF.
    ro = go_instance.
  ENDMETHOD.
  METHOD add.
    APPEND |{ sy-uzeit TIME = ISO } { iv_msg }| TO mt_log.
  ENDMETHOD.
  METHOD show.
    LOOP AT mt_log INTO DATA(lv_line).
      WRITE / lv_line.
    ENDLOOP.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  lcl_logger=>get_instance( )->add( 'Program started' ).
  lcl_logger=>get_instance( )->add( 'Data selected' ).   " same object
  lcl_logger=>get_instance( )->show( ).
```

> **Note: Singleton scope in ABAP**
>
> A singleton lives per **internal session** (one program run), not system-wide. For cross-session sharing use shared memory objects (`CL_SHM_AREA`) or the database. Real SAP example: `CL_ABAP_TSTMP`, many `*_FACTORY=>get_instance` methods, BAL application log wrappers.

Factory – centralise object creation

The caller asks for "a document handler for type X" and gets the right subclass without knowing it. Combined with `CREATE PRIVATE`/`FRIENDS`, only the factory can create objects.

```
INTERFACE lif_doc_handler.
  METHODS process IMPORTING iv_id TYPE string.
ENDINTERFACE.

CLASS lcl_invoice_handler DEFINITION.
  PUBLIC SECTION. INTERFACES lif_doc_handler.
ENDCLASS.
CLASS lcl_invoice_handler IMPLEMENTATION.
  METHOD lif_doc_handler~process. WRITE: / 'Posting invoice', iv_id. ENDMETHOD.
ENDCLASS.

CLASS lcl_credit_handler DEFINITION.
  PUBLIC SECTION. INTERFACES lif_doc_handler.
ENDCLASS.
CLASS lcl_credit_handler IMPLEMENTATION.
  METHOD lif_doc_handler~process. WRITE: / 'Posting credit memo', iv_id. ENDMETHOD.
ENDCLASS.

CLASS lcl_handler_factory DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS create IMPORTING iv_type   TYPE string
                         RETURNING VALUE(ro) TYPE REF TO lif_doc_handler
                         RAISING   cx_abap_invalid_value.
ENDCLASS.
CLASS lcl_handler_factory IMPLEMENTATION.
  METHOD create.
    ro = SWITCH #( iv_type
                   WHEN 'INV' THEN NEW lcl_invoice_handler( )
                   WHEN 'CRM' THEN NEW lcl_credit_handler( )
                   ELSE THROW cx_abap_invalid_value( ) ).
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  lcl_handler_factory=>create( 'INV' )->process( '9000001' ).
```

A more flexible variant reads the class name from a customizing table and uses **dynamic instantiation**: `CREATE OBJECT ro TYPE (lv_class_name).` – new document types need only a new class + a table entry (Open/Closed principle).

Strategy – interchangeable algorithms

Define a family of algorithms behind one interface and choose at runtime (e.g. different pricing or tax calculations per country).

```
INTERFACE lif_tax_strategy.
  METHODS calc IMPORTING iv_net TYPE decfloat34 RETURNING VALUE(rv_tax) TYPE decfloat34.
ENDINTERFACE.

CLASS lcl_tax_eg DEFINITION.
  PUBLIC SECTION. INTERFACES lif_tax_strategy.
ENDCLASS.
CLASS lcl_tax_eg IMPLEMENTATION.
  METHOD lif_tax_strategy~calc. rv_tax = iv_net * '0.14'. ENDMETHOD.   " Egypt VAT 14 %
ENDCLASS.
```

```
CLASS lcl_tax_sa DEFINITION.
  PUBLIC SECTION. INTERFACES lif_tax_strategy.
ENDCLASS.
CLASS lcl_tax_sa IMPLEMENTATION.
  METHOD lif_tax_strategy~calc. rv_tax = iv_net * '0.15'. ENDMETHOD.   " KSA VAT 15 %
ENDCLASS.

CLASS lcl_invoice_calc DEFINITION.
  PUBLIC SECTION.
    METHODS constructor IMPORTING io_tax TYPE REF TO lif_tax_strategy.
    METHODS gross IMPORTING iv_net TYPE decfloat34 RETURNING VALUE(rv) TYPE decfloat34.
  PRIVATE SECTION.
    DATA mo_tax TYPE REF TO lif_tax_strategy.
ENDCLASS.
CLASS lcl_invoice_calc IMPLEMENTATION.
  METHOD constructor. mo_tax = io_tax. ENDMETHOD.
  METHOD gross. rv = iv_net + mo_tax->calc( iv_net ). ENDMETHOD.
ENDCLASS.
```

Observer

Implemented natively with ABAP **events** (Chapter 8): the subject raises an event; observers register with `SET HANDLER`.

Decorator

Wrap an object implementing an interface with another object implementing the same interface to add behaviour (logging, caching, authorization checks) – see Lab 7 (`LCL_LOGGING_STORAGE`).

Facade

A single simple class in front of a complex subsystem: e.g. `ZCL_SALES_ORDER_API=>create( )` hides BAPI_SALESORDER_CREATEFROMDAT2, BAPI_TRANSACTION_COMMIT, message conversion and locking.

MVC (Model–View–Controller)

Split a report into: **Model** (data selection and business logic class), **View** (SALV/ALV display class), **Controller** (reacts to user actions, coordinates). This is how SAP structures Web Dynpro and BSP, and the idea behind Fiori (UI5 view ↔ OData model).

Persistence layer / Repository (DAO)

All database access for an entity goes through one class (`ZCL_ORDER_REPOSITORY` with `read`, `query`, `save`). Business logic never contains `SELECT` statements directly → easier testing (replace the repository with a test double) and reuse.

Practical Lab: Lab 10 – Discount engine

Use Strategy + Factory: interface `LIF_DISCOUNT` with `apply( iv_amount ) RETURNING rv`. Implementations: `LCL_NO_DISCOUNT`, `LCL_PERCENT_DISCOUNT` (constructor takes percent), `LCL_VIP_DISCOUNT` (15 % but at most 500). A factory method `for_customer( iv_category )` returns the right strategy for categories '', 'REGULAR' (5 %) and 'VIP'. Calculate the final price of 5000 for each category.

Solution: Lab 10

```
INTERFACE lif_discount.
  METHODS apply IMPORTING iv_amount TYPE decfloat34 RETURNING VALUE(rv) TYPE decfloat34.
ENDINTERFACE.

CLASS lcl_no_discount DEFINITION.
  PUBLIC SECTION. INTERFACES lif_discount.
ENDCLASS.
CLASS lcl_no_discount IMPLEMENTATION.
  METHOD lif_discount~apply. rv = iv_amount. ENDMETHOD.
ENDCLASS.

CLASS lcl_percent_discount DEFINITION.
  PUBLIC SECTION.
    INTERFACES lif_discount.
    METHODS constructor IMPORTING iv_percent TYPE decfloat34.
  PRIVATE SECTION.
    DATA mv_percent TYPE decfloat34.
ENDCLASS.
CLASS lcl_percent_discount IMPLEMENTATION.
  METHOD constructor. mv_percent = iv_percent. ENDMETHOD.
  METHOD lif_discount~apply. rv = iv_amount * ( 1 - mv_percent / 100 ). ENDMETHOD.
ENDCLASS.

CLASS lcl_vip_discount DEFINITION.
  PUBLIC SECTION. INTERFACES lif_discount.
ENDCLASS.
CLASS lcl_vip_discount IMPLEMENTATION.
  METHOD lif_discount~apply.
    rv = iv_amount - nmin( val1 = iv_amount * '0.15' val2 = 500 ).
  ENDMETHOD.
ENDCLASS.

CLASS lcl_discount_factory DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS for_customer IMPORTING iv_category TYPE string
                               RETURNING VALUE(ro) TYPE REF TO lif_discount.
ENDCLASS.
CLASS lcl_discount_factory IMPLEMENTATION.
  METHOD for_customer.
    ro = SWITCH #( iv_category
                   WHEN 'REGULAR' THEN NEW lcl_percent_discount( 5 )
                   WHEN 'VIP'     THEN NEW lcl_vip_discount( )
                   ELSE                NEW lcl_no_discount( ) ).
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  LOOP AT VALUE string_table( ( `` ) ( `REGULAR` ) ( `VIP` ) ) INTO DATA(lv_cat).
    WRITE: / lv_cat, lcl_discount_factory=>for_customer( lv_cat )->apply( 5000 ).
  ENDLOOP.
```

```
  " -> 5000 / 4750 / 4500
```

11. ABAP Unit Testing

ABAP Unit is the built-in xUnit framework. Test classes are **local classes** with the addition `FOR TESTING`, written in the *Test Classes* include of a global class (ADT tab "Test Classes") or at the end of a report. Run with **Ctrl+Shift+F10** in ADT or *Execute → Unit Test* in SE80/SE24. Code Inspector / ATC can enforce unit tests.

```
" ---------- code under test (global class ZCL_PRICE_CALC) ----------
CLASS zcl_price_calc DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS net_to_gross IMPORTING iv_net TYPE decfloat34
                                   iv_vat TYPE decfloat34
                         RETURNING VALUE(rv) TYPE decfloat34
                         RAISING   cx_abap_invalid_value.
ENDCLASS.
CLASS zcl_price_calc IMPLEMENTATION.
  METHOD net_to_gross.
    IF iv_net < 0 OR iv_vat < 0.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.
    rv = iv_net * ( 1 + iv_vat / 100 ).
  ENDMETHOD.
ENDCLASS.

" ---------- Test Classes include ----------
CLASS ltc_price_calc DEFINITION FINAL FOR TESTING
  DURATION SHORT
  RISK LEVEL HARMLESS.

  PRIVATE SECTION.
    DATA mo_cut TYPE REF TO zcl_price_calc.     " class under test
    METHODS setup.
    METHODS gross_with_14_percent FOR TESTING RAISING cx_static_check.
    METHODS zero_vat_returns_net  FOR TESTING RAISING cx_static_check.
    METHODS negative_net_raises   FOR TESTING.
ENDCLASS.

CLASS ltc_price_calc IMPLEMENTATION.
  METHOD setup.                 " runs before EACH test method
    mo_cut = NEW #( ).
  ENDMETHOD.

  METHOD gross_with_14_percent.
    " given / when / then
    DATA(lv_act) = mo_cut->net_to_gross( iv_net = 100 iv_vat = 14 ).
    cl_abap_unit_assert=>assert_equals( act = lv_act exp = 114
                                        msg = 'VAT 14% not applied' ).
  ENDMETHOD.

  METHOD zero_vat_returns_net.
    cl_abap_unit_assert=>assert_equals(
      act = mo_cut->net_to_gross( iv_net = 50 iv_vat = 0 ) exp = 50 ).
  ENDMETHOD.

  METHOD negative_net_raises.
    TRY.
        mo_cut->net_to_gross( iv_net = -1 iv_vat = 14 ).
        cl_abap_unit_assert=>fail( msg = 'Exception expected' ).
      CATCH cx_abap_invalid_value.
        " expected - test passes
    ENDTRY.
  ENDMETHOD.
```

```
ENDCLASS.
```

Fixture methods

Method	When it runs
`class_setup` (static)	Once before the first test method of the class
`setup`	Before **each** test method
`teardown`	After each test method
`class_teardown` (static)	Once after the last test method

Test attributes

- `RISK LEVEL HARMLESS | DANGEROUS | CRITICAL` – what the test may change (harmless = no persistent changes).
- `DURATION SHORT | MEDIUM | LONG` – expected runtime (< 1 s / < 10 s / longer by default settings).
- Main assertions: `assert_equals`, `assert_differs`, `assert_true`, `assert_false`, `assert_bound`, `assert_not_bound`, `assert_initial`, `assert_not_initial`, `assert_subrc`, `assert_char_cp`, `assert_table_contains`, `fail`.

Test doubles: isolating dependencies

Unit tests must not depend on database content or external systems. Techniques, from best to "last resort":

1. **Dependency injection via interface** – pass a fake implementation in the constructor (see Chapter 7).
2. **ABAP Test Double Framework** – generate doubles of interfaces at runtime: `cl_abap_testdouble=>create( 'ZIF_ORDER_REPO' )` and configure return values.
3. **SQL / CDS Test Double Framework** – replace DB tables or CDS views with test data: `cl_osql_test_environment=>create( VALUE #( ( 'VBAK' ) ) )`, `cl_cds_test_environment`.
4. **Test seams** (`TEST-SEAM` / `TEST-INJECTION`) – replace code blocks in legacy code you cannot refactor.

```
" Fake repository for tests - implements the same interface
CLASS ltd_order_repo DEFINITION FOR TESTING.
  PUBLIC SECTION.
    INTERFACES zif_order_repo PARTIALLY IMPLEMENTED.
    DATA mt_orders TYPE zif_order_repo=>tt_orders.
ENDCLASS.
CLASS ltd_order_repo IMPLEMENTATION.
  METHOD zif_order_repo~get_open_orders.
    rt_orders = mt_orders.            " deterministic data, no DB access
  ENDMETHOD.
ENDCLASS.

" in the test class
METHOD sums_open_orders.
  DATA(lo_repo) = NEW ltd_order_repo( ).
  lo_repo->mt_orders = VALUE #( ( vbeln = '1' netwr = 100 )
                                ( vbeln = '2' netwr = 250 ) ).
  DATA(lo_cut) = NEW zcl_order_stats( io_repo = lo_repo ).
  cl_abap_unit_assert=>assert_equals( act = lo_cut->total_open( ) exp = 350 ).
ENDMETHOD.
```

```
" SQL test double: SELECTs on VBAK inside the code under test read this data
CLASS ltc_with_db DEFINITION FOR TESTING DURATION SHORT RISK LEVEL HARMLESS.
```

```
  PRIVATE SECTION.
    CLASS-DATA go_env TYPE REF TO if_osql_test_environment.
    CLASS-METHODS class_setup.
    CLASS-METHODS class_teardown.
    METHODS setup.
    METHODS reads_orders FOR TESTING.
ENDCLASS.
CLASS ltc_with_db IMPLEMENTATION.
  METHOD class_setup.
    go_env = cl_osql_test_environment=>create( i_dependency_list = VALUE #( ( 'VBAK' ) ) ).
  ENDMETHOD.
  METHOD setup.
    go_env->clear_doubles( ).
    DATA lt_vbak TYPE STANDARD TABLE OF vbak WITH EMPTY KEY.
    lt_vbak = VALUE #( ( vbeln = '0000000001' netwr = 10 ) ).
    go_env->insert_test_data( lt_vbak ).
  ENDMETHOD.
  METHOD reads_orders.
    " ... call code that selects from VBAK and assert ...
  ENDMETHOD.
  METHOD class_teardown.
    go_env->destroy( ).
  ENDMETHOD.
ENDCLASS.
```

Tip: TDD in one sentence

Red (write a failing test) → Green (write the minimum code to pass) → Refactor (clean up while tests stay green). Mention it in interviews and show you write tests for business logic classes, not for UI code.

Practical Lab: Lab 11 – Test your Lab 10 discount engine

Write test class `LTC_DISCOUNT` with methods: regular customer gets 5 %, VIP discount is capped at 500 for 10 000, unknown category returns the full amount, and the factory returns an object of type `LCL_VIP_DISCOUNT` for 'VIP' (use `IS INSTANCE OF` + `assert_true`).

Solution: Lab 11

```
CLASS ltc_discount DEFINITION FINAL FOR TESTING DURATION SHORT RISK LEVEL HARMLESS.
  PRIVATE SECTION.
    METHODS regular_gets_5_percent FOR TESTING.
    METHODS vip_is_capped          FOR TESTING.
    METHODS unknown_pays_full      FOR TESTING.
    METHODS factory_returns_vip    FOR TESTING.
ENDCLASS.

CLASS ltc_discount IMPLEMENTATION.
  METHOD regular_gets_5_percent.
    cl_abap_unit_assert=>assert_equals(
      act = lcl_discount_factory=>for_customer( 'REGULAR' )->apply( 1000 ) exp = 950 ).
  ENDMETHOD.
  METHOD vip_is_capped.
    cl_abap_unit_assert=>assert_equals(
      act = lcl_discount_factory=>for_customer( 'VIP' )->apply( 10000 ) exp = 9500 ).
  ENDMETHOD.
  METHOD unknown_pays_full.
    cl_abap_unit_assert=>assert_equals(
      act = lcl_discount_factory=>for_customer( 'XYZ' )->apply( 700 ) exp = 700 ).
  ENDMETHOD.
  METHOD factory_returns_vip.
    DATA(lo) = lcl_discount_factory=>for_customer( 'VIP' ).
    cl_abap_unit_assert=>assert_true( xsdbool( lo IS INSTANCE OF lcl_vip_discount ) ).
  ENDMETHOD.
```

```
ENDCLASS.
```

12. Modern ABAP, SALV, RTTS and Real-World OO

Modern syntax you will use inside every method (7.40 – 7.58)

Feature	Example
Inline declaration	`DATA(lv_x) = 5.` / `SELECT ... INTO TABLE @DATA(lt_tab).` / `LOOP AT lt INTO DATA(ls).`
Constructor `NEW`	`DATA(lo) = NEW zcl_x( iv = 1 ).`
`VALUE`	`lt = VALUE #( ( id = 1 ) ( id = 2 ) ).`
`CORRESPONDING`	`ls_b = CORRESPONDING #( ls_a MAPPING kunnr = customer EXCEPT mandt ).`
`CONV` / `CAST` / `REF`	`CONV string( lv_num )`, `CAST zif_x( lo )`, `REF #( ls )`
`COND` / `SWITCH`	`COND #( WHEN a > 0 THEN 'P' ELSE 'N' )`
`REDUCE` / `FOR`	`REDUCE i( INIT s = 0 FOR w IN lt NEXT s = s + w-qty )`
`FILTER`	`FILTER #( lt_sorted WHERE status = 'A' )` (needs sorted/hashed key)
Table expressions	`lt[ id = 5 ]-name`, `line_exists( lt[ id = 5 ] )`, `line_index( ... )`
String templates	` `
`xsdbool`	`DATA(lv_ok) = xsdbool( lv_a > lv_b ).`
New Open SQL	`SELECT FROM vbak FIELDS vbeln, netwr WHERE erdat >= @lv_d INTO TABLE @DATA(lt).`
GROUP BY loop	`LOOP AT lt INTO DATA(ls) GROUP BY ls-kunnr INTO DATA(grp).`

SALV – the OO way to display an ALV

```
CLASS lcl_report DEFINITION.
  PUBLIC SECTION.
    METHODS run.
  PRIVATE SECTION.
    TYPES tt_vbak TYPE STANDARD TABLE OF vbak WITH EMPTY KEY.
    DATA mt_data TYPE tt_vbak.
    DATA mo_alv  TYPE REF TO cl_salv_table.
    METHODS select_data.
    METHODS display.
    METHODS on_double_click FOR EVENT double_click OF cl_salv_events_table
      IMPORTING row column.
ENDCLASS.

CLASS lcl_report IMPLEMENTATION.
  METHOD run.
    select_data( ).
    display( ).
  ENDMETHOD.

  METHOD select_data.
    SELECT * FROM vbak UP TO 200 ROWS INTO TABLE @mt_data.
  ENDMETHOD.
```

```
  METHOD display.
    TRY.
        cl_salv_table=>factory( IMPORTING r_salv_table = mo_alv
                                CHANGING  t_table      = mt_data ).
        mo_alv->get_functions( )->set_all( abap_true ).          " toolbar
        mo_alv->get_columns( )->set_optimize( abap_true ).
        mo_alv->get_columns( )->get_column( 'MANDT' )->set_visible( abap_false ).
        mo_alv->get_display_settings( )->set_striped_pattern( abap_true ).
        SET HANDLER on_double_click FOR mo_alv->get_event( ).
        mo_alv->display( ).
      CATCH cx_salv_msg cx_salv_not_found INTO DATA(lx).
        MESSAGE lx TYPE 'I'.
    ENDTRY.
  ENDMETHOD.

  METHOD on_double_click.
    DATA(ls_row) = mt_data[ row ].
    SET PARAMETER ID 'AUN' FIELD ls_row-vbeln.
    CALL TRANSACTION 'VA03' AND SKIP FIRST SCREEN.
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  NEW lcl_report( )->run( ).
```

RTTS – Run Time Type Services

RTTS lets you inspect (RTTI) and create (RTTC) types at runtime – essential for generic tools, dynamic ALV, file uploads, and OData frameworks.

```
" RTTI - describe a structure
DATA(lo_struct) = CAST cl_abap_structdescr(
                    cl_abap_typedescr=>describe_by_name( 'VBAK' ) ).
LOOP AT lo_struct->components INTO DATA(ls_comp).
  WRITE: / ls_comp-name, ls_comp-type_kind, ls_comp-length.
ENDLOOP.

" class name of an object at runtime
DATA(lv_class) = cl_abap_classdescr=>get_class_name( lo_shape ).

" RTTC - build an internal table type dynamically and create data
DATA(lo_table_type) = cl_abap_tabledescr=>create( p_line_type = lo_struct ).
DATA lr_data TYPE REF TO data.
CREATE DATA lr_data TYPE HANDLE lo_table_type.
ASSIGN lr_data->* TO FIELD-SYMBOL(<lt_any>).
SELECT * FROM vbak UP TO 10 ROWS INTO TABLE @<lt_any>.
```

Dynamic programming with objects

```
DATA lo_obj TYPE REF TO object.
DATA(lv_cls)  = `ZCL_INVOICE_HANDLER`.
DATA(lv_meth) = `PROCESS`.
CREATE OBJECT lo_obj TYPE (lv_cls).
CALL METHOD lo_obj->(lv_meth) EXPORTING iv_id = '1001'.

" with a parameter table
DATA(lv_id) = CONV string( '1001' ).
DATA(lt_params) = VALUE abap_parmbind_tab(
  ( name = 'IV_ID' kind = cl_abap_objectdescr=>exporting value = REF #( lv_id ) ) ).
CALL METHOD lo_obj->(lv_meth) PARAMETER-TABLE lt_params.
```

BAdIs – OO-based enhancements

A **BAdI** (Business Add-In) is SAP's object-oriented enhancement technique: SAP defines a **BAdI interface** (`IF_EX_*` / `IF_BADI_*`) and calls it at a specific point; you create an **implementation class** that implements the interface.

- **Classic BAdI**: SE18 (definition) / SE19 (implementation); called with `CL_EXITHANDLER=>get_instance`.
- **New (kernel) BAdI**: part of an *Enhancement Spot*; called with `GET BADI lo_badi FILTERS ...` and `CALL BADI lo_badi->method`. Faster, supports filters and fallback classes.
- Find BAdIs: put a breakpoint in `CL_EXITHANDLER=>GET_INSTANCE` (classic) or search for `GET BADI` / `CALL BADI` in the program, or use SE84 / the enhancement spot.

```
" Your implementation class (created via SE19)
CLASS zcl_im_sd_check_order DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_ex_badi_sd_sales.        " example interface name
ENDCLASS.
```

Clean ABAP & SOLID principles

Principle	In ABAP terms
Single responsibility	One class = one reason to change (repository ≠ calculator ≠ ALV view)
Open/closed	Add new behaviour with new classes (strategy/factory), not by editing CASE statements
Liskov substitution	A subclass must work anywhere the superclass is expected (don't throw "not supported" in redefinitions)
Interface segregation	Several small interfaces > one huge interface
Dependency inversion	Depend on interfaces (`REF TO zif_repo`), inject implementations

Clean ABAP rules worth quoting: prefer `RETURNING` over `EXPORTING`; keep methods short (< 20 statements); prefer class-based exceptions; no `FORM` routines in new code; prefer `NEW` over `CREATE OBJECT`; use `IS BOUND` / `IS NOT INITIAL` intentionally; avoid static methods for business logic (hard to test – no polymorphism).

Other advanced OO topics (know the definition)

- **Persistent classes** (Object Services) – classes mapped to DB tables, generated via SE24 *Persistence*; managed by `CA_*`/`CB_*` agent classes. Rare in new projects.
- **Shared memory objects** – `CL_SHM_AREA`, area classes created in SHMA; share data between sessions on one application server.
- **AMDP** – ABAP Managed Database Procedures: a class implementing `IF_AMDP_MARKER_HDB`, methods `BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT` – code pushdown to HANA.
- **Local types in global classes** – *Class-relevant local types* (definitions include) and *Local types* (implementation include); test classes in the *Test classes* include.
- **Garbage collection & weak references** – `CL_ABAP_WEAK_REFERENCE` holds an object without keeping it alive.

Practical Lab: Lab 12 – OO sales report with SALV

Build a report `ZOO_SALES_REPORT` with select-options for sales org (`VKORG`) and creation date (`ERDAT`). Structure: `LCL_MODEL` (selects VBAK fields VBELN, ERDAT, KUNNR, NETWR, WAERK and computes the total per customer with `GROUP BY` or `REDUCE`), `LCL_VIEW` (SALV with total line and double-click to VA03), `LCL_CONTROLLER` (connects them). No global data except the selection screen.

Solution: Lab 12

```
REPORT zoo_sales_report.

TABLES vbak.
SELECT-OPTIONS: s_vkorg FOR vbak-vkorg,
                s_erdat FOR vbak-erdat.

CLASS lcl_model DEFINITION.
  PUBLIC SECTION.
    TYPES: BEGIN OF ty_order,
             vbeln TYPE vbak-vbeln,
             erdat TYPE vbak-erdat,
             kunnr TYPE vbak-kunnr,
             netwr TYPE vbak-netwr,
             waerk TYPE vbak-waerk,
           END OF ty_order,
           tt_orders TYPE STANDARD TABLE OF ty_order WITH EMPTY KEY.
    METHODS constructor IMPORTING it_vkorg LIKE s_vkorg[] it_erdat LIKE s_erdat[].
    METHODS get_orders RETURNING VALUE(rt) TYPE tt_orders.
  PRIVATE SECTION.
    DATA: mt_vkorg LIKE s_vkorg[], mt_erdat LIKE s_erdat[].
ENDCLASS.

CLASS lcl_model IMPLEMENTATION.
  METHOD constructor.
    mt_vkorg = it_vkorg. mt_erdat = it_erdat.
  ENDMETHOD.
  METHOD get_orders.
    SELECT FROM vbak
      FIELDS vbeln, erdat, kunnr, netwr, waerk
      WHERE vkorg IN @mt_vkorg AND erdat IN @mt_erdat
      ORDER BY kunnr, vbeln
      INTO TABLE @rt
      UP TO 1000 ROWS.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_view DEFINITION.
  PUBLIC SECTION.
    METHODS show CHANGING ct_data TYPE lcl_model=>tt_orders.
  PRIVATE SECTION.
```

```
    DATA mr_data TYPE REF TO lcl_model=>tt_orders.
    METHODS on_dbl FOR EVENT double_click OF cl_salv_events_table IMPORTING row.
ENDCLASS.

CLASS lcl_view IMPLEMENTATION.
  METHOD show.
    mr_data = REF #( ct_data ).
    TRY.
        cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv)
                                CHANGING  t_table      = ct_data ).
        lo_alv->get_functions( )->set_all( ).
        lo_alv->get_columns( )->set_optimize( ).
        DATA(lo_aggr) = lo_alv->get_aggregations( ).
        lo_aggr->add_aggregation( columnname  = 'NETWR'
                                  aggregation = if_salv_c_aggregation=>total ).
        DATA(lo_sort) = lo_alv->get_sorts( ).
        lo_sort->add_sort( columnname = 'KUNNR' subtotal = abap_true ).
        SET HANDLER on_dbl FOR lo_alv->get_event( ).
        lo_alv->display( ).
      CATCH cx_salv_error INTO DATA(lx).
        MESSAGE lx TYPE 'I'.
    ENDTRY.
  ENDMETHOD.
  METHOD on_dbl.
    SET PARAMETER ID 'AUN' FIELD mr_data->*[ row ]-vbeln.
    CALL TRANSACTION 'VA03' AND SKIP FIRST SCREEN.
  ENDMETHOD.
ENDCLASS.

CLASS lcl_controller DEFINITION.
  PUBLIC SECTION.
    METHODS run.
ENDCLASS.
CLASS lcl_controller IMPLEMENTATION.
  METHOD run.
    DATA(lt_orders) = NEW lcl_model( it_vkorg = s_vkorg[] it_erdat = s_erdat[]
                                  )->get_orders( ).
    IF lt_orders IS INITIAL.
      MESSAGE 'No orders found' TYPE 'S' DISPLAY LIKE 'W'.
      RETURN.
    ENDIF.
    NEW lcl_view( )->show( CHANGING ct_data = lt_orders ).
  ENDMETHOD.
ENDCLASS.

START-OF-SELECTION.
  NEW lcl_controller( )->run( ).
```

Part II — OData & SAP Gateway

REST & OData protocol, Gateway architecture, SEGW, CRUD,
query options, deep insert, $batch, CDS-based services & RAP

13. REST and OData Fundamentals

Why OData?

SAP Fiori apps (SAPUI5), mobile apps, SAP BTP extensions and third-party systems all need to read and change SAP data over HTTP. **OData (Open Data Protocol)** is the OASIS standard SAP chose for this: a **REST-based** protocol with a uniform way to describe data (metadata), query it (URL options) and change it (HTTP verbs). It is the backbone of Fiori.

REST in five minutes

REST (*Representational State Transfer*) is an architectural style:

- Everything is a **resource** identified by a **URI**: `/SalesOrderSet('4711')`.
- You act on resources using standard **HTTP methods**.
- Communication is **stateless** – every request carries everything needed.
- Representations are exchanged as **JSON** or **XML (Atom)**.

HTTP method	CRUD	OData example	SEGW (DPC) method
`GET` (collection)	Read list	`GET /OrderSet`	`*_GET_ENTITYSET`
`GET` (single)	Read one	`GET /OrderSet('4711')`	`*_GET_ENTITY`
`POST`	Create	`POST /OrderSet` + body	`*_CREATE_ENTITY`
`PUT`	Replace / update	`PUT /OrderSet('4711')` + body	`*_UPDATE_ENTITY`
`PATCH` / `MERGE`	Partial update	`PATCH /OrderSet('4711')`	`*_UPDATE_ENTITY` (or PATCH_ENTITY)
`DELETE`	Delete	`DELETE /OrderSet('4711')`	`*_DELETE_ENTITY`

HTTP status codes you must know

Code	Meaning	Typical OData situation
200 OK	Success with body	GET, also PUT in some versions
201 Created	Entity created	POST successful
204 No Content	Success, no body	PUT / PATCH / DELETE successful
400 Bad Request	Wrong request / business error	Validation failed (`/IWBEP/CX_MGW_BUSI_EXCEPTION`)
401 Unauthorized	Not logged on	Missing/wrong credentials
403 Forbidden	No authorization / CSRF failed	Missing `S_SERVICE` role or CSRF token
404 Not Found	Resource/service not found	Wrong key, service not activated
405 Method Not Allowed	Operation not implemented	e.g. DELETE not redefined
412 Precondition Failed	ETag mismatch	Someone else changed the entity
428 Precondition Required	ETag missing	`If-Match` header required
500 Internal Server Error	Technical error / dump	`/IWBEP/CX_MGW_TECH_EXCEPTION`, short dump
501 Not Implemented	Method not implemented	Default DPC method raised "not implemented"

OData versions

	OData V2	OData V4
Status	Most widespread in SAP (SEGW, Fiori elements V2)	Current standard (RAP, Fiori elements V4, CAP)
Build tool in SAP	SEGW, CDS `@OData.publish`, RAP binding V2	RAP service binding V4 (SEGW V4 exists but rare)
Count	`$inlinecount=allpages`, `/$count`	`$count=true`, `/$count`
Partial update	`MERGE` / `PATCH`	`PATCH`
Operations	Function imports	Actions (POST, can change data) & functions (GET, read-only)
Payload	JSON verbose (`d`, `results`, `__metadata`)	Leaner JSON (`value`, `@odata.context`)
Search	SAP extension `search=`	Standard `$search`
Expand options	`$expand=Items`	`$expand=Items($select=..;$filter=..)` nested options

The building blocks of an OData service (EDM)

The **Entity Data Model (EDM)** describes the service:

- **Entity Type** – a structure with properties and a key (like a DB table definition): `SalesOrder` with `OrderId`, `Customer`, `NetAmount`.
- **Property** – a field with an EDM type (`Edm.String`, `Edm.Decimal`, `Edm.DateTime` (V2) / `Edm.Date` (V4), `Edm.Boolean`, `Edm.Guid`, `Edm.Int32` ...).
- **Key** – one or more properties identifying an entity.
- **Entity Set** – a collection of entities of one type – what you call in the URL: `SalesOrderSet`.
- **Association** (V2) – a relationship between two entity types with cardinality (1:1, 1:n, n:m).
- **Navigation Property** – a named path through an association: `SalesOrder/ToItems`.
- **Complex Type** – a structure without a key, used as a property type or function return.
- **Function Import** (V2) / **Action & Function** (V4) – custom operations that are not plain CRUD: `ReleaseOrder`.

Service document and $metadata

```
GET /sap/opu/odata/sap/ZSALES_SRV/              -> service document (list of entity sets)
GET /sap/opu/odata/sap/ZSALES_SRV/$metadata     -> full EDM in XML (CSDL)
GET /sap/opu/odata/sap/ZSALES_SRV/OrderSet?$format=json
GET /sap/opu/odata/sap/ZSALES_SRV/OrderSet('4711')/ToItems
```

Excerpt of `$metadata` (V2):

```
<EntityType Name="Order" sap:content-version="1">
  <Key><PropertyRef Name="OrderId"/></Key>
  <Property Name="OrderId"   Type="Edm.String"  Nullable="false" MaxLength="10"/>
  <Property Name="Customer"  Type="Edm.String"  MaxLength="10"/>
  <Property Name="NetAmount" Type="Edm.Decimal" Precision="15" Scale="2"/>
  <Property Name="Currency"  Type="Edm.String"  MaxLength="5" sap:semantics="currency-code"/>
  <NavigationProperty Name="ToItems" Relationship="ZSALES_SRV.Order_Items"
```

```
                      FromRole="FromRole_Order" ToRole="ToRole_Items"/>
</EntityType>
<EntitySet Name="OrderSet" EntityType="ZSALES_SRV.Order" sap:creatable="true"
           sap:updatable="true" sap:deletable="true" sap:pageable="true"/>
```

A V2 JSON response

```
{
  "d": {
    "results": [
      { "__metadata": { "uri": ".../OrderSet('4711')", "type": "ZSALES_SRV.Order" },
        "OrderId": "4711", "Customer": "100001",
        "NetAmount": "1500.00", "Currency": "EGP" }
    ]
  }
}
```

Practical Lab: Lab 13 – Read a public OData service

Open `https://services.odata.org/V2/Northwind/Northwind.svc/` in a browser (or Postman). 1) Read `$metadata` and find the entity type `Product`, its key and the navigation property to `Category`. 2) Read the first 5 products sorted by price descending as JSON. 3) Read product 1 with its category expanded. 4) Count products with UnitPrice > 50.

Solution: Lab 13

```
1) .../Northwind.svc/$metadata  -> EntityType Product, Key ProductID,
   NavigationProperty "Category" (also Order_Details, Supplier)
2) .../Products?$top=5&$orderby=UnitPrice desc&$format=json
3) .../Products(1)?$expand=Category&$format=json
4) .../Products/$count?$filter=UnitPrice gt 50
```

14. SAP Gateway Architecture

SAP Gateway is the SAP NetWeaver / ABAP Platform component that exposes OData services. Since 7.40 it is part of the standard stack (`SAP_GWFND`). It has two logical layers:

Layer	Component (old add-ons)	Role	Key transactions
Gateway Hub / Frontend server	`IW_FND` (now in SAP_GWFND)	Receives HTTP requests, routes to backend, service registry, caching, ICF	`/IWFND/MAINT_SERVICE`, `/IWFND/GW_CLIENT`, `/IWFND/ERROR_LOG`, `/IWFND/CACHE_CLEANUP`
Backend (BEP)	`IW_BEP`	Contains the service implementation (MPC/DPC classes) and business logic	`SEGW`, `/IWBEP/REG_SERVICE`, `/IWBEP/ERROR_LOG`, `/IWBEP/CACHE_CLEANUP`

Deployment options

- **Embedded deployment** – Gateway hub and backend in the **same** system (typical for S/4HANA). Simpler, no RFC; recommended by SAP for S/4HANA.
- **Hub deployment (central hub)** – a separate Gateway/Fiori Frontend Server (FES) connected via **trusted RFC** to one or more backends (ECC, CRM...). The UI5 apps and Fiori Launchpad run on the hub. *System alias* (`/IWFND/MAINT_SERVICE` → *Add System Alias*) tells the hub which backend serves the service.

Request flow

```
Browser / Fiori app
   |  HTTPS  GET /sap/opu/odata/sap/ZSALES_SRV/OrderSet?$top=10
   v
ICF node /sap/opu/odata  (SICF)  ->  Gateway Hub runtime (IWFND)
   |  checks service registry + system alias (+ RFC if hub deployment)
   v
Backend runtime (IWBEP) -> Model Provider Class (MPC_EXT) builds the model
                        -> Data Provider Class  (DPC_EXT) ORDERSET_GET_ENTITYSET
   |                         -> your ABAP: SELECT / BAPI / CDS / class call
   v
Response serialized to JSON/XML and returned
```

Key transactions & tools

Transaction	Purpose
`SEGW`	Gateway Service Builder – create projects, model, generate classes
`/IWFND/MAINT_SERVICE`	Register/activate services on the hub, assign system alias, ICF node, roles
`/IWFND/GW_CLIENT`	Built-in REST client to test requests (incl. $batch, headers, CSRF)
`/IWFND/ERROR_LOG` / `/IWBEP/ERROR_LOG`	Error logs with call stack, replay of failing request
`/IWFND/TRACES`	Performance & payload traces for a user/service
`/IWFND/CACHE_CLEANUP` / `/IWBEP/CACHE_CLEANUP`	Clear metadata cache after model changes
`SICF`	Activate ICF nodes (`/sap/opu/odata`)

Transaction	Purpose
`/IWFND/V4_ADMIN`	Publish OData V4 service groups
`SE80` / ADT	Edit DPC_EXT / MPC_EXT classes

15. Building Your First Service in SEGW

We will build `ZSALES_SRV` with an entity `Order` based on table VBAK.

Step-by-step

1. **Create project** – `SEGW` → *Create Project* → `ZSALES`, description, *Project Type: Service with SAP Annotations*, *Generation Strategy: Standard*, package (or `$TMP` for practice).
2. **Model the data** – right-click *Data Model* → *Import* → *DDIC Structure* → `VBAK` (or your own structure `ZSTR_ORDER`). Name the entity type `Order`, select fields VBELN, ERDAT, KUNNR, NETWR, WAERK, mark **VBELN as key**, tick *Create Default Entity Set* (`OrderSet`). Rename ABAP field names to readable OData names (`OrderId`, `CreatedOn`, `Customer`, `NetAmount`, `Currency`).

Alternatives: *Create* → *Entity Type* manually, import from *RFC/BOR interface* (wizard maps a BAPI), or *Redefine* from CDS / BW query.

3. **Set properties** – on the Entity Set: Creatable, Updatable, Deletable, Pageable, Addressable; on properties: Nullable, Filterable, Sortable, label (`sap:label`).
4. **Generate Runtime Objects** (red/white wheel icon). SEGW generates:

Generated object	Name	Purpose	Edit it?
Model Provider base class	`ZCL_ZSALES_MPC`	Defines the model in code (from SEGW)	**No** – regenerated
Model Provider extension	`ZCL_ZSALES_MPC_EXT`	Redefine `DEFINE` to add annotations / changes in code	Yes
Data Provider base class	`ZCL_ZSALES_DPC`	Generated CRUD method stubs raising "not implemented"	**No** – regenerated
Data Provider extension	`ZCL_ZSALES_DPC_EXT`	**Your business logic**: redefine CRUD methods	Yes
Technical model name	`ZSALES_MDL`	Registered model (`/IWBEP/REG_MODEL`)	–
Technical service name	`ZSALES_SRV`	Registered service (`/IWBEP/REG_SERVICE`)	–

5. **Register the service** – `/IWFND/MAINT_SERVICE` → *Add Service* → system alias `LOCAL` (embedded) → search `ZSALES_SRV` → *Add selected services* → package. (Or in SEGW: *Service Maintenance* folder → system → *Register*.)
6. **Test** – in `/IWFND/MAINT_SERVICE` select the service → *SAP Gateway Client* → `GET /sap/opu/odata/sap/ZSALES_SRV/$metadata` → status 200.

> **Watch out: Golden rule**
>
> Never put code in the `*_MPC` or `*_DPC` base classes – they are **overwritten** every time you click *Generate*. Always redefine the methods in `*_DPC_EXT` / `*_MPC_EXT`.

16. Implementing CRUD in the DPC_EXT

Open `ZCL_ZSALES_DPC_EXT` in SE80/ADT, select a method such as `ORDERSET_GET_ENTITYSET` and click *Redefine*. Every CRUD method follows the naming pattern `<ENTITYSET>_<OPERATION>`.

Method signature essentials

Parameter	Content
`iv_entity_name`, `iv_entity_set_name`, `iv_source_name`	Names of the requested entity (set) and of the source entity when navigating
`it_key_tab`	Key values from the URI: `('4711')` → name/value pairs
`it_navigation_path`	Path of navigation properties used
`it_filter_select_options`, `iv_filter_string`	`$filter` as select-options / as string (old style)
`is_paging` (`top`, `skip`), `it_order`	`$top`, `$skip`, `$orderby`
`io_tech_request_context`	**Recommended**: object with everything (keys, filter, select, paging, navigation) using **technical (ABAP) names**
`io_data_provider`	(CREATE/UPDATE) Reads the request body: `io_data_provider->read_entry_data( IMPORTING es_data = ls_entity )`
`er_entity` / `et_entityset`	Your result
`es_response_context`	Inline count, delta token, etc.

GET_ENTITYSET – read a list

```
METHOD orderset_get_entityset.
  " 1) $filter as SQL WHERE clause built by the framework (technical names!)
  DATA(lv_where) = io_tech_request_context->get_osql_where_clause_convert( ).

  " 2) select data (use a field list - never SELECT * in real services)
  SELECT vbeln, erdat, kunnr, netwr, waerk
    FROM vbak
    WHERE (lv_where)
    ORDER BY vbeln
    INTO CORRESPONDING FIELDS OF TABLE @et_entityset
    UP TO 1000 ROWS.

  " 3) $inlinecount=allpages
  IF io_tech_request_context->has_inlinecount( ) = abap_true.
    es_response_context-inlinecount = lines( et_entityset ).
  ENDIF.

  " 4) $orderby, $top / $skip - generic helpers
  /iwbep/cl_mgw_data_util=>orderby( EXPORTING it_order = it_order
                                    CHANGING  ct_data  = et_entityset ).
  /iwbep/cl_mgw_data_util=>paging(  EXPORTING is_paging = is_paging
                                    CHANGING  ct_data   = et_entityset ).
ENDMETHOD.
```

Tip: Paging on the database

> The helper above pages in ABAP memory. For big tables push paging to the DB: read `io_tech_request_context->get_top( )` / `get_skip( )` and use `UP TO lv_top ROWS OFFSET lv_skip` in the SELECT (with an `ORDER BY`). Count separately with `SELECT COUNT(*)` for `$inlinecount`.

GET_ENTITY – read one

```
METHOD orderset_get_entity.
  DATA ls_key TYPE zcl_zsales_mpc=>ts_order.

  " converts key values into the entity structure (technical names)
  io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).

  SELECT SINGLE vbeln, erdat, kunnr, netwr, waerk
    FROM vbak
    WHERE vbeln = @ls_key-vbeln
    INTO CORRESPONDING FIELDS OF @er_entity.

  IF sy-subrc <> 0.
    RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
      EXPORTING textid  = /iwbep/cx_mgw_busi_exception=>resource_not_found
                entity_type = iv_entity_name.
  ENDIF.
ENDMETHOD.
```

`zcl_zsales_mpc=>ts_order` is the structure type generated for the entity (also `tt_order` for the table type).

CREATE_ENTITY – POST

```
METHOD orderset_create_entity.
  DATA ls_order TYPE zcl_zsales_mpc=>ts_order.

  io_data_provider->read_entry_data( IMPORTING es_data = ls_order ).

  " validation -> 400 with readable message
  IF ls_order-kunnr IS INITIAL.
    DATA(lo_msg) = mo_context->get_message_container( ).
    lo_msg->add_message_text_only( iv_msg_type = 'E'
                                   iv_msg_text = 'Customer is mandatory' ).
    RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
      EXPORTING message_container = lo_msg.
  ENDIF.

  " real life: call BAPI_SALESORDER_CREATEFROMDAT2 / a RAP BO / your API class
  DATA(lo_api) = NEW zcl_order_api( ).
  TRY.
      ls_order-vbeln = lo_api->create( ls_order ).
    CATCH zcx_order_error INTO DATA(lx).
      RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
        EXPORTING previous = lx
                  message  = CONV #( lx->get_text( ) ).
  ENDTRY.

  er_entity = ls_order.          " return the created entity (with new key) -> 201
ENDMETHOD.
```

Request in the Gateway Client:

```
POST /sap/opu/odata/sap/ZSALES_SRV/OrderSet
Content-Type: application/json
```

```
X-CSRF-Token: <token from a previous GET with X-CSRF-Token: Fetch>

{ "Customer": "100001", "NetAmount": "250.00", "Currency": "EGP" }
```

UPDATE_ENTITY – PUT / PATCH

```
METHOD orderset_update_entity.
  DATA: ls_key   TYPE zcl_zsales_mpc=>ts_order,
        ls_order TYPE zcl_zsales_mpc=>ts_order.

  io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).
  io_data_provider->read_entry_data( IMPORTING es_data = ls_order ).
  ls_order-vbeln = ls_key-vbeln.                " key always from the URI

  " ... lock, update via BAPI / API class, unlock ...
  er_entity = ls_order.                         " -> 204 No Content
ENDMETHOD.
```

> **Note: PATCH / MERGE**
>
> For partial updates the framework calls `PATCH_ENTITY` (if redefined in `/IWBEP/IF_MGW_APPL_SRV_RUNTIME~PATCH_ENTITY`) – by default it reads the current entity via GET_ENTITY, merges the sent fields and calls UPDATE_ENTITY. So a working GET_ENTITY is required for PATCH.

DELETE_ENTITY

```
METHOD orderset_delete_entity.
  DATA ls_key TYPE zcl_zsales_mpc=>ts_order.
  io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).

  DATA(lo_api) = NEW zcl_order_api( ).
  TRY.
      lo_api->delete( ls_key-vbeln ).           " -> 204 No Content
    CATCH zcx_order_error INTO DATA(lx).
      RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
        EXPORTING message = CONV #( lx->get_text( ) ).
  ENDTRY.
ENDMETHOD.
```

Error handling in Gateway

Exception	HTTP	Use
`/IWBEP/CX_MGW_BUSI_EXCEPTION`	400 (or as set via `http_status_code`)	Business / validation errors – shown to the user
`/IWBEP/CX_MGW_TECH_EXCEPTION`	500	Technical errors (unexpected)
`/IWBEP/CX_MGW_NOT_IMPL_EXC`	501	Raised by the generated base class for non-redefined methods

The **message container** (`mo_context->get_message_container( )`) collects one or many messages (from BAPI `RETURN` tables with `add_messages_from_bapi`) that the UI shows in a message popover. Warnings and info can be sent with a successful response via the `sap-message` header (`add_message` with

`iv_add_to_response_header = abap_true`).

Practical Lab: Lab 16 – Your first CRUD service

Create table `ZTB_STUDENT` (MANDT, STUDENT_ID NUMC 8 key, NAME CHAR 40, EMAIL CHAR 60, CITY CHAR 30, CREATED_ON DATS). In SEGW create project `ZSTUDENT` with entity `Student` (set `StudentSet`) imported from the table. Implement all five CRUD methods. Test: create 3 students with POST, read all, read one, update the city with PUT, delete one. Return error 400 if the email does not contain '@' and 404 if a student does not exist.

Solution: Lab 16

```
CLASS zcl_zstudent_dpc_ext DEFINITION PUBLIC INHERITING FROM zcl_zstudent_dpc CREATE PUBLIC.
  PROTECTED SECTION.
    METHODS studentset_get_entityset REDEFINITION.
    METHODS studentset_get_entity    REDEFINITION.
    METHODS studentset_create_entity REDEFINITION.
    METHODS studentset_update_entity REDEFINITION.
    METHODS studentset_delete_entity REDEFINITION.
  PRIVATE SECTION.
    METHODS raise_busi IMPORTING iv_text TYPE bapi_msg
                       RAISING   /iwbep/cx_mgw_busi_exception.
    METHODS check_email IMPORTING iv_email TYPE ztb_student-email
                        RAISING   /iwbep/cx_mgw_busi_exception.
ENDCLASS.

CLASS zcl_zstudent_dpc_ext IMPLEMENTATION.
  METHOD raise_busi.
    DATA(lo_msg) = mo_context->get_message_container( ).
    lo_msg->add_message_text_only( iv_msg_type = 'E' iv_msg_text = iv_text ).
    RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
      EXPORTING message_container = lo_msg.
  ENDMETHOD.

  METHOD check_email.
    IF iv_email NS '@'.
      raise_busi( |Invalid e-mail { iv_email }| ).
    ENDIF.
  ENDMETHOD.

  METHOD studentset_get_entityset.
    DATA(lv_where) = io_tech_request_context->get_osql_where_clause_convert( ).
    SELECT * FROM ztb_student WHERE (lv_where)
      INTO CORRESPONDING FIELDS OF TABLE @et_entityset.
    /iwbep/cl_mgw_data_util=>orderby( EXPORTING it_order = it_order
                                      CHANGING  ct_data  = et_entityset ).
    /iwbep/cl_mgw_data_util=>paging( EXPORTING is_paging = is_paging
                                     CHANGING  ct_data   = et_entityset ).
  ENDMETHOD.

  METHOD studentset_get_entity.
    DATA ls_key TYPE zcl_zstudent_mpc=>ts_student.
    io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).
    SELECT SINGLE * FROM ztb_student WHERE student_id = @ls_key-student_id
      INTO CORRESPONDING FIELDS OF @er_entity.
    IF sy-subrc <> 0.
      RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
        EXPORTING textid           = /iwbep/cx_mgw_busi_exception=>resource_not_found
                  entity_type      = iv_entity_name
                  http_status_code = /iwbep/cx_mgw_busi_exception=>gcs_http_status_codes-not_fo
und.
    ENDIF.
  ENDMETHOD.

  METHOD studentset_create_entity.
    DATA ls_db TYPE ztb_student.
```

```
    io_data_provider->read_entry_data( IMPORTING es_data = er_entity ).
    check_email( er_entity-email ).
    IF er_entity-student_id IS INITIAL.        " simple number: max + 1
      SELECT MAX( student_id ) FROM ztb_student INTO @DATA(lv_max).
      er_entity-student_id = lv_max + 1.
    ENDIF.
    er_entity-created_on = sy-datum.
    ls_db = CORRESPONDING #( er_entity ).
    INSERT ztb_student FROM @ls_db.
    IF sy-subrc <> 0.
      raise_busi( 'Student already exists' ).
    ENDIF.
  ENDMETHOD.

  METHOD studentset_update_entity.
    DATA: ls_key TYPE zcl_zstudent_mpc=>ts_student,
          ls_db  TYPE ztb_student.
    io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).
    io_data_provider->read_entry_data( IMPORTING es_data = er_entity ).
    er_entity-student_id = ls_key-student_id.
    check_email( er_entity-email ).
    ls_db = CORRESPONDING #( er_entity ).
    UPDATE ztb_student FROM @ls_db.
    IF sy-subrc <> 0.
      raise_busi( 'Student not found' ).
    ENDIF.
  ENDMETHOD.

  METHOD studentset_delete_entity.
    DATA ls_key TYPE zcl_zstudent_mpc=>ts_student.
    io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).
    DELETE FROM ztb_student WHERE student_id = @ls_key-student_id.
    IF sy-subrc <> 0.
      raise_busi( 'Student not found' ).
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

```
GET    /sap/opu/odata/sap/ZSTUDENT_SRV/StudentSet?$format=json
GET    /sap/opu/odata/sap/ZSTUDENT_SRV/StudentSet('00000001')
POST   /sap/opu/odata/sap/ZSTUDENT_SRV/StudentSet  {"Name":"Ali","Email":"ali@x.com","City":"Ca
iro"}
PUT    /sap/opu/odata/sap/ZSTUDENT_SRV/StudentSet('00000001')  {"Name":"Ali","Email":"ali@x.com
","City":"Giza"}
DELETE /sap/opu/odata/sap/ZSTUDENT_SRV/StudentSet('00000002')
```

In real projects never write directly to the DB from the DPC – call an API/BO class (and use locks and `COMMIT WORK` handled properly). Direct `INSERT` is fine for a learning table.

17. Query Options in Depth

System query options start with `$` and are passed in the URL. The table shows what the framework does for you and what **you** must implement in the DPC_EXT (SEGW services). For CDS/RAP-based services the SADL framework handles all of them automatically.

Option	Example	Meaning	Who handles it in SEGW?
`$filter`	`$filter=Customer eq '100001' and NetAmount gt 500`	Restrict rows	**You** (use `get_osql_where_clause_convert` or `it_filter_select_options`)
`$orderby`	`$orderby=NetAmount desc,OrderId`	Sort	**You** (`it_order` / helper `orderby`)
`$top` / `$skip`	`$top=20&$skip=40`	Paging	**You** (`is_paging` / helper `paging` / DB `OFFSET`)
`$select`	`$select=OrderId,Customer`	Only these fields in response	Framework trims output; you *may* optimise the SELECT with `get_select_with_mandtry_fields( )`
`$expand`	`$expand=ToItems,ToCustomer`	Include related entities	Framework (calls navigation GET_ENTITYSET per row) or **you** (GET_EXPANDED_ENTITYSET for performance)
`$inlinecount` (V2)	`$inlinecount=allpages`	Total count in response (`__count`)	**You** set `es_response_context-inlinecount`
`/$count`	`OrderSet/$count?$filter=...`	Only the number	Framework counts GET_ENTITYSET result (or you optimise)
`$format`	`$format=json`	Response format	Framework
`search` (V2 SAP) / `$search` (V4)	`search=cairo`	Free text search	**You** (`io_tech_request_context->get_search_string( )`)
`$skiptoken`	server-driven paging	Next page token	You (`es_response_context-skiptoken`)

$filter operators

Operator	Meaning	Example
`eq` / `ne`	equal / not equal	`Status eq 'A'`
`gt` `ge` `lt` `le`	comparisons	`NetAmount ge 1000`
`and` / `or` / `not`	logical	`City eq 'Cairo' or City eq 'Giza'`
`substringof` (V2) / `contains` (V4)	text contains	`substringof('Ali',Name)`
`startswith` / `endswith`	text prefix/suffix	`startswith(Name,'A')`
Date (V2)	datetime literal	`CreatedOn ge datetime'2026-01-01T00:00:00'`

Reading the filter manually (select-options)

```
METHOD orderset_get_entityset.
  DATA: lr_kunnr TYPE RANGE OF kunnr,
        lr_erdat TYPE RANGE OF erdat.

  " technical (ABAP) property names via the tech request context
  DATA(lo_filter) = io_tech_request_context->get_filter( ).
  LOOP AT lo_filter->get_filter_select_options( ) INTO DATA(ls_so).
    CASE ls_so-property.
      WHEN 'KUNNR'.
        lr_kunnr = CORRESPONDING #( ls_so-select_options ).
      WHEN 'ERDAT'.
        lr_erdat = CORRESPONDING #( ls_so-select_options ).
    ENDCASE.
  ENDLOOP.

  " paging pushed to the DB
  DATA(lv_top)  = CONV i( io_tech_request_context->get_top( ) ).
  DATA(lv_skip) = CONV i( io_tech_request_context->get_skip( ) ).
  IF lv_top IS INITIAL.
    lv_top = 1000.                    " protect the system
  ENDIF.

  SELECT vbeln, erdat, kunnr, netwr, waerk
    FROM vbak
    WHERE kunnr IN @lr_kunnr AND erdat IN @lr_erdat
    ORDER BY vbeln
    INTO CORRESPONDING FIELDS OF TABLE @et_entityset
    UP TO @lv_top ROWS OFFSET @lv_skip.

  IF io_tech_request_context->has_inlinecount( ) = abap_true.
    SELECT COUNT(*) FROM vbak
      WHERE kunnr IN @lr_kunnr AND erdat IN @lr_erdat
      INTO @DATA(lv_count).
    es_response_context-inlinecount = lv_count.
  ENDIF.
ENDMETHOD.
```

Watch out: Filter that cannot be converted

Complex filters (`or` across different fields, functions like `substringof`) do not appear in `get_filter_select_options( )` – the table is empty but the filter string is filled. Either use `get_osql_where_clause_convert( )` (handles most cases) or parse `iv_filter_string`. Never silently ignore a filter – the user would see wrong data.

18. Associations, Navigation and $expand

Modelling a header–item relationship in SEGW

1. Create entity types `Order` (key `OrderId`) and `OrderItem` (keys `OrderId`, `ItemNo`) with sets `OrderSet`, `OrderItemSet`.
2. Right-click *Associations* → *Create*: name `Order_Items`, principal `Order` cardinality **1**, dependent `OrderItem` cardinality **M** (0..n), create navigation property `ToItems` in Order (and optionally `ToOrder` in OrderItem).
3. *Referential constraint*: principal key `OrderId` ↔ dependent property `OrderId`.
4. Generate runtime objects. The association set is created automatically.

Now these URLs work:

```
GET .../OrderSet('4711')/ToItems                -> items of one order
GET .../OrderSet('4711')?$expand=ToItems        -> order + its items in one call
GET .../OrderSet?$expand=ToItems&$top=10        -> 10 orders, each with items
GET .../OrderItemSet(OrderId='4711',ItemNo='10')/ToOrder
```

Implementing navigation – ORDERITEMSET_GET_ENTITYSET

When called via navigation (`OrderSet('4711')/ToItems`), the **item** entity set method is executed with the source entity's key available.

```
METHOD orderitemset_get_entityset.
  DATA ls_order_key TYPE zcl_zsales_mpc=>ts_order.
  DATA lr_vbeln     TYPE RANGE OF vbeln.

  " called through navigation from Order?
  IF io_tech_request_context->get_source_entity_type_name( )
       = zcl_zsales_mpc=>gc_order.
    io_tech_request_context->get_converted_source_keys(
      IMPORTING es_key_values = ls_order_key ).
    lr_vbeln = VALUE #( ( sign = 'I' option = 'EQ' low = ls_order_key-vbeln ) ).
  ENDIF.

  SELECT vbeln, posnr, matnr, kwmeng, netwr
    FROM vbap
    WHERE vbeln IN @lr_vbeln
    INTO CORRESPONDING FIELDS OF TABLE @et_entityset.
ENDMETHOD.
```

$expand: default behaviour vs. GET_EXPANDED_ENTITYSET

By default, for `OrderSet?$expand=ToItems` the framework calls `ORDERSET_GET_ENTITYSET` once and then `ORDERITEMSET_GET_ENTITYSET` **once per order** (N+1 problem → slow). For performance, redefine `/IWBEP/IF_MGW_APPL_SRV_RUNTIME~GET_EXPANDED_ENTITYSET` (and `GET_EXPANDED_ENTITY` for single entity) and read everything with two SELECTs:

```
METHOD /iwbep/if_mgw_appl_srv_runtime~get_expanded_entityset.
  " deep structure: order fields + table of items named like the nav property
  TYPES: BEGIN OF ty_order_deep.
           INCLUDE TYPE zcl_zsales_mpc=>ts_order.
  TYPES:   toitems TYPE zcl_zsales_mpc=>tt_orderitem,
         END OF ty_order_deep,
         tt_order_deep TYPE STANDARD TABLE OF ty_order_deep WITH EMPTY KEY.
  DATA lt_deep TYPE tt_order_deep.

  CASE iv_entity_set_name.
    WHEN 'OrderSet'.
      DATA(lt_expand) = io_tech_request_context->get_expand( ).    " e.g. 'TOITEMS'
      SELECT vbeln, erdat, kunnr, netwr, waerk FROM vbak
        INTO CORRESPONDING FIELDS OF TABLE @lt_deep UP TO 100 ROWS.

      IF lt_deep IS NOT INITIAL AND lt_expand CS 'TOITEMS'.
        SELECT vbeln, posnr, matnr, kwmeng, netwr FROM vbap
          FOR ALL ENTRIES IN @lt_deep WHERE vbeln = @lt_deep-vbeln
          INTO TABLE @DATA(lt_items).
        LOOP AT lt_deep ASSIGNING FIELD-SYMBOL(<ls_deep>).
          <ls_deep>-toitems = VALUE #( FOR ls_it IN lt_items WHERE ( vbeln = <ls_deep>-vbeln )
                                       ( CORRESPONDING #( ls_it ) ) ).
        ENDLOOP.
        " tell the framework that we already handled the expand
        APPEND 'TOITEMS' TO et_expanded_tech_clauses.
      ENDIF.

      copy_data_to_ref( EXPORTING is_data = lt_deep
                        CHANGING  cr_data = er_entityset ).
    WHEN OTHERS.
      super->/iwbep/if_mgw_appl_srv_runtime~get_expanded_entityset(
        EXPORTING iv_entity_name = iv_entity_name iv_entity_set_name = iv_entity_set_name
                  iv_source_name = iv_source_name it_filter_select_options = it_filter_select_opt
ions
                  it_order = it_order is_paging = is_paging it_navigation_path = it_navigation_pa
th
                  it_key_tab = it_key_tab iv_filter_string = iv_filter_string
                  iv_search_string = iv_search_string io_expand = io_expand
                  io_tech_request_context = io_tech_request_context
        IMPORTING er_entityset = er_entityset
                  et_expanded_clauses = et_expanded_clauses
                  et_expanded_tech_clauses = et_expanded_tech_clauses
                  es_response_context = es_response_context ).
  ENDCASE.
ENDMETHOD.
```

> **Watch out: Deep structure names**
>
> The component holding the child table must have the **same name as the navigation property** (in technical/ABAP form, upper case `TOITEMS`), otherwise the framework cannot map it and returns empty `results`.

19. Deep Insert, $batch, Function Imports, Media and ETags

Deep insert – create header and items in one POST

```
POST .../OrderSet
{ "Customer": "100001", "Currency": "EGP",
  "ToItems": [ { "ItemNo": "10", "Material": "MAT-01", "Quantity": "2" },
               { "ItemNo": "20", "Material": "MAT-02", "Quantity": "5" } ] }
```

Redefine `/IWBEP/IF_MGW_APPL_SRV_RUNTIME~CREATE_DEEP_ENTITY`:

```
METHOD /iwbep/if_mgw_appl_srv_runtime~create_deep_entity.
  TYPES: BEGIN OF ty_deep.
           INCLUDE TYPE zcl_zsales_mpc=>ts_order.
  TYPES:   toitems TYPE zcl_zsales_mpc=>tt_orderitem,
         END OF ty_deep.
  DATA ls_deep TYPE ty_deep.

  CASE iv_entity_set_name.
    WHEN 'OrderSet'.
      io_data_provider->read_entry_data( IMPORTING es_data = ls_deep ).

      " map to BAPI structures and create the order
      DATA: lt_items_in TYPE STANDARD TABLE OF bapisditm,
            lt_partners TYPE STANDARD TABLE OF bapiparnr,
            lt_return   TYPE STANDARD TABLE OF bapiret2.
      lt_items_in = VALUE #( FOR i IN ls_deep-toitems
                             ( itm_number = i-posnr material = i-matnr
                               target_qty = i-kwmeng ) ).
      lt_partners = VALUE #( ( partn_role = 'AG' partn_numb = ls_deep-kunnr ) ).

      CALL FUNCTION 'BAPI_SALESORDER_CREATEFROMDAT2'
        EXPORTING order_header_in = VALUE bapisdhd1( doc_type   = 'OR'
                                                     sales_org  = '1000'
                                                     distr_chan = '10'
                                                     division   = '00' )
        IMPORTING salesdocument   = ls_deep-vbeln
        TABLES    return          = lt_return
                  order_items_in  = lt_items_in
                  order_partners  = lt_partners.
      IF line_exists( lt_return[ type = 'E' ] ) OR line_exists( lt_return[ type = 'A' ] ).
        CALL FUNCTION 'BAPI_TRANSACTION_ROLLBACK'.
        DATA(lo_msg) = mo_context->get_message_container( ).
        lo_msg->add_messages_from_bapi( it_bapi_messages = lt_return ).
        RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
          EXPORTING message_container = lo_msg.
      ENDIF.
      CALL FUNCTION 'BAPI_TRANSACTION_COMMIT' EXPORTING wait = abap_true.

      copy_data_to_ref( EXPORTING is_data = ls_deep CHANGING cr_data = er_deep_entity ).
  ENDCASE.
ENDMETHOD.
```

> **Note: COMMIT inside Gateway**

> Inside a `$batch` changeset you must **not** commit (the framework commits at the end). Outside `$batch`, single CRUD calls can commit (as above). For clean code, check `mo_context`/use the changeset methods or move the commit to `changeset_end`.

$batch – many operations in one HTTP request

UI5 sends `$batch` by default (ODataModel V2). It contains **query parts** (GETs) and **changesets** (groups of POST/PUT/DELETE executed as one logical unit of work: all or nothing).

```
POST .../$batch
Content-Type: multipart/mixed; boundary=batch_1

--batch_1
Content-Type: application/http

GET OrderSet('4711') HTTP/1.1

--batch_1
Content-Type: multipart/mixed; boundary=changeset_1

--changeset_1
Content-Type: application/http

PUT OrderSet('4711') HTTP/1.1
Content-Type: application/json

{"Customer":"100002"}
--changeset_1--
--batch_1--
```

In the DPC_EXT redefine:

- `/IWBEP/IF_MGW_APPL_SRV_RUNTIME~CHANGESET_BEGIN` – decide if you support several operations in one changeset (set `cv_defer_mode = abap_true` to collect them).
- `~CHANGESET_PROCESS` – (defer mode) process all operations of the changeset together, e.g. one BAPI call for many items.
- `~CHANGESET_END` – `COMMIT WORK` once for the changeset.

```
METHOD /iwbep/if_mgw_appl_srv_runtime~changeset_begin.
  " allow more than one operation per changeset (default: only 1)
  LOOP AT it_operation_info TRANSPORTING NO FIELDS
       WHERE entity_set <> 'OrderSet' AND entity_set <> 'OrderItemSet'.
    RAISE EXCEPTION TYPE /iwbep/cx_mgw_tech_exception
      EXPORTING textid = /iwbep/cx_mgw_tech_exception=>changeset_not_supported.
  ENDLOOP.
ENDMETHOD.

METHOD /iwbep/if_mgw_appl_srv_runtime~changeset_end.
  COMMIT WORK.
ENDMETHOD.
```

Function imports (custom operations)

Use when the operation is not CRUD: *release order*, *calculate price*, *get dashboard KPIs*.

1. SEGW → *Function Imports* → *Create* `ReleaseOrder`; return type kind *Entity Type* `Order` (cardinality 1) or Complex Type; HTTP method **POST** (changes data) or **GET** (read); add parameters `OrderId` (Edm.String).
2. Generate, then redefine `/IWBEP/IF_MGW_APPL_SRV_RUNTIME~EXECUTE_ACTION`:

```
METHOD /iwbep/if_mgw_appl_srv_runtime~execute_action.
  DATA ls_order TYPE zcl_zsales_mpc=>ts_order.

  CASE iv_action_name.
    WHEN 'ReleaseOrder'.
      DATA(ls_param) = VALUE /iwbep/s_mgw_name_value_pair(
                         it_parameter[ name = 'OrderId' ] OPTIONAL ).
      IF ls_param IS INITIAL.
        RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
          EXPORTING message = 'OrderId is required'.
      ENDIF.
      ls_order-vbeln = |{ ls_param-value ALPHA = IN }|.
      " ... release logic (BAPI / API class) ...
      SELECT SINGLE vbeln, erdat, kunnr, netwr, waerk FROM vbak
        WHERE vbeln = @ls_order-vbeln INTO CORRESPONDING FIELDS OF @ls_order.
      copy_data_to_ref( EXPORTING is_data = ls_order CHANGING cr_data = er_data ).
  ENDCASE.
ENDMETHOD.
```

```
POST .../ReleaseOrder?OrderId='4711'
```

Media / streams (files, images, PDFs)

1. Mark the entity type as **Media** (checkbox *Media* in SEGW) – e.g. `Attachment` with key `DocId`.
2. In `MPC_EXT->DEFINE` set the MIME type property: `lo_entity->get_property( 'MimeType' )->set_as_content_type( ).`
3. Redefine `GET_STREAM` (download: `GET .../AttachmentSet('1')/$value`) and `CREATE_STREAM` (upload: POST with binary body + header `slug` for the file name).

```
METHOD /iwbep/if_mgw_appl_srv_runtime~get_stream.
  DATA ls_stream TYPE /iwbep/if_mgw_appl_types=>ty_s_media_resource. " value, mime_type
  DATA ls_key    TYPE zcl_zsales_mpc=>ts_attachment.
  io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key ).
  SELECT SINGLE content, mime_type FROM zattach WHERE doc_id = @ls_key-doc_id
    INTO (@ls_stream-value, @ls_stream-mime_type).
  copy_data_to_ref( EXPORTING is_data = ls_stream CHANGING cr_data = er_stream ).
ENDMETHOD.

METHOD /iwbep/if_mgw_appl_srv_runtime~create_stream.
  DATA ls_attach TYPE zattach.
  TRY.
      ls_attach-doc_id = cl_system_uuid=>create_uuid_c32_static( ).
    CATCH cx_uuid_error INTO DATA(lx_uuid).
      RAISE EXCEPTION TYPE /iwbep/cx_mgw_tech_exception EXPORTING previous = lx_uuid.
  ENDTRY.
  ls_attach-file_name = iv_slug.                    " header 'slug'
  ls_attach-mime_type = is_media_resource-mime_type.
  ls_attach-content   = is_media_resource-value.
  INSERT zattach FROM @ls_attach.
  DATA(ls_entity) = CORRESPONDING zcl_zsales_mpc=>ts_attachment( ls_attach ).
  copy_data_to_ref( EXPORTING is_data = ls_entity CHANGING cr_data = er_entity ).
```

```
ENDMETHOD.
```

PDF forms (Adobe/SmartForms) for Fiori apps are typically delivered this way: generate the PDF as XSTRING in `GET_STREAM` and return `application/pdf`.

ETags – optimistic concurrency

An **ETag** is a version stamp (e.g. last-changed timestamp). The client sends it back in the `If-Match` header on PUT/DELETE; if the server's current value differs, the update is rejected with **412 Precondition Failed** – preventing "lost updates".

- In SEGW select the entity type → property `ChangedAt` → set as **ETag** (*Entity Type → Properties → ETag*).
- The framework compares `If-Match` with the value read by **GET_ENTITY** (so GET_ENTITY must return the current timestamp).
- UI5 handles ETags automatically.

Soft state, caching and annotations (brief)

- **Soft state** – keeps the backend session alive between requests for a short time (redefine `/IWBEP/IF_MGW_SOST_SRV_RUNTIME~OPERATION_START/END`); useful for expensive buffers. Use carefully.
- **Annotations** – add UI semantics (labels, value helps, Fiori elements UI annotations) in `MPC_EXT->DEFINE` or via an annotation model (SEGW *Service with Vocabulary-Based Annotations*) – in modern development you put them in CDS metadata extensions instead.
- **Value help** – `sap:value-list` / `Common.ValueList` annotations pointing to a value-help entity set.

Practical Lab: Lab 19 – Header/Item service with deep insert

Extend your service with entity `OrderItem` (VBAP: VBELN, POSNR, MATNR, KWMENG, NETWR), association `Order_Items` (1:n) with navigation `ToItems`. Implement: (a) items via navigation, (b) `GET_EXPANDED_ENTITYSET` for `$expand=ToItems`, (c) `CREATE_DEEP_ENTITY` that creates a custom header table `ZORD_H` and item table `ZORD_I` (no BAPI needed) and returns the new order number from a number range or `MAX+1`, (d) function import `GetOrderTotal` (GET, parameter OrderId, returns complex type `Total` with Amount and Currency).

Solution: Lab 19 (key parts)

```
" (c) deep insert into custom tables
METHOD /iwbep/if_mgw_appl_srv_runtime~create_deep_entity.
  TYPES: BEGIN OF ty_deep.
           INCLUDE TYPE zcl_zorder_mpc=>ts_order.
  TYPES:   toitems TYPE zcl_zorder_mpc=>tt_orderitem,
         END OF ty_deep.
  DATA ls_deep TYPE ty_deep.

  io_data_provider->read_entry_data( IMPORTING es_data = ls_deep ).
  IF ls_deep-toitems IS INITIAL.
    RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
      EXPORTING message = 'An order needs at least one item'.
  ENDIF.

  SELECT MAX( vbeln ) FROM zord_h INTO @DATA(lv_last).
  DATA lv_num TYPE n LENGTH 10.
  lv_num = lv_last + 1.                    " keeps leading zeros
  ls_deep-vbeln = lv_num.
  ls_deep-netwr = REDUCE #( INIT s = CONV netwr_ap( 0 )
                            FOR i IN ls_deep-toitems NEXT s = s + i-netwr ).

  DATA(ls_head)  = CORRESPONDING zord_h( ls_deep ).
```

```abap
  DATA(lt_items) = VALUE zord_i_tt( FOR i IN ls_deep-toitems
                    ( CORRESPONDING #( i ) ) ).
  LOOP AT lt_items ASSIGNING FIELD-SYMBOL(<i>).
    <i>-vbeln = ls_deep-vbeln.
    <i>-posnr = sy-tabix * 10.
  ENDLOOP.
  INSERT zord_h FROM @ls_head.
  INSERT zord_i FROM TABLE @lt_items.

  ls_deep-toitems = CORRESPONDING #( lt_items ).
  copy_data_to_ref( EXPORTING is_data = ls_deep CHANGING cr_data = er_deep_entity ).
ENDMETHOD.

" (d) function import returning a complex type
METHOD /iwbep/if_mgw_appl_srv_runtime~execute_action.
  DATA ls_total TYPE zcl_zorder_mpc=>ts_total.  " generated complex type (check name in MPC)
  IF iv_action_name = 'GetOrderTotal'.
    DATA(lv_id) = CONV vbeln( it_parameter[ name = 'OrderId' ]-value ).
    SELECT SINGLE netwr, waerk FROM zord_h WHERE vbeln = @lv_id
      INTO (@ls_total-amount, @ls_total-currency).
    copy_data_to_ref( EXPORTING is_data = ls_total CHANGING cr_data = er_data ).
  ENDIF.
ENDMETHOD.
```

```
POST .../OrderSet
{"Customer":"100001","Currency":"EGP","ToItems":[{"Material":"M1","Quantity":"2","NetAmount":"1
00.00"}]}
GET  .../GetOrderTotal?OrderId='0000000001'
```

`zord_i_tt` is a table type of `ZORD_I` created in SE11 (or declare a local `TYPES ... TABLE OF zord_i`).

20. CDS-Based OData Services

Writing SELECTs, filters and paging by hand in the DPC is a lot of code. With **Core Data Services (CDS)** views, the **SADL** framework (Service Adaptation Definition Language) translates OData requests (`$filter`, `$orderby`, `$top`, `$select`, `$expand`, `$count`, search) **directly into optimised SQL** on HANA – no ABAP code for read operations.

There are three ways to expose CDS as OData (oldest → newest):

Approach	How	Write operations	Status
@OData.publish : true	Annotation in the CDS view; activates a generated service `<VIEW>_CDS`	Only via BOPF (`@ObjectModel.transactionalProcessingEnabled`)	Legacy (7.50 – S/4 1909 programming model) – still common in projects
SEGW Reference Data Source (RDS)	SEGW → *Data Model* → *Reference* → *Data Source* → CDS view; associations become navigation	Redefine DPC_EXT methods for create/update/delete	Common when you need SEGW + custom logic
SEGW Mapping to Data Source	Map entity set to a CDS view in *Service Implementation*	DPC_EXT	Less common
RAP Service Binding	Service definition + binding (V2/V4)	Behavior definition (managed/unmanaged)	**Recommended** (S/4HANA 1909+, BTP) – Chapter 21

Example: CDS view with @OData.publish

```
@AbapCatalog.sqlViewName: 'ZVSALESORD'
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Sales orders'
@OData.publish: true
define view ZC_SalesOrder
  as select from vbak
  association [0..*] to ZC_SalesOrderItem as _Items
    on $projection.SalesOrder = _Items.SalesOrder
  association [0..1] to I_Customer as _Customer
    on $projection.SoldToParty = _Customer.Customer
{
  key vbeln as SalesOrder,
      erdat as CreatedOn,
      @ObjectModel.text.association: '_Customer'
      kunnr as SoldToParty,
      @Semantics.amount.currencyCode: 'Currency'
      netwr as NetAmount,
      @Semantics.currencyCode: true
      waerk as Currency,
      _Items,
      _Customer
}
```

After activation, ADT shows a marker "Service ZC_SALESORDER_CDS is not active" → register it in `/IWFND/MAINT_SERVICE` (Add service, search `ZC_SALESORDER_CDS`). Then:

```
GET /sap/opu/odata/sap/ZC_SALESORDER_CDS/ZC_SalesOrder?$filter=SoldToParty eq '100001'
GET /sap/opu/odata/sap/ZC_SALESORDER_CDS/ZC_SalesOrder('4711')/to_Items
```

Associations that are exposed in the projection list become navigation properties named `to_<Association>` (e.g. `to_Items`).

SEGW with Reference Data Source

1. Create SEGW project → right-click *Data Model* → *Reference* → *Data Source* → choose CDS view `ZC_SalesOrder` (and associated views).
2. Generate. All reads (incl. filter, paging, expand via associations) are handled by SADL.
3. For create/update/delete or special logic redefine methods in `DPC_EXT`. You can still add custom entity sets (manual ones) to the same project.

CDS concepts you need for OData

Concept	Purpose
Basic / Interface view (`I_`)	Reusable, close to the DB table, clean names
Consumption view (`C_`)	Tailored for one app / service, with UI annotations
Association	Lazy join – becomes a navigation property; only joined when used (`$expand`)
`@AccessControl.authorizationCheck` + DCL (`define role`)	Row-level authorizations applied automatically to OData reads
`@UI.*` annotations / Metadata extension (`@Metadata.allowExtensions: true`)	Fiori elements layout: `@UI.lineItem`, `@UI.selectionField`, `@UI.identification`, `@UI.facet`
`@Consumption.valueHelpDefinition`	Value help for a field
`@Search.searchable` + `@Search.defaultSearchElement`	Enables free-text search (`search=` / `$search`)
`@ObjectModel.text.element` / `text.association`	Shows descriptions next to IDs
CDS view entity (`define view entity`)	New syntax (7.55+) without SQL view, recommended for new development

Practical Lab: Lab 20 – CDS-based read service

Create interface view entity `ZI_Student` on table `ZTB_STUDENT` (from Lab 16) and consumption view `ZC_Student` with `@OData.publish: true` (use classic `define view` with sqlViewName if your release requires it), UI annotations so that Fiori elements shows Name, Email, City in the list and City as filter field, and `@Search.searchable` on Name. Register and test `$filter`, `$orderby`, `$top`, `$inlinecount` and `search` without writing any ABAP code.

Solution: Lab 20

```
@AbapCatalog.sqlViewName: 'ZISTUDENT'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Student - interface view'
define view ZI_Student as select from ztb_student {
  key student_id as StudentId,
      name       as Name,
      email      as Email,
      city       as City,
      created_on as CreatedOn
}
```

```
@AbapCatalog.sqlViewName: 'ZCSTUDENT'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Students'
@OData.publish: true
@Search.searchable: true
```

```
define view ZC_Student as select from ZI_Student {
      @UI.lineItem: [{ position: 10 }]
  key StudentId,
      @UI.lineItem: [{ position: 20 }]
      @Search.defaultSearchElement: true
      @Search.fuzzinessThreshold: 0.8
      Name,
      @UI.lineItem: [{ position: 30 }]
      Email,
      @UI.lineItem: [{ position: 40 }]
      @UI.selectionField: [{ position: 10 }]
      City,
      CreatedOn
}
```

```
/sap/opu/odata/sap/ZC_STUDENT_CDS/ZC_Student?$filter=City eq 'Cairo'&$orderby=Name&$top=5&$inli
necount=allpages
/sap/opu/odata/sap/ZC_STUDENT_CDS/ZC_Student?search=ali
```

21. RAP – ABAP RESTful Application Programming Model (OData V4)

RAP is SAP's current model for building Fiori apps and Web APIs on S/4HANA (1909+) and SAP BTP ABAP Environment ("Steampunk"). It replaces SEGW + BOPF for new development. Everything is built in **ADT (Eclipse)**.

The RAP layers

Layer	Artifact	Purpose
Data model	CDS view entities: `define root view entity` + child entities (`composition`)	The Business Object (BO) structure
Behavior	**Behavior Definition** (BDEF)	Which operations: create/update/delete, actions, validations, determinations, locking, ETag, draft
Behavior implementation	**Behavior pool** – ABAP class (`ZBP_...`) with local handler (`LHC_`) and saver (`LSC_`) classes	Your ABAP logic (in EML)
Projection	Projection views (`as projection on`) + projection BDEF	Expose only what this app/API needs, UI annotations
Business service	**Service Definition** (`define service`)	Which entities are exposed
Binding	**Service Binding** (OData V2/V4, UI/Web API)	Protocol + publish → URL; *Preview* Fiori elements app

Managed vs. unmanaged

- **Managed** – the RAP framework does CRUD, buffering, locking and saving to DB tables automatically. You add validations, determinations and actions. Best for new applications ("greenfield").
- **Unmanaged** – you implement CRUD and save yourself (e.g. calling existing BAPIs / legacy code) – "brownfield".
- **Managed with unmanaged save** / **additional save** – framework handles transactional buffer, you control saving.
- **Draft** – `with draft;` lets users save incomplete data in draft tables and continue later (typical Fiori elements behaviour).

Example: Travel booking (managed, simplified)

1. Database table

```
@EndUserText.label : 'Travel'
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table ztravel_a {
  key client      : abap.clnt not null;
  key travel_uuid : sysuuid_x16 not null;
  travel_id       : abap.numc(8);
  customer_id     : abap.numc(6);
  begin_date      : abap.dats;
  end_date        : abap.dats;
  @Semantics.amount.currencyCode : 'ztravel_a.currency_code'
```

```
  total_price     : abap.curr(16,2);
  currency_code   : abap.cuky;
  status          : abap.char(1);
  created_by      : abp_creation_user;
  created_at      : abp_creation_tstmpl;
  last_changed_by : abp_lastchange_user;
  last_changed_at : abp_lastchange_tstmpl;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

2. Root view entity

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Travel - root'
define root view entity ZI_Travel
  as select from ztravel_a
{
  key travel_uuid     as TravelUUID,
      travel_id       as TravelID,
      customer_id     as CustomerID,
      begin_date      as BeginDate,
      end_date        as EndDate,
      @Semantics.amount.currencyCode: 'CurrencyCode'
      total_price     as TotalPrice,
      currency_code   as CurrencyCode,
      status          as Status,
      @Semantics.user.createdBy: true
      created_by      as CreatedBy,
      @Semantics.systemDateTime.createdAt: true
      created_at      as CreatedAt,
      @Semantics.user.lastChangedBy: true
      last_changed_by as LastChangedBy,
      @Semantics.systemDateTime.lastChangedAt: true
      last_changed_at as LastChangedAt,
      @Semantics.systemDateTime.localInstanceLastChangedAt: true
      local_last_changed_at as LocalLastChangedAt
}
```

3. Behavior definition

```
managed implementation in class zbp_i_travel unique;
strict ( 2 );

define behavior for ZI_Travel alias Travel
persistent table ztravel_a
lock master
authorization master ( instance )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) TravelID, Status, CreatedBy, CreatedAt, LastChangedBy, LastChangedAt;
  field ( mandatory ) CustomerID, BeginDate, EndDate;

  action acceptTravel result [1] $self;
  action rejectTravel result [1] $self;

  determination setInitialValues on modify { create; }
  validation validateDates on save { create; field BeginDate, EndDate; }
```

```
  mapping for ztravel_a
  {
    TravelUUID = travel_uuid;   TravelID = travel_id;     CustomerID = customer_id;
    BeginDate = begin_date;     EndDate = end_date;       TotalPrice = total_price;
    CurrencyCode = currency_code; Status = status;        CreatedBy = created_by;
    CreatedAt = created_at;     LastChangedBy = last_changed_by;
    LastChangedAt = last_changed_at; LocalLastChangedAt = local_last_changed_at;
  }
}
```

4. Behavior implementation (EML – Entity Manipulation Language)

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
      IMPORTING keys REQUEST requested_authorizations FOR Travel RESULT result.
    METHODS setinitialvalues FOR DETERMINE ON MODIFY
      IMPORTING keys FOR Travel~setInitialValues.
    METHODS validatedates FOR VALIDATE ON SAVE
      IMPORTING keys FOR Travel~validateDates.
    METHODS accepttravel FOR MODIFY
      IMPORTING keys FOR ACTION Travel~acceptTravel RESULT result.
    METHODS rejecttravel FOR MODIFY
      IMPORTING keys FOR ACTION Travel~rejectTravel RESULT result.
ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  METHOD get_instance_authorizations.
    " everyone may do everything in this demo
  ENDMETHOD.

  METHOD setinitialvalues.
    READ ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel FIELDS ( Status TravelID ) WITH CORRESPONDING #( keys )
      RESULT DATA(lt_travel).

    SELECT MAX( travel_id ) FROM ztravel_a INTO @DATA(lv_max_id).

    MODIFY ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel UPDATE FIELDS ( Status TravelID )
      WITH VALUE #( FOR ls IN lt_travel INDEX INTO i
                    ( %tky     = ls-%tky
                      Status   = 'O'                    " open
                      TravelID = lv_max_id + i ) ).
  ENDMETHOD.

  METHOD validatedates.
    READ ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel FIELDS ( BeginDate EndDate ) WITH CORRESPONDING #( keys )
      RESULT DATA(lt_travel).

    LOOP AT lt_travel INTO DATA(ls_travel).
      IF ls_travel-EndDate < ls_travel-BeginDate.
        APPEND VALUE #( %tky = ls_travel-%tky ) TO failed-travel.
        APPEND VALUE #( %tky = ls_travel-%tky
                        %msg = new_message_with_text(
                                 severity = if_abap_behv_message=>severity-error
                                 text     = 'End date must be after begin date' )
                        %element-EndDate = if_abap_behv=>mk-on )
               TO reported-travel.
      ENDIF.
    ENDLOOP.
```

```
  ENDMETHOD.

  METHOD accepttravel.
    MODIFY ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel UPDATE FIELDS ( Status )
      WITH VALUE #( FOR key IN keys ( %tky = key-%tky Status = 'A' ) ).

    READ ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel ALL FIELDS WITH CORRESPONDING #( keys )
      RESULT DATA(lt_travel).
    result = VALUE #( FOR ls IN lt_travel ( %tky = ls-%tky %param = ls ) ).
  ENDMETHOD.

  METHOD rejecttravel.
    MODIFY ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel UPDATE FIELDS ( Status )
      WITH VALUE #( FOR key IN keys ( %tky = key-%tky Status = 'X' ) ).
    READ ENTITIES OF zi_travel IN LOCAL MODE
      ENTITY Travel ALL FIELDS WITH CORRESPONDING #( keys )
      RESULT DATA(lt_travel).
    result = VALUE #( FOR ls IN lt_travel ( %tky = ls-%tky %param = ls ) ).
  ENDMETHOD.
ENDCLASS.
```

5. Projection, service definition and binding

```
@EndUserText.label: 'Travel - projection'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Metadata.allowExtensions: true
@Search.searchable: true
define root view entity ZC_Travel
  provider contract transactional_query
  as projection on ZI_Travel
{
  key TravelUUID,
      @Search.defaultSearchElement: true
      TravelID,
      CustomerID,
      BeginDate,
      EndDate,
      TotalPrice,
      CurrencyCode,
      Status,
      LocalLastChangedAt
}
```

```
projection;
strict ( 2 );

define behavior for ZC_Travel alias Travel
use etag
{
  use create;
  use update;
  use delete;
  use action acceptTravel;
  use action rejectTravel;
}
```

```
@EndUserText.label: 'Travel service'
define service ZUI_TRAVEL {
```

```
  expose ZC_Travel as Travel;
}
```

Finally: *New* → *Service Binding* `ZUI_TRAVEL_O4`, binding type **OData V4 – UI**, service definition `ZUI_TRAVEL` → **Publish** → select the entity → **Preview** to open a generated Fiori elements List Report. Choose **OData V4 – Web API** for an API without UI annotations.

EML cheat sheet

Statement	Purpose
`READ ENTITIES OF bo ENTITY e FIELDS ( ... ) WITH keys RESULT lt`	Read BO data (incl. transactional buffer)
`MODIFY ENTITIES OF bo ENTITY e CREATE / UPDATE / DELETE / EXECUTE action ...`	Change BO data
`COMMIT ENTITIES.`	Save (outside behavior pool – e.g. in a report or class)
`IN LOCAL MODE`	Skip authorization & feature control (inside own BO)
`%tky` / `%key` / `%cid` / `%param` / `%msg` / `%element`	Derived-type components: transactional key, key, content ID (for create), action result, message, field marker
`failed` / `reported` / `mapped`	Response structures: failed keys, messages, mapping of `%cid` to new keys

```
" Using a RAP BO from any ABAP code (e.g. a report)
MODIFY ENTITIES OF zi_travel
  ENTITY Travel
  CREATE FIELDS ( CustomerID BeginDate EndDate CurrencyCode )
  WITH VALUE #( ( %cid = 'T1' CustomerID = '000001'
                  BeginDate = '20261101' EndDate = '20261110' CurrencyCode = 'EUR' ) )
  MAPPED DATA(ls_mapped) FAILED DATA(ls_failed) REPORTED DATA(ls_reported).

COMMIT ENTITIES RESPONSE OF zi_travel
  FAILED DATA(ls_commit_failed) REPORTED DATA(ls_commit_reported).
```

SEGW (classic) vs. RAP – interview comparison

SEGW / Gateway	RAP
Model in SEGW, code in DPC_EXT/MPC_EXT	Model in CDS, behavior in BDEF + behavior pool
You implement query options manually (or use RDS)	SADL handles queries automatically
OData V2 mainly	OData V2 and **V4**, UI and Web API
Transaction handling: you (BAPIs, COMMIT)	Transactional buffer, save sequence, locking, ETag, draft by framework
SAP GUI based	ADT (Eclipse) only
Still used for legacy/complex custom logic	Strategic for S/4HANA and BTP (Clean Core)

> **Practical Lab: Lab 21 – RAP student app**

Using ADT: create table `ZSTUDENT_A` (UUID key + admin fields), root view entity `ZI_StudentR`, managed BDEF with: create/update/delete, managed UUID numbering, determination that sets `Status = 'N'` on create, validation that the e-mail contains '@', action `activate` that sets `Status = 'A'`. Create projection, service definition, **OData V4 UI** binding, publish and preview the Fiori app. Then test the same service with an HTTP client: `GET .../Student?$top=2&$count=true`.

Solution: Lab 21 (behavior part)

```
managed implementation in class zbp_i_studentr unique;
strict ( 2 );
define behavior for ZI_StudentR alias Student
persistent table zstudent_a
lock master
authorization master ( instance )
etag master LocalLastChangedAt
{
  create; update; delete;
  field ( numbering : managed, readonly ) StudentUUID;
  field ( readonly ) Status;
  action activate result [1] $self;
  determination setStatusNew on modify { create; }
  validation validateEmail on save { create; field Email; }
  mapping for zstudent_a corresponding;
}
```

```
METHOD setstatusnew.
  MODIFY ENTITIES OF zi_studentr IN LOCAL MODE
    ENTITY Student UPDATE FIELDS ( Status )
    WITH VALUE #( FOR k IN keys ( %tky = k-%tky Status = 'N' ) ).
ENDMETHOD.

METHOD validateemail.
  READ ENTITIES OF zi_studentr IN LOCAL MODE
    ENTITY Student FIELDS ( Email ) WITH CORRESPONDING #( keys )
    RESULT DATA(lt_students).
  LOOP AT lt_students INTO DATA(ls) WHERE Email NS '@'.
    APPEND VALUE #( %tky = ls-%tky ) TO failed-student.
    APPEND VALUE #( %tky = ls-%tky
                    %msg = new_message_with_text( text = 'Invalid e-mail address' )
                    %element-Email = if_abap_behv=>mk-on ) TO reported-student.
  ENDLOOP.
ENDMETHOD.

METHOD activate.
  MODIFY ENTITIES OF zi_studentr IN LOCAL MODE
    ENTITY Student UPDATE FIELDS ( Status )
    WITH VALUE #( FOR k IN keys ( %tky = k-%tky Status = 'A' ) ).
  READ ENTITIES OF zi_studentr IN LOCAL MODE
    ENTITY Student ALL FIELDS WITH CORRESPONDING #( keys ) RESULT DATA(lt).
  result = VALUE #( FOR s IN lt ( %tky = s-%tky %param = s ) ).
ENDMETHOD.
```

`mapping for zstudent_a corresponding;` works only when the CDS element names equal the table field names (case is ignored) – otherwise list the mapping explicitly as in the Travel example. Because of `authorization master ( instance )` you also need an (empty) `get_instance_authorizations` method, exactly as in the Travel example.

22. Consuming OData Services

From SAPUI5 (Fiori)

```
// manifest.json - data source and model
"dataSources": {
  "mainService": { "uri": "/sap/opu/odata/sap/ZSALES_SRV/", "type": "OData",
                   "settings": { "odataVersion": "2.0" } } },
"models": { "": { "dataSource": "mainService" } }

// Controller (JavaScript) - V2 model
const oModel = this.getView().getModel();
oModel.read("/OrderSet", {
  urlParameters: { "$expand": "ToItems", "$top": "20" },
  filters: [ new sap.ui.model.Filter("Customer", "EQ", "100001") ],
  success: (oData) => console.log(oData.results),
  error:   (oErr)  => sap.m.MessageBox.error(oErr.message)
});
oModel.create("/OrderSet", { Customer: "100001", Currency: "EGP" }, { success: ... });
oModel.update("/OrderSet('4711')", { Customer: "100002" });
oModel.remove("/OrderSet('4711')");
oModel.callFunction("/ReleaseOrder", { method: "POST", urlParameters: { OrderId: "4711" } });
```

The V2 ODataModel fetches the **CSRF token**, bundles requests into **$batch**, and handles ETags automatically.

From Postman / any HTTP client – the CSRF dance

1. `GET /sap/opu/odata/sap/ZSALES_SRV/` with header `X-CSRF-Token: Fetch` (and Basic Auth). Response header contains the token + session cookies.
2. `POST/PUT/DELETE` with header `X-CSRF-Token: <token>` and the same cookies. Without it you get **403 – CSRF token validation failed**.

From ABAP (consume a remote OData service)

Option A – generic HTTP client:

```
DATA lo_client TYPE REF TO if_http_client.
cl_http_client=>create_by_destination( EXPORTING destination = 'Z_REMOTE_ODATA'    " SM59
                                       IMPORTING client      = lo_client ).
lo_client->request->set_method( if_http_request=>co_request_method_get ).
cl_http_utility=>set_request_uri( request = lo_client->request
                                  uri     = '/sap/opu/odata/sap/ZSALES_SRV/OrderSet?$format=json&
$top=5' ).
lo_client->send( ).
lo_client->receive( ).
lo_client->response->get_status( IMPORTING code = DATA(lv_code) ).
DATA(lv_json) = lo_client->response->get_cdata( ).
lo_client->close( ).

" parse JSON into ABAP
TYPES: BEGIN OF ty_order, orderid TYPE string, customer TYPE string, END OF ty_order.
DATA: BEGIN OF ls_resp,
        BEGIN OF d,
          results TYPE STANDARD TABLE OF ty_order WITH EMPTY KEY,
        END OF d,
      END OF ls_resp.
```

```
/ui2/cl_json=>deserialize( EXPORTING json = lv_json
                           CHANGING  data = ls_resp ).
```

Option B – **OData Client Proxy** (`/IWBEP/CL_CP_FACTORY_REMOTE` / `CL_WEB_ODATA_CLIENT_FACTORY` on BTP) with a **Service Consumption Model** generated in ADT from the `$metadata` – typed, recommended in S/4HANA Cloud and BTP.

OData in SAP integration

- **SAP Integration Suite (CPI)** – OData receiver/sender adapters.
- **S/4HANA standard APIs** – published on the **SAP Business Accelerator Hub** (api.sap.com), e.g. `API_SALES_ORDER_SRV` (V2), `api_salesorder` (V4). Prefer them over custom services when available (Clean Core).

23. Performance, Security and Troubleshooting

Performance checklist

- Push filter, sorting and paging **to the database** (`WHERE`, `ORDER BY`, `UP TO ... OFFSET`) – or use CDS/RAP so SADL does it.
- Never `SELECT *` – select only fields of the entity, honour `$select` for wide entities.
- Avoid the N+1 problem with `$expand` → implement `GET_EXPANDED_ENTITYSET` or use CDS associations.
- Always enforce a **maximum page size** if `$top` is missing.
- Implement `$count` / `$inlinecount` with `SELECT COUNT(*)`, not by reading all rows.
- Use `/IWFND/TRACES` and the *Performance Trace* (`sap-statistics=true` in the URL → response header shows gateway / backend / DB time).
- Use `ST05` (SQL trace) and `SAT` (runtime analysis) on the backend for slow methods.
- Metadata cache: after model changes clear `/IWFND/CACHE_CLEANUP` and `/IWBEP/CACHE_CLEANUP` – but keep caches active in production.

Security checklist

- Users need authorization object **`S_SERVICE`* *for the service (add via role in PFCG – *Authorization Default → TADIR Service → IWSG / IWSV*) plus `S_RFC` (hub) and the **business authorizations** inside your code (`AUTHORITY-CHECK` in DPC or DCL in CDS).
- CSRF protection is on by default for modifying requests – don't switch it off.
- Validate all input – never build dynamic WHERE clauses from raw strings (SQL injection); use `cl_abap_dyn_prg=>escape_quotes` / the framework's converted where clause.
- Don't return more fields than needed (e.g. salary data) – the client can see everything in the payload.
- Use HTTPS and SSO (SAML/X.509) in productive landscapes.

Troubleshooting guide

Symptom	Where to look / typical cause
404 *Service not found*	Service not registered/activated in `/IWFND/MAINT_SERVICE`; ICF node inactive (SICF); wrong system alias
403 *Forbidden*	Missing CSRF token or `S_SERVICE`/role; check `SU53` for the user
500 *Internal Server Error*	Short dump → `ST22`; `/IWFND/ERROR_LOG` and `/IWBEP/ERROR_LOG` (call stack + *Replay*)
501 *Not implemented*	The DPC_EXT method for that operation is not redefined
`$metadata` shows old model	Clear metadata cache (`/IWFND/CACHE_CLEANUP`, `/IWBEP/CACHE_CLEANUP`)
Expanded data empty	Deep structure component name ≠ navigation property name; `et_expanded_tech_clauses` not filled
Filter ignored	Filter not implemented in GET_ENTITYSET, or complex filter not in select-options
Debugging	External breakpoint (for the service user) in DPC_EXT method; then call from browser/Gateway Client/Fiori

Practical Lab: Lab 23 – Debug and analyse

1) Set an external breakpoint in your `STUDENTSET_GET_ENTITYSET`, call the service from the browser and step through. 2) Call `...StudentSet('99999999')` and find the entry in `/IWFND/ERROR_LOG`; use *Replay*. 3) Call `...StudentSet?sap-statistics=true` and read the `sap-statistics` response header. 4) Remove `S_SERVICE` from a test user and observe the 403 + `SU53`.

Solution: Lab 23 – what you should observe

- The debugger stops in the DPC_EXT method with `io_tech_request_context` filled – inspect `get_filter( )`, `get_top( )`.
- The error log shows the 404 `resource_not_found` exception with the entity type; Replay opens the Gateway Client with the same request.
- `sap-statistics` shows `total`, `fw` (framework), `app` (your code), `gwtotal` times in ms – use it to prove where time is spent.
- SU53 shows the failed check on `S_SERVICE` with the service hash value.

Part III — Interview Questions

ABAP Objects and OData questions with model answers,
grouped Beginner → Intermediate → Advanced, plus scenario questions

24. ABAP Objects – Interview Questions

> **Tip: How to answer**
> Give a one-line definition, then a concrete example from SAP or from your own project, then (if relevant) a pitfall. Interviewers care more about *why* and *when* than about memorised definitions.

Beginner

1. What is the difference between a class and an object?

A class is the blueprint (definition of attributes and methods); an object is a runtime instance of that class, created with `NEW` or `CREATE OBJECT`, living in memory and accessed through a reference variable. One class can have many objects, each with its own instance attribute values.

2. What are the visibility sections in ABAP classes?

PUBLIC (accessible everywhere), PROTECTED (class + subclasses), PRIVATE (only the class and its friends). They must be declared in this order. Interfaces' components are always public.

3. What is the difference between instance and static components?

Instance components (`DATA`, `METHODS`, `EVENTS`) exist once per object and are accessed with `->`. Static components (`CLASS-DATA`, `CLASS-METHODS`, `CLASS-EVENTS`) exist once per class per internal session, are shared by all objects and accessed with `=>`. Static methods cannot access instance attributes or `me`.

4. What is a constructor? How many types exist?

A special method executed automatically on creation. The instance constructor `constructor` runs once per object (only IMPORTING parameters and exceptions). The static constructor `class_constructor` runs once per class per internal session before the first access, has no parameters.

5. What is `me`?

The self-reference inside instance methods – points to the current object. Used to resolve name conflicts (`me->mv_name = mv_name`) or to pass the current object to another method (`lo_x->register( me )`).

6. What is the difference between local and global classes?

Global classes are stored in the class library (SE24/ADT) and usable system-wide; local classes are defined inside a program/include and visible only there. Local classes are often used for helpers and for unit tests (test classes are always local).

7. What are the parameter types of a method?

IMPORTING, EXPORTING, CHANGING, RETURNING (one, always by value – makes a functional method), and RAISING for class-based exceptions (or EXCEPTIONS for classic ones).

8. What is a functional method?

A method with a RETURNING parameter. It can be used directly in expressions and operand positions: `IF lo_order->is_released( ) = abap_true.` or `lv_total = lo_calc->sum( ) * 2.`

9. What is the difference between `NEW` and `CREATE OBJECT`?

Both create an object. `NEW` (7.40+) is a constructor expression that returns the reference, allows inline declaration and chaining (`NEW zcl_x( )->run( )`). `CREATE OBJECT` is the older statement; it is still the way to create objects of a class whose name is only known at runtime: `CREATE OBJECT lo TYPE (lv_class)`.

10. What happens if you call a method on an initial reference?

Runtime error `OBJECTS_OBJREF_NOT_ASSIGNED` (exception `CX_SY_REF_IS_INITIAL`). Always check `IS BOUND` when a reference might not be set.

11. What is encapsulation? Give an example.

Hiding internal state and allowing changes only through methods that enforce rules. Example: a `ZCL_BANK_ACCOUNT` has a private balance and a `withdraw` method that refuses to go below zero; nobody can set the balance directly.

12. What does READ-ONLY mean for an attribute?

A public attribute declared READ-ONLY can be read from outside but only changed inside the class (and its subclasses for protected). It avoids writing a getter for simple values.

Intermediate

13. What is inheritance and which kind does ABAP support?

A subclass takes over all components of its superclass and can add new ones or redefine instance methods. ABAP supports single inheritance only (one direct superclass); multiple contracts are realised with interfaces. All classes ultimately inherit from `OBJECT`.

14. What can and cannot be redefined?

Public and protected **instance** methods can be redefined (signature unchanged). You cannot redefine static methods, constructors (a subclass defines its own), private methods, or FINAL methods. Attributes cannot be redefined.

15. Why must the subclass constructor call `super->constructor( )`?

To initialise the superclass part of the object. If the superclass has an explicit instance constructor, the compiler forces the call before instance components are accessed in the subclass constructor.

16. What is the difference between an abstract class and an interface?

An abstract class can contain implemented methods, state (attributes), constructors and visibility sections; a class can inherit from only one. An interface only declares a public contract (no implementation, apart from DEFAULT IGNORE/FAIL), and a class can implement many. Use an abstract class for an "is-a" family with shared code, an interface for a capability/contract and loose coupling.

17. What is polymorphism? Give an ABAP example.

The same call executes different implementations depending on the object's dynamic type. Example: a table of `REF TO zif_payment` containing cash, card and wallet objects; looping and calling `pay( )` runs each class's own logic without any CASE on type.

18. Explain up-cast and down-cast.

Up-cast (narrowing cast) assigns a subclass/implementing-class reference to a superclass/interface reference – implicit with `=`, always safe. Down-cast (widening cast) assigns a general reference to a more specific one – requires `?=` or `CAST`, checked at runtime, raises `CX_SY_MOVE_CAST_ERROR` if the object is not compatible. Use `IS INSTANCE OF` or `CASE TYPE OF` to check first.

19. What are static type and dynamic type of a reference?

The static type is the declared type (`TYPE REF TO zcl_shape`), fixed at compile time and defines which components you can address. The dynamic type is the class of the object it currently points to; it determines which implementation runs (dynamic binding).

20. What is an alias in an interface context?

`ALIASES name FOR intf~method` gives an interface component a short name in the implementing class, so callers can write `lo->print( )` instead of `lo->zif_printable~print( )`. Also used when composing interfaces.

21. Explain events in ABAP Objects – the four steps.

Declare (`EVENTS`/`CLASS-EVENTS` with EXPORTING parameters), raise (`RAISE EVENT`), define a handler method (`FOR EVENT evt OF class IMPORTING ... sender`), register at runtime (`SET HANDLER h FOR obj` / `FOR ALL INSTANCES`; static events without FOR). Handlers run synchronously; without registered handlers nothing happens.

22. What is the implicit `sender` parameter?

Every instance event automatically passes a reference to the object that raised it. The handler can import `sender` to know which object triggered the event (e.g. which ALV grid was double-clicked).

23. What are the three exception class categories?

`CX_STATIC_CHECK` (must be handled or declared; checked by compiler – expected business errors), `CX_DYNAMIC_CHECK` (checked only at runtime; caller can prevent them – e.g. `CX_SY_ZERODIVIDE`), `CX_NO_CHECK` (can occur anywhere, not declared – e.g. resource shortages). All inherit from `CX_ROOT`.

24. What is the purpose of CLEANUP?

The CLEANUP block runs when an exception raised in the TRY block is **not** caught by this TRY but propagates to an outer handler. It is used to restore a consistent state (release locks, close files) before leaving the context.

25. What is the difference between RETRY and RESUME?

RETRY (in CATCH) re-executes the whole TRY block – after fixing the cause. RESUME continues processing right after the point where a **resumable** exception was raised (`RAISE RESUMABLE EXCEPTION`, method declared with `RAISING RESUMABLE(...)`, handled with `CATCH BEFORE UNWIND`).

26. How do you attach a message class text to an exception class?

Implement `IF_T100_MESSAGE` (or `IF_T100_DYN_MSG`), define text-ID constants with msgid/msgno and attribute names for placeholders `&1..&4`, and raise with `RAISE EXCEPTION TYPE zcx_x EXPORTING textid = zcx_x=>not_found attr = value` (or `RAISE EXCEPTION TYPE zcx_x MESSAGE e001(zsd) WITH lv_vbeln` with IF_T100_DYN_MSG).

27. What is a friend class?

A class granted access to the protected and private components of another class via `FRIENDS`. Friendship is one-way. Typical use: test classes (`FRIENDS ltc_test`) or a factory creating instances of a CREATE PRIVATE class.

28. What do CREATE PUBLIC / PROTECTED / PRIVATE control?

Who may instantiate the class: anyone / the class and subclasses / only the class itself (and friends). CREATE PRIVATE is used for Singleton and for forcing creation through a factory method.

Advanced

29. Implement a Singleton in ABAP. What is its scope?

`CLASS ... CREATE PRIVATE`, a private static attribute `go_instance`, and a static `get_instance( )` that creates the object on first call (`IF go_instance IS NOT BOUND`). Its scope is one internal session – not shared across users or programs (use shared memory for that).

30. Which design patterns have you used in ABAP?

Example answer: *Factory* to create the right document handler per document type read from a customizing table (dynamic `CREATE OBJECT TYPE (lv_class)`); *Strategy* for country-specific tax calculation; *Singleton* for an application log wrapper; *Observer* via events for status changes; *Facade* class wrapping BAPI calls + commit; *MVC* in reports (model/view/controller classes); *Decorator* for adding logging/caching around a repository interface.

31. How do you write unit tests in ABAP?

Local test classes `FOR TESTING` with `RISK LEVEL` and `DURATION`, fixture methods (`class_setup`, `setup`, `teardown`, `class_teardown`), test methods `FOR TESTING`, assertions from `CL_ABAP_UNIT_ASSERT`. Run with Ctrl+Shift+F10, check coverage. Isolate dependencies with interfaces + test doubles (manual or `CL_ABAP_TESTDOUBLE`), `CL_OSQL_TEST_ENVIRONMENT`/`CL_CDS_TEST_ENVIRONMENT` for DB, and test seams for legacy code.

32. What is dependency injection and why is it important in ABAP?

Passing collaborators (e.g. a repository or notifier as `REF TO zif_*`) into an object from outside (constructor/setter/parameter) instead of creating them inside. It decouples classes and makes unit tests possible by injecting test doubles.

33. What are test seams?

`TEST-SEAM name ... END-TEST-SEAM` marks code in production code that can be replaced in unit tests by `TEST-INJECTION name ... END-TEST-INJECTION` in a test class. A last resort for legacy code that cannot be refactored to use DI.

34. What is RTTS? Where have you used it?

Run Time Type Services: RTTI (describe types at runtime: `CL_ABAP_TYPEDESCR=>describe_by_data/name`, `CL_ABAP_STRUCTDESCR`, `CL_ABAP_CLASSDESCR`) and RTTC (create types at runtime: `CL_ABAP_TABLEDESCR=>create` + `CREATE DATA ... TYPE HANDLE`). Used for generic uploads/downloads, dynamic ALV, generic table display, field catalog building.

35. How do you call a method dynamically?

`CALL METHOD lo_obj->(lv_method_name) EXPORTING ...` or with `PARAMETER-TABLE` (`ABAP_PARMBIND_TAB`) and `EXCEPTION-TABLE`. Static: `CALL METHOD (lv_class)=>(lv_method)`. Errors raise `CX_SY_DYN_CALL_*` exceptions.

36. What is a BAdI and how is it related to OO?

A Business Add-In is an OO enhancement: SAP defines an interface and calls implementations at defined points. Classic BAdIs (SE18/SE19, `CL_EXITHANDLER`), new kernel BAdIs inside enhancement spots (`GET BADI`/`CALL BADI`, filters, fallback class). Your implementation is a class implementing the BAdI interface.

37. What is the difference between `CX_STATIC_CHECK` and `CX_DYNAMIC_CHECK` in terms of design?

Use static check for errors the caller must consciously handle (business conditions); use dynamic check when the caller can avoid the error by checking preconditions (invalid argument) – forcing every caller to handle it would clutter code.

38. Explain SOLID in an ABAP context.

Single responsibility (repository vs. calculator vs. view class), Open/closed (add new strategy classes instead of editing CASE), Liskov (subclasses fully usable as superclass), Interface segregation (small focused interfaces), Dependency inversion (depend on interfaces, inject implementations).

39. Why are static methods considered harmful for business logic?

They cannot be redefined or replaced by test doubles (no polymorphism), so code calling them is hard to test and extend. Use instances + interfaces; static methods are fine for factories and pure utilities.

40. What happens with a superclass constructor that calls a redefined method?

During the superclass constructor the object is not yet a complete subclass instance, so the **superclass** implementation is executed, not the redefinition. A common source of bugs.

41. How does garbage collection work in ABAP?

Objects that are no longer referenced by any reference variable (including event handler registrations and internal tables) are deleted by the garbage collector automatically. `CLEAR`/`FREE` of the last reference makes an object eligible. Weak references (`CL_ABAP_WEAK_REFERENCE`) don't keep objects alive.

42. What is AMDP?

ABAP Managed Database Procedures – methods of a class implementing `IF_AMDP_MARKER_HDB`, declared `BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT USING <tables>`, executed on HANA. Used for complex calculations close to the data (code pushdown); also as implementation of CDS table functions.

43. What is the difference between `?=` and `CAST`?

Same down-cast semantics. `?=` is a statement; `CAST type( ref )` is a constructor expression usable inline (e.g. `CAST zcl_wallet( lo_pay )->get_cashback( )`), and `CAST #( )` infers the type from the target.

44. Can an interface have attributes and events? Can it be instantiated?

Yes, interfaces can declare constants, attributes (discouraged), types, events and methods. They cannot be instantiated, but reference variables `TYPE REF TO zif_x` can point to any implementing object.

45. How do you persist objects?

Usually via a repository/DAO class with Open SQL, or in modern development with RAP business objects. Classic Object Services (persistent classes with `CA_`/`CB_` agents) exist but are rarely used today.

25. OData & SAP Gateway – Interview Questions

Beginner

1. What is OData?

An OASIS standard REST-based protocol for querying and updating data over HTTP, using URIs for resources, HTTP methods for CRUD, a metadata document describing the model, and JSON/XML payloads. SAP uses it as the main interface for Fiori/UI5 and APIs.

2. What is SAP Gateway?

The ABAP platform component (SAP_GWFND) that exposes OData services: tools (SEGW), runtime (hub/frontend IWFND and backend IWBEP), registry, logs and caching.

3. What are entity type, entity set, property, association and navigation property?

Entity type = structure with key (like a table definition); entity set = collection of entities addressed in the URL; property = field; association = relationship with cardinality between two entity types; navigation property = named path to follow an association (`/OrderSet('1')/ToItems`).

4. Which HTTP methods map to which DPC methods?

GET collection → GET_ENTITYSET, GET single → GET_ENTITY, POST → CREATE_ENTITY (or CREATE_DEEP_ENTITY), PUT/PATCH/MERGE → UPDATE_ENTITY (PATCH_ENTITY), DELETE → DELETE_ENTITY. Function imports → EXECUTE_ACTION; media → GET_STREAM/CREATE_STREAM/UPDATE_STREAM.

5. Which classes does SEGW generate and which ones do you change?

MPC + MPC_EXT (model provider), DPC + DPC_EXT (data provider), plus the technical model and service. You change only the `_EXT` classes; base classes are regenerated and overwritten.

6. How do you register and test a service?

`/IWFND/MAINT_SERVICE` → Add Service → system alias (LOCAL for embedded) → select technical service → package. Test with SAP Gateway Client (`/IWFND/GW_CLIENT`) – first `$metadata`, then the entity sets.

7. What is `$metadata`?

The service metadata document (CSDL/EDMX XML) that describes entity types, sets, properties with types and facets, associations, function imports and annotations. Clients like UI5 build their model from it.

8. What is the service document?

The root URL of the service (`.../ZSALES_SRV/`), listing all entity sets (collections) available.

9. Name the main system query options.

`$filter`, `$orderby`, `$top`, `$skip`, `$select`, `$expand`, `$format`, `$inlinecount` (V2) / `$count` (V4), `/$count`, `$skiptoken`, `search`/`$search`.

10. What are the main transactions for Gateway?

SEGW, /IWFND/MAINT_SERVICE, /IWFND/GW_CLIENT, /IWFND/ERROR_LOG, /IWBEP/ERROR_LOG, /IWFND/TRACES, /IWFND/CACHE_CLEANUP, /IWBEP/CACHE_CLEANUP, SICF, /IWFND/V4_ADMIN.

Intermediate

11. Hub vs. embedded deployment?

Embedded: Gateway and backend in one system (standard for S/4HANA, no RFC). Hub: separate frontend server with UI5/FLP and Gateway hub connected via trusted RFC to backends; system alias routes the service. Hub allows one UI layer for multiple backends and separate upgrade cycles.

12. How do you implement $filter in GET_ENTITYSET?

Use `io_tech_request_context->get_filter( )->get_filter_select_options( )` to build ranges for the SELECT, or `get_osql_where_clause_convert( )` for a dynamic WHERE using ABAP names. For non-convertible filters read `iv_filter_string`. With CDS/RDS or RAP, SADL does it automatically.

13. How do you implement paging and why push it to the DB?

Read `is_paging-top/skip` or `get_top( )/get_skip( )` and use `UP TO n ROWS OFFSET m` with `ORDER BY` in the SELECT. Doing it in ABAP (`/iwbep/cl_mgw_data_util=>paging`) still reads all rows from the DB – slow and memory heavy for big tables.

14. What is the difference between `$expand` and navigation?

Navigation (`/OrderSet('1')/ToItems`) returns only the related entities. `$expand=ToItems` returns the main entity **plus** the related ones inline in one response, avoiding a second round trip.

15. How does the framework handle $expand and how do you optimise it?

By default it calls GET_ENTITYSET for the parent and then the child entity set method once per parent row (N+1 calls). Optimise by redefining GET_EXPANDED_ENTITYSET / GET_EXPANDED_ENTITY, reading children in one SELECT (`FOR ALL ENTRIES` or join), building a deep structure whose child component is named like the navigation property, and filling `et_expanded_tech_clauses`.

16. What is deep insert?

Creating a parent and its children in one POST with nested JSON (`"ToItems":[...]`). Implemented in CREATE_DEEP_ENTITY by reading the payload into a deep structure with `io_data_provider->read_entry_data`.

17. What is $batch and what is a changeset?

`$batch` bundles multiple requests into one HTTP POST (multipart). A changeset groups modifying requests that must succeed or fail together (one LUW). Implement CHANGESET_BEGIN (allow multiple ops, defer mode), CHANGESET_PROCESS (process together) and CHANGESET_END (commit). Don't COMMIT inside individual CRUD methods when in a changeset.

18. What is a function import? GET or POST?

A custom operation outside CRUD (e.g. ReleaseOrder, CalculatePrice) defined in SEGW with parameters and return type (entity, complex type or none). Implemented in EXECUTE_ACTION. Use GET for read-only, POST for operations with side effects. In V4 they are split into functions (read-only) and actions (modifying).

19. How do you return error messages to the UI?

Raise `/IWBEP/CX_MGW_BUSI_EXCEPTION` (HTTP 400, or set `http_status_code`) with a message or a message container filled via `mo_context->get_message_container( )` (`add_message`, `add_message_text_only`, `add_messages_from_bapi`). Technical errors: `/IWBEP/CX_MGW_TECH_EXCEPTION` (500). Warnings with success go in the `sap-message` header.

20. What is the CSRF token and why is it needed?

A token protecting against Cross-Site Request Forgery. The client fetches it with `X-CSRF-Token: Fetch` on a GET and sends it in modifying requests (POST/PUT/PATCH/DELETE). Missing/invalid token → 403. UI5 does it automatically.

21. What is an ETag?

A version identifier (e.g. last-changed timestamp) used for optimistic locking. The client sends it in `If-Match`; if it differs from the current value, the server returns 412 Precondition Failed, preventing lost updates. Defined per entity type in SEGW (property as ETag) or `etag master` in RAP.

22. How do you handle media/file upload and download?

Mark the entity type as media, set the content-type property in MPC_EXT, implement CREATE_STREAM (upload; file name from `slug` header) and GET_STREAM (download via `/$value`), returning `ty_s_media_resource` with value (XSTRING) and mime_type.

23. What is the difference between PUT and PATCH/MERGE?

PUT replaces the whole entity (missing fields may be initialised); PATCH (V2 also MERGE) updates only the sent fields. Gateway's default PATCH implementation reads the entity via GET_ENTITY, merges and calls UPDATE_ENTITY.

24. What is `io_tech_request_context` and why use it?

The request context object passed to DPC methods. It provides keys, filter, select, paging, navigation and expand information with **technical ABAP names** (independent of the external OData property names), e.g. `get_converted_keys`, `get_filter`, `get_top`, `get_expand`, `get_source_entity_type_name`.

25. How do you read the keys in GET_ENTITY?

`io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls_key )` where `ls_key` is the entity structure; or loop `it_key_tab` (name/value pairs, external names). For navigation use `get_converted_source_keys`.

Advanced

26. How do you expose a CDS view as OData? Compare the options.

(1) `@OData.publish: true` – quick, read-only unless BOPF is used; (2) SEGW Reference Data Source – SADL handles reads, custom write logic in DPC_EXT; (3) RAP service definition + binding – modern, full transactional support, V2/V4. Prefer RAP for new development on S/4HANA ≥ 1909 or BTP.

27. What is SADL?

Service Adaptation Definition Language – the framework that maps OData requests on CDS-based entities into optimised SQL (filter, sort, paging, count, search, expand) and handles authorizations from DCL. It is the engine behind RDS, @OData.publish and RAP queries.

28. Explain the RAP architecture.

Data model with CDS view entities (root + compositions) → behavior definition (operations, numbering, validations, determinations, actions, locking, ETag, draft) → behavior pool class (handler and saver classes, EML) → projection layer (views + projection BDEF) → service definition → service binding (OData V2/V4, UI/Web API).

29. Managed vs. unmanaged RAP?

Managed: framework provides CRUD, transactional buffer, locking and saving – you add business logic. Unmanaged: you implement create/update/delete/read/lock and the save sequence yourself – used to wrap existing legacy code/BAPIs. Hybrid: managed with unmanaged/additional save.

30. What are determinations and validations in RAP?

Determinations compute/derive values when trigger conditions occur (`on modify` or `on save`, triggered by create/field changes). Validations check data consistency `on save` and reject invalid instances by filling `failed` and `reported`.

31. What is EML?

Entity Manipulation Language – ABAP statements to access RAP BOs: `READ ENTITIES`, `MODIFY ENTITIES` (create, update, delete, execute action), `COMMIT ENTITIES`, `ROLLBACK ENTITIES`, with derived types `%tky`, `%cid`, `%param`, and response structures `failed`, `reported`, `mapped`.

32. What is draft in RAP?

`with draft;` enables storing in-progress changes in a draft table so users can leave and continue later, with exclusive locks per draft and the actions Edit/Activate/Discard/Resume/Prepare handled by the framework. Typical for Fiori elements object pages.

33. Differences between OData V2 and V4?

V4 has leaner JSON, `$count=true`, standard `$search`, nested expand options, actions vs. functions, better batch/delta, and is used by RAP V4 bindings and Fiori elements V4. V2 is still most common with SEGW and existing Fiori apps.

34. How do you consume an external OData service from ABAP?

Via HTTP client (`CL_HTTP_CLIENT` with SM59 destination + JSON parsing with `/UI2/CL_JSON` or `XCO`), or the typed OData client proxy (`/IWBEP/CL_CP_FACTORY_REMOTE`, Service Consumption Model in ADT; `CL_WEB_ODATA_CLIENT_FACTORY` on BTP).

35. How do you secure an OData service?

Role with `S_SERVICE` (service authorization), business authority checks in DPC or DCL access control on CDS, CSRF protection, HTTPS/SSO, input validation (no SQL injection in dynamic WHERE), expose only needed fields, limit page sizes.

36. How do you analyse a slow OData service?

`sap-statistics=true` to split framework/application/DB time, `/IWFND/TRACES` (performance trace), ST05 SQL trace and SAT on the backend, check for N+1 `$expand`, missing DB paging, SELECT *, missing indexes; use CDS/SADL push-down.

37. Where do you see Gateway errors?

`/IWFND/ERROR_LOG` (hub) and `/IWBEP/ERROR_LOG` (backend) with error context, call stack and replay; `ST22` for dumps; browser network tab; UI5 message popover.

38. What is soft state?

A mode where the backend session (and its ABAP memory/buffers) is kept alive between requests for a configurable timeout, enabled per service and by implementing `/IWBEP/IF_MGW_SOST_SRV_RUNTIME`. Improves performance for expensive initialisations, but costs memory.

39. How do you add annotations to a SEGW service?

In MPC_EXT->DEFINE via `create_annotation` on entity/property (SAP annotations like `sap:label`, `sap:filterable`), or with an annotation model (vocabulary-based) in SEGW, or – for CDS-based services – via CDS annotations / metadata extensions. Clear the metadata cache afterwards.

40. What is a system alias?

A configuration on the hub (`/IWFND/MAINT_SERVICE` + SPRO) that maps a service to a backend RFC destination. `LOCAL` is used in embedded deployment.

Scenario questions (senior level)

S1. A Fiori list takes 15 s to load. How do you approach it?

Reproduce with `sap-statistics=true` → see whether time is in app or DB. Check `/IWFND/TRACES` for number of calls (e.g. $expand N+1). In the backend use ST05/SAT on the GET_ENTITYSET: missing WHERE push-down, SELECT *, paging in ABAP, FOR ALL ENTRIES with empty table, missing index. Fix by pushing filter/paging to DB, implementing GET_EXPANDED_ENTITYSET, or moving the read to a CDS view with SADL. Add a maximum page size.

S2. The UI must create an order with 50 items atomically. Design it.

Deep insert (CREATE_DEEP_ENTITY) with header + items, one BAPI call, error messages via message container from BAPI RETURN, rollback on error and commit on success; alternatively $batch with one changeset in defer mode processed in CHANGESET_PROCESS. In RAP: composition root/child, `create by \_Items`, one MODIFY ENTITIES with `%cid` references.

S3. Two users edit the same record. How do you avoid lost updates?

ETag (last changed timestamp) on the entity; UI sends If-Match, backend returns 412 if outdated. Plus pessimistic locks (ENQUEUE) during the actual update. RAP: `etag master` + `lock master` (or draft with exclusive locks).

S4. You need a new API for an external system to read customer credit data. SEGW or RAP?

On S/4HANA 1909+ or BTP: RAP with a CDS-based read-only BO (or custom entity if data comes from logic) exposed as OData V4 Web API, with DCL authorizations. First check the SAP Business Accelerator Hub for a released standard API (Clean Core). SEGW only on older ECC/NetWeaver releases.

S5. How do you design OO code behind a Gateway service?

Keep DPC_EXT thin: map OData ↔ ABAP and translate exceptions. Put business logic into separate classes (service/API class, repository for DB access, validator) behind interfaces – reusable from reports, RFC and RAP, and unit-testable with test doubles.

Part IV — Capstone & Cheat Sheets

One end-to-end project combining ABAP OO + OData,
exercise bank and one-page revision sheets

26. Capstone Project: Leave Request Management

Build a small but realistic application end-to-end. It uses **every** concept of this guide and is a great story to tell in interviews ("I built a leave request service with a layered OO design, unit tests and an OData service consumed by Fiori").

Requirements

- Employees create leave requests (type ANNUAL / SICK / UNPAID, from date, to date, reason).
- Rules: end date ≥ start date; ANNUAL leave max 21 working days per year; SICK leave > 2 days requires an attachment; no overlapping requests.
- Managers approve or reject (with a comment). An e-mail/notification is triggered on status change.
- A Fiori list (or Postman) reads requests with filter by status/employee, paging and count; header shows employee name.
- Attachments (PDF/JPG) can be uploaded and downloaded.

Architecture (layers)

```
Fiori / Postman
     |  OData V2 (SEGW)   or   OData V4 (RAP)
     v
ZCL_ZLEAVE_DPC_EXT      <- thin: map OData <-> ABAP, convert exceptions
     |
ZCL_LEAVE_SERVICE       <- business logic (validations, approval), events
  |-- ZIF_LEAVE_REPOSITORY  -> ZCL_LEAVE_REPOSITORY (Open SQL on ZLEAVE_REQ)
  |-- ZIF_LEAVE_RULE (Strategy) -> ZCL_RULE_DATES, ZCL_RULE_QUOTA, ZCL_RULE_OVERLAP
  |-- ZIF_NOTIFIER          -> ZCL_MAIL_NOTIFIER (CL_BCS) / test double
ZCX_LEAVE_ERROR         <- static check exception with T100 messages (ZLEAVE)
LTC_* test classes      <- ABAP Unit with injected test doubles
```

Step 1 – Data model (SE11)

Table ZLEAVE_REQ field	Type	Note
MANDT	MANDT	key
REQ_ID	NUMC 10	key
PERNR	PERSNO	employee
LEAVE_TYPE	CHAR 10	ANNUAL / SICK / UNPAID
BEGDA / ENDDA	DATS	period
DAYS	INT4	calculated working days
STATUS	CHAR 1	N=new, A=approved, R=rejected
REASON / MGR_COMMENT	CHAR 255	texts
CHANGED_AT	TIMESTAMPL	ETag

Step 2 – Interfaces and exception

```
INTERFACE zif_leave_repository PUBLIC.
  TYPES ty_request  TYPE zleave_req.
  TYPES tt_requests TYPE STANDARD TABLE OF zleave_req WITH EMPTY KEY.
  METHODS read     IMPORTING iv_id TYPE zleave_req-req_id
                   RETURNING VALUE(rs) TYPE ty_request
                   RAISING   zcx_leave_error.
  METHODS by_employee IMPORTING iv_pernr TYPE persno
                      RETURNING VALUE(rt) TYPE tt_requests.
  METHODS save     IMPORTING is_request TYPE ty_request.
ENDINTERFACE.

INTERFACE zif_leave_rule PUBLIC.
  METHODS check IMPORTING is_request  TYPE zleave_req
                          it_existing TYPE zif_leave_repository=>tt_requests
                RAISING   zcx_leave_error.
ENDINTERFACE.

INTERFACE zif_notifier PUBLIC.
  METHODS notify IMPORTING is_request TYPE zleave_req.
ENDINTERFACE.
```

Step 3 – Rules (Strategy pattern)

```
CLASS zcl_rule_dates DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION. INTERFACES zif_leave_rule.
ENDCLASS.
CLASS zcl_rule_dates IMPLEMENTATION.
  METHOD zif_leave_rule~check.
    IF is_request-endda < is_request-begda.
      RAISE EXCEPTION TYPE zcx_leave_error
        EXPORTING textid = zcx_leave_error=>invalid_period.
    ENDIF.
  ENDMETHOD.
ENDCLASS.

CLASS zcl_rule_overlap DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION. INTERFACES zif_leave_rule.
ENDCLASS.
CLASS zcl_rule_overlap IMPLEMENTATION.
  METHOD zif_leave_rule~check.
    LOOP AT it_existing INTO DATA(ls_old)
         WHERE status <> 'R' AND req_id <> is_request-req_id
           AND begda <= is_request-endda AND endda >= is_request-begda.
      RAISE EXCEPTION TYPE zcx_leave_error
        EXPORTING textid = zcx_leave_error=>overlap
                  mv_req_id = ls_old-req_id.
    ENDLOOP.
  ENDMETHOD.
ENDCLASS.
```

`ZCL_RULE_QUOTA` sums `days` of approved ANNUAL requests of the year and compares with 21.

Step 4 – Service class with events

```
CLASS zcl_leave_service DEFINITION PUBLIC FINAL CREATE PUBLIC.
  PUBLIC SECTION.
    TYPES tt_rules TYPE STANDARD TABLE OF REF TO zif_leave_rule WITH EMPTY KEY.
    EVENTS status_changed EXPORTING VALUE(es_request) TYPE zleave_req.

    METHODS constructor IMPORTING io_repo  TYPE REF TO zif_leave_repository
                                  it_rules TYPE tt_rules
                                  io_notifier TYPE REF TO zif_notifier OPTIONAL.
    METHODS create  IMPORTING is_request TYPE zleave_req
                    RETURNING VALUE(rs)  TYPE zleave_req
                    RAISING   zcx_leave_error.
    METHODS approve IMPORTING iv_id TYPE zleave_req-req_id iv_comment TYPE string OPTIONAL
                    RETURNING VALUE(rs) TYPE zleave_req
                    RAISING   zcx_leave_error.
    METHODS reject  IMPORTING iv_id TYPE zleave_req-req_id iv_comment TYPE string
                    RETURNING VALUE(rs) TYPE zleave_req
                    RAISING   zcx_leave_error.
  PRIVATE SECTION.
    DATA mo_repo     TYPE REF TO zif_leave_repository.
    DATA mt_rules    TYPE tt_rules.
    DATA mo_notifier TYPE REF TO zif_notifier.
    METHODS set_status IMPORTING iv_id TYPE zleave_req-req_id iv_status TYPE char1
                                 iv_comment TYPE string
                       RETURNING VALUE(rs) TYPE zleave_req
                       RAISING   zcx_leave_error.
    METHODS working_days IMPORTING iv_from TYPE d iv_to TYPE d RETURNING VALUE(rv) TYPE i.
ENDCLASS.

CLASS zcl_leave_service IMPLEMENTATION.
  METHOD constructor.
    mo_repo = io_repo. mt_rules = it_rules. mo_notifier = io_notifier.
  ENDMETHOD.

  METHOD create.
    rs = is_request.
    rs-status = 'N'.
    rs-days   = working_days( iv_from = rs-begda iv_to = rs-endda ).
    DATA(lt_existing) = mo_repo->by_employee( rs-pernr ).
    LOOP AT mt_rules INTO DATA(lo_rule).          " polymorphic rule check
      lo_rule->check( is_request = rs it_existing = lt_existing ).
    ENDLOOP.
    GET TIME STAMP FIELD rs-changed_at.
    mo_repo->save( rs ).
  ENDMETHOD.

  METHOD approve.
    rs = set_status( iv_id = iv_id iv_status = 'A' iv_comment = iv_comment ).
  ENDMETHOD.

  METHOD reject.
    rs = set_status( iv_id = iv_id iv_status = 'R' iv_comment = iv_comment ).
  ENDMETHOD.

  METHOD set_status.
    rs = mo_repo->read( iv_id ).
    IF rs-status <> 'N'.
      RAISE EXCEPTION TYPE zcx_leave_error
        EXPORTING textid = zcx_leave_error=>already_decided mv_req_id = iv_id.
    ENDIF.
    rs-status      = iv_status.
    rs-mgr_comment = iv_comment.
```

```
    GET TIME STAMP FIELD rs-changed_at.
    mo_repo->save( rs ).
    RAISE EVENT status_changed EXPORTING es_request = rs.
    IF mo_notifier IS BOUND.
      mo_notifier->notify( rs ).
    ENDIF.
  ENDMETHOD.

  METHOD working_days.
    DATA(lv_day) = iv_from.
    WHILE lv_day <= iv_to.
      " 1 = Monday ... 7 = Sunday ; weekend here: Friday + Saturday
      DATA(lv_wd) = ( lv_day - CONV d( '19000101' ) ) MOD 7 + 1.
      IF lv_wd <> 5 AND lv_wd <> 6.
        rv += 1.
      ENDIF.
      lv_day += 1.
    ENDWHILE.
  ENDMETHOD.
ENDCLASS.
```

> **Note: Calendar**
> In a real system use the factory calendar (`DATE_CONVERT_TO_FACTORYDATE` / `CL_SCAL_UTILS` / HR work schedules) instead of the simple weekday formula. 01.01.1900 was a Monday, which is why the formula works.

Step 5 – Unit tests with test doubles

```
CLASS ltd_repo DEFINITION FOR TESTING.
  PUBLIC SECTION.
    INTERFACES zif_leave_repository.
    DATA mt_db TYPE zif_leave_repository=>tt_requests.
ENDCLASS.
CLASS ltd_repo IMPLEMENTATION.
  METHOD zif_leave_repository~read.
    rs = VALUE #( mt_db[ req_id = iv_id ] OPTIONAL ).
    IF rs IS INITIAL.
      RAISE EXCEPTION TYPE zcx_leave_error.
    ENDIF.
  ENDMETHOD.
  METHOD zif_leave_repository~by_employee.
    rt = VALUE #( FOR r IN mt_db WHERE ( pernr = iv_pernr ) ( r ) ).
  ENDMETHOD.
  METHOD zif_leave_repository~save.
    DELETE mt_db WHERE req_id = is_request-req_id.
    APPEND is_request TO mt_db.
  ENDMETHOD.
ENDCLASS.

CLASS ltc_leave_service DEFINITION FINAL FOR TESTING DURATION SHORT RISK LEVEL HARMLESS.
  PRIVATE SECTION.
    DATA mo_repo TYPE REF TO ltd_repo.
    DATA mo_cut  TYPE REF TO zcl_leave_service.
    METHODS setup.
    METHODS overlap_is_rejected  FOR TESTING.
    METHODS approve_sets_status  FOR TESTING RAISING cx_static_check.
    METHODS cannot_approve_twice FOR TESTING RAISING cx_static_check.
ENDCLASS.

CLASS ltc_leave_service IMPLEMENTATION.
```

```
  METHOD setup.
    mo_repo = NEW #( ).
    mo_cut  = NEW #( io_repo  = mo_repo
                     it_rules = VALUE #( ( NEW zcl_rule_dates( ) )
                                         ( NEW zcl_rule_overlap( ) ) ) ).
  ENDMETHOD.

  METHOD overlap_is_rejected.
    mo_repo->mt_db = VALUE #( ( req_id = '1' pernr = '100' status = 'A'
                                begda = '20261101' endda = '20261105' ) ).
    TRY.
        mo_cut->create( VALUE #( req_id = '2' pernr = '100'
                                 begda = '20261104' endda = '20261108' ) ).
        cl_abap_unit_assert=>fail( msg = 'Overlap not detected' ).
      CATCH zcx_leave_error.
    ENDTRY.
  ENDMETHOD.

  METHOD approve_sets_status.
    mo_repo->mt_db = VALUE #( ( req_id = '1' pernr = '100' status = 'N' ) ).
    DATA(ls) = mo_cut->approve( '1' ).
    cl_abap_unit_assert=>assert_equals( act = ls-status exp = 'A' ).
  ENDMETHOD.

  METHOD cannot_approve_twice.
    mo_repo->mt_db = VALUE #( ( req_id = '1' pernr = '100' status = 'A' ) ).
    TRY.
        mo_cut->approve( '1' ).
        cl_abap_unit_assert=>fail( msg = 'Second approval must fail' ).
      CATCH zcx_leave_error.
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

Step 6 – OData service (SEGW)

- Project `ZLEAVE`: entity `LeaveRequest` (set `LeaveRequestSet`) from `ZLEAVE_REQ`, ETag on `ChangedAt`; entity `Employee` (`EmployeeSet`, from PA0001/PA0002 or a CDS) with association `Employee 1 : n LeaveRequest` (`ToRequests`); media entity `Attachment` (key `ReqId`, `FileName`); function imports `Approve` (POST, ReqId, Comment → LeaveRequest) and `Reject`.
- DPC_EXT creates the service in a factory method and maps exceptions:

```
CLASS zcl_zleave_dpc_ext DEFINITION PUBLIC INHERITING FROM zcl_zleave_dpc CREATE PUBLIC.
  PUBLIC SECTION.
    METHODS /iwbep/if_mgw_appl_srv_runtime~execute_action REDEFINITION.
  PROTECTED SECTION.
    METHODS leaverequestset_create_entity REDEFINITION.
    METHODS leaverequestset_get_entityset REDEFINITION.
  PRIVATE SECTION.
    METHODS service RETURNING VALUE(ro) TYPE REF TO zcl_leave_service.
    METHODS to_gw_error IMPORTING ix TYPE REF TO zcx_leave_error
                        RAISING   /iwbep/cx_mgw_busi_exception.
ENDCLASS.

CLASS zcl_zleave_dpc_ext IMPLEMENTATION.
  METHOD service.
    ro = NEW #( io_repo     = NEW zcl_leave_repository( )
                it_rules    = VALUE #( ( NEW zcl_rule_dates( ) )
                                       ( NEW zcl_rule_overlap( ) )
                                       ( NEW zcl_rule_quota( ) ) )
                io_notifier = NEW zcl_mail_notifier( ) ).
```

```
  ENDMETHOD.

  METHOD to_gw_error.
    DATA(lo_msg) = mo_context->get_message_container( ).
    lo_msg->add_message_text_only( iv_msg_type = 'E'
                                   iv_msg_text = CONV #( ix->get_text( ) ) ).
    RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
      EXPORTING message_container = lo_msg previous = ix.
  ENDMETHOD.

  METHOD leaverequestset_create_entity.
    DATA ls_req TYPE zleave_req.
    io_data_provider->read_entry_data( IMPORTING es_data = er_entity ).
    ls_req = CORRESPONDING #( er_entity ).
    TRY.
        er_entity = CORRESPONDING #( service( )->create( ls_req ) ).
      CATCH zcx_leave_error INTO DATA(lx).
        to_gw_error( lx ).
    ENDTRY.
  ENDMETHOD.

  METHOD leaverequestset_get_entityset.
    DATA(lv_where) = io_tech_request_context->get_osql_where_clause_convert( ).
    DATA(lv_top)   = CONV i( io_tech_request_context->get_top( ) ).
    DATA(lv_skip)  = CONV i( io_tech_request_context->get_skip( ) ).
    IF lv_top = 0. lv_top = 200. ENDIF.
    SELECT * FROM zleave_req WHERE (lv_where)
      ORDER BY req_id DESCENDING
      INTO CORRESPONDING FIELDS OF TABLE @et_entityset
      UP TO @lv_top ROWS OFFSET @lv_skip.
    IF io_tech_request_context->has_inlinecount( ) = abap_true.
      SELECT COUNT(*) FROM zleave_req WHERE (lv_where) INTO @DATA(lv_cnt).
      es_response_context-inlinecount = lv_cnt.
    ENDIF.
  ENDMETHOD.

  METHOD /iwbep/if_mgw_appl_srv_runtime~execute_action.
    DATA(lv_id) = CONV zleave_req-req_id(
                    VALUE string( it_parameter[ name = 'ReqId' ]-value OPTIONAL ) ).
    DATA(lv_comment) = VALUE string( it_parameter[ name = 'Comment' ]-value OPTIONAL ).
    TRY.
        DATA(ls_req) = SWITCH zleave_req( iv_action_name
          WHEN 'Approve' THEN service( )->approve( iv_id = lv_id iv_comment = lv_comment )
          WHEN 'Reject'  THEN service( )->reject(  iv_id = lv_id iv_comment = lv_comment ) ).
        DATA(ls_entity) = CORRESPONDING zcl_zleave_mpc=>ts_leaverequest( ls_req ).
        copy_data_to_ref( EXPORTING is_data = ls_entity CHANGING cr_data = er_data ).
      CATCH zcx_leave_error INTO DATA(lx).
        to_gw_error( lx ).
    ENDTRY.
  ENDMETHOD.
ENDCLASS.
```

Step 7 – Test script (Gateway Client / Postman)

```
GET  /sap/opu/odata/sap/ZLEAVE_SRV/$metadata
POST /sap/opu/odata/sap/ZLEAVE_SRV/LeaveRequestSet
     {"Pernr":"00000100","LeaveType":"ANNUAL","Begda":"\/Date(1793404800000)\/",
      "Endda":"\/Date(1793750400000)\/","Reason":"Family trip"}
GET  /sap/opu/odata/sap/ZLEAVE_SRV/LeaveRequestSet?$filter=Status eq 'N'&$top=10&$inlinecount=all
pages
POST /sap/opu/odata/sap/ZLEAVE_SRV/Approve?ReqId='0000000001'&Comment='OK'
GET  /sap/opu/odata/sap/ZLEAVE_SRV/EmployeeSet('00000100')?$expand=ToRequests
POST /sap/opu/odata/sap/ZLEAVE_SRV/AttachmentSet   (binary body, header slug: sick_note.pdf)
GET  /sap/opu/odata/sap/ZLEAVE_SRV/AttachmentSet('0000000001')/$value
```

V2 JSON dates use the format `\/Date(<milliseconds since 1970>)\/` for `Edm.DateTime`.

Stretch goals

- Rebuild the same BO with **RAP** (managed, draft, actions `approve`/`reject`, validations = your rule classes reused) and an OData V4 UI binding – compare the amount of code.
- Add a Fiori elements List Report on top of the RAP service using CDS UI annotations.
- Add an ATC check run and reach > 80 % unit test coverage on `ZCL_LEAVE_SERVICE`.

27. Exercise Bank (extra practice)

Use these to keep practising after the labs. Each one should take 30–90 minutes.

ABAP Objects

1. **Library system** – classes Book, Member, Library; members can borrow max 3 books; raise `ZCX_LIBRARY` for violations; event `book_returned` notifies a waiting list.
2. **Shape area calculator** – abstract Shape; Circle, Rectangle, Triangle; sort a table of shapes by area. (Hint: `SORT ... BY table_line->attr` works only with public attributes, not methods – build a helper table of area + reference and sort that.)
3. **Generic table exporter** – a class that receives `ANY TABLE`, uses RTTI to build a CSV header and lines, returns a string table.
4. **Plug-in architecture** – read handler class names from table `ZHANDLERS` and instantiate them dynamically through an interface.
5. **Refactor a legacy report** – take an old report with FORMs and global data and rewrite it into model/view/controller classes with unit tests for the model.
6. **Application log wrapper** – singleton `ZCL_LOG` around `BAL_LOG_CREATE`, `BAL_LOG_MSG_ADD`, `BAL_DB_SAVE`, method `add_exception( ix )` that walks the `previous` chain.

OData

7. **Material master read service** – MARA/MAKT with language-dependent description, `$filter` on material type, search by description.
8. **Customer with addresses** – 1:n navigation, `$expand` optimised with GET_EXPANDED_ENTITYSET.
9. **Purchase requisition creation** – deep insert calling `BAPI_PR_CREATE`, messages from BAPI RETURN in the message container.
10. **Invoice PDF download** – media entity that returns a SmartForm/Adobe form converted to PDF in GET_STREAM.
11. **Batch update** – update 20 items in one `$batch` changeset with defer mode, single COMMIT in CHANGESET_END.
12. **RAP inventory app** – managed BO with draft, validation of stock quantity, action `reserve`, determination for total value, Fiori preview.

28. Cheat Sheets

ABAP Objects syntax

Need	Syntax
Define class	`CLASS lcl_x DEFINITION [FINAL] [ABSTRACT] [CREATE PRIVATE] [INHERITING FROM lcl_super] [FRIENDS lcl_f].`
Sections	`PUBLIC SECTION. PROTECTED SECTION. PRIVATE SECTION.`
Attributes	`DATA`, `CLASS-DATA`, `CONSTANTS`, `... READ-ONLY`
Methods	`METHODS m IMPORTING .. EXPORTING .. CHANGING .. RETURNING VALUE(r) .. RAISING cx_..`
Static method	`CLASS-METHODS m ...` → call `lcl_x=>m( )`
Constructors	`METHODS constructor IMPORTING ...` / `CLASS-METHODS class_constructor.`
Create object	`DATA(lo) = NEW lcl_x( p = 1 ).` / `CREATE OBJECT lo TYPE (lv_name).`
Inherit/redefine	`INHERITING FROM`, `METHODS m REDEFINITION.`, `super->m( )`
Abstract	`CLASS ... ABSTRACT`, `METHODS m ABSTRACT`
Interface	`INTERFACE lif. METHODS m. ENDINTERFACE.` / `INTERFACES lif.` / `METHOD lif~m.` / `ALIASES m FOR lif~m.`
Casting	up: `lo_super = lo_sub.` down: `lo_sub ?= lo_super.` / `CAST lcl_sub( lo )` / `IS INSTANCE OF` / `CASE TYPE OF`
Events	`EVENTS e EXPORTING VALUE(p) TYPE t.` → `RAISE EVENT e EXPORTING p = v.` → `METHODS h FOR EVENT e OF lcl IMPORTING p sender.` → `SET HANDLER lo->h FOR lo_pub.`
Exceptions	`TRY. ... CATCH cx_a cx_b INTO DATA(lx). ... CLEANUP. ... ENDTRY.` / `RAISE EXCEPTION TYPE zcx EXPORTING ...` / `RAISE EXCEPTION NEW zcx( )`
Unit test	`CLASS ltc DEFINITION FOR TESTING DURATION SHORT RISK LEVEL HARMLESS.` / `METHODS t FOR TESTING.` / `cl_abap_unit_assert=>assert_equals( act = .. exp = .. )`
RTTI	`cl_abap_typedescr=>describe_by_data( x )`, `cl_abap_classdescr=>get_class_name( lo )`

OData / Gateway quick reference

Need	Where / how
Create service	SEGW → project → import DDIC/RFC/CDS → Generate
Register	`/IWFND/MAINT_SERVICE` → Add Service (alias LOCAL)
Test	`/IWFND/GW_CLIENT`, Postman (`X-CSRF-Token: Fetch`)
Read keys	`io_tech_request_context->get_converted_keys( IMPORTING es_key_values = ls )`
Read body	`io_data_provider->read_entry_data( IMPORTING es_data = ls )`
Filter	`get_osql_where_clause_convert( )` / `get_filter( )->get_filter_select_options( )`

Need	Where / how
Paging	`get_top( )`, `get_skip( )`, `UP TO n ROWS OFFSET m`
Count	`has_inlinecount( )` → `es_response_context-inlinecount`
Navigation source	`get_source_entity_type_name( )`, `get_converted_source_keys( )`
Expand	Redefine `GET_EXPANDED_ENTITYSET`, fill `et_expanded_tech_clauses`
Deep insert	Redefine `CREATE_DEEP_ENTITY`
Function import	Redefine `EXECUTE_ACTION` (`iv_action_name`, `it_parameter`)
Media	`GET_STREAM` / `CREATE_STREAM`, `$value`, header `slug`
Batch	`CHANGESET_BEGIN` / `CHANGESET_PROCESS` / `CHANGESET_END`
Errors	`/IWBEP/CX_MGW_BUSI_EXCEPTION` + `mo_context->get_message_container( )`
Logs	`/IWFND/ERROR_LOG`, `/IWBEP/ERROR_LOG`, `ST22`, `/IWFND/TRACES`, `sap-statistics=true`
Cache	`/IWFND/CACHE_CLEANUP`, `/IWBEP/CACHE_CLEANUP`

RAP quick reference

Artifact	Keyword
Root entity	`define root view entity ZI_X as select from ztab`
Child entity	`composition [0..*] of ZI_Child as _Child` / `association to parent ZI_X as _Parent on ...`
BDEF	`managed implementation in class zbp_i_x unique; strict ( 2 ); define behavior for ZI_X alias X persistent table ztab lock master authorization master ( instance ) etag master LocalLastChangedAt { create; update; delete; ... }`
Numbering	`field ( numbering : managed, readonly ) Uuid;`
Action	`action approve result [1] $self;`
Determination	`determination setDefaults on modify { create; }`
Validation	`validation checkDates on save { create; field BeginDate, EndDate; }`
Draft	`with draft;` + `draft table ztab_d`
Projection	`define root view entity ZC_X provider contract transactional_query as projection on ZI_X`
Service	`define service ZUI_X { expose ZC_X as X; }`
Binding	Service Binding → OData V4 – UI / Web API → Publish → Preview
EML	`READ ENTITIES OF`, `MODIFY ENTITIES OF`, `COMMIT ENTITIES`, `IN LOCAL MODE`, `%tky`, `%cid`, `failed`, `reported`, `mapped`

Final interview checklist

- I can write a class with constructor, static & instance members from memory.
- I can explain inheritance vs. interface with a real example, and up-cast vs. down-cast.
- I can describe events with the 4 steps and name where SAP uses them (ALV).
- I can create an exception class with T100 texts and explain the 3 categories.
- I can name and sketch 5 design patterns in ABAP.
- I can write a unit test with a test double.
- I can create a SEGW service with CRUD, filter, paging, navigation, expand, deep insert and function import.
- I know the Gateway transactions and how to debug/analyse errors and performance.
- I can explain CDS-based OData and the RAP layers, managed vs. unmanaged, EML basics.
- I can explain V2 vs. V4, CSRF, ETag, $batch.